Diseño de Sistemas Interactivos

Curso 2025 - 2026

Profesor: José Manuel Velasco

Despacho 309, 3ª planta. Facultad de Informática.

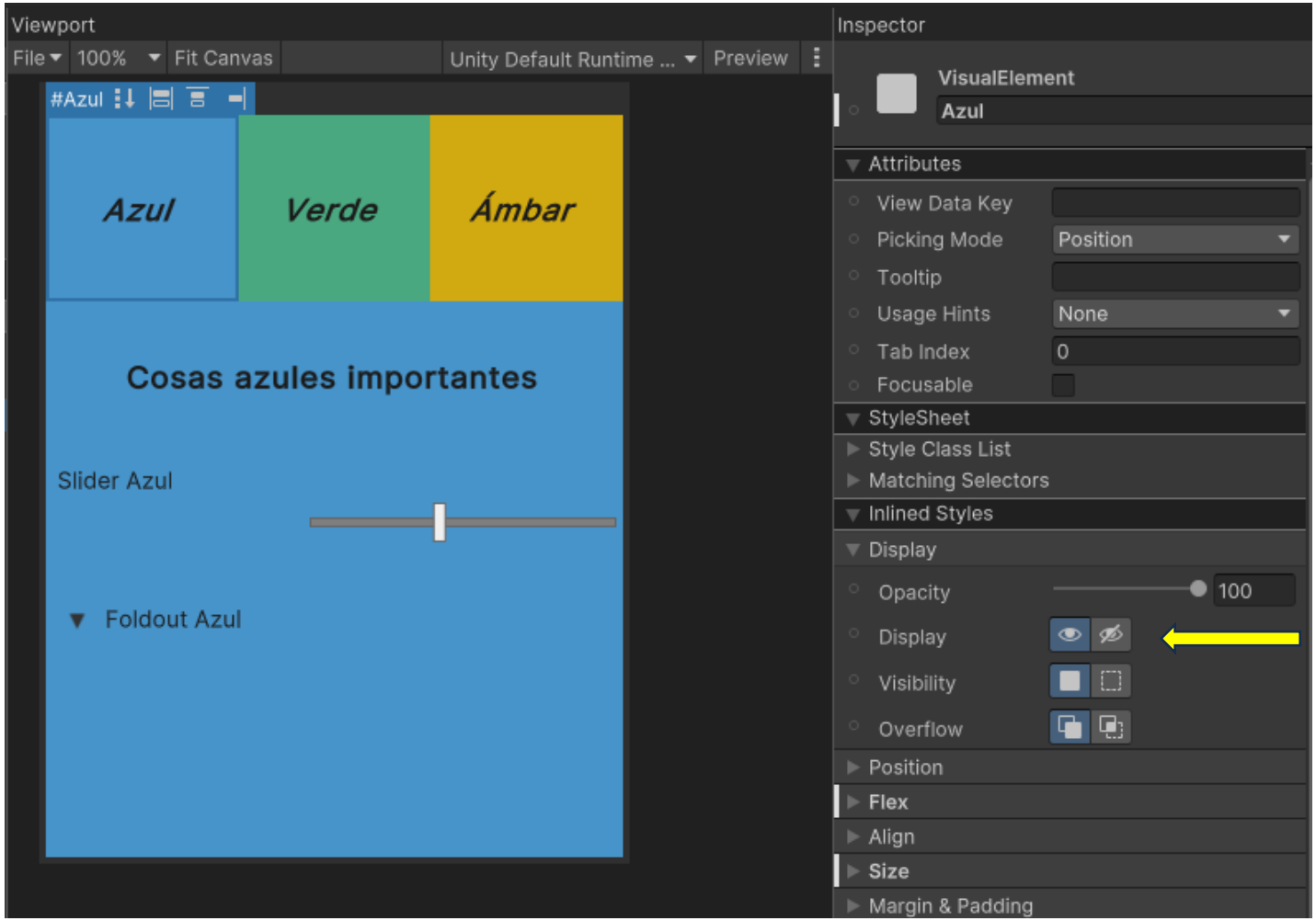
Laboratorio 8: Proyecto final. Proyectos de ejemplo.

Índice

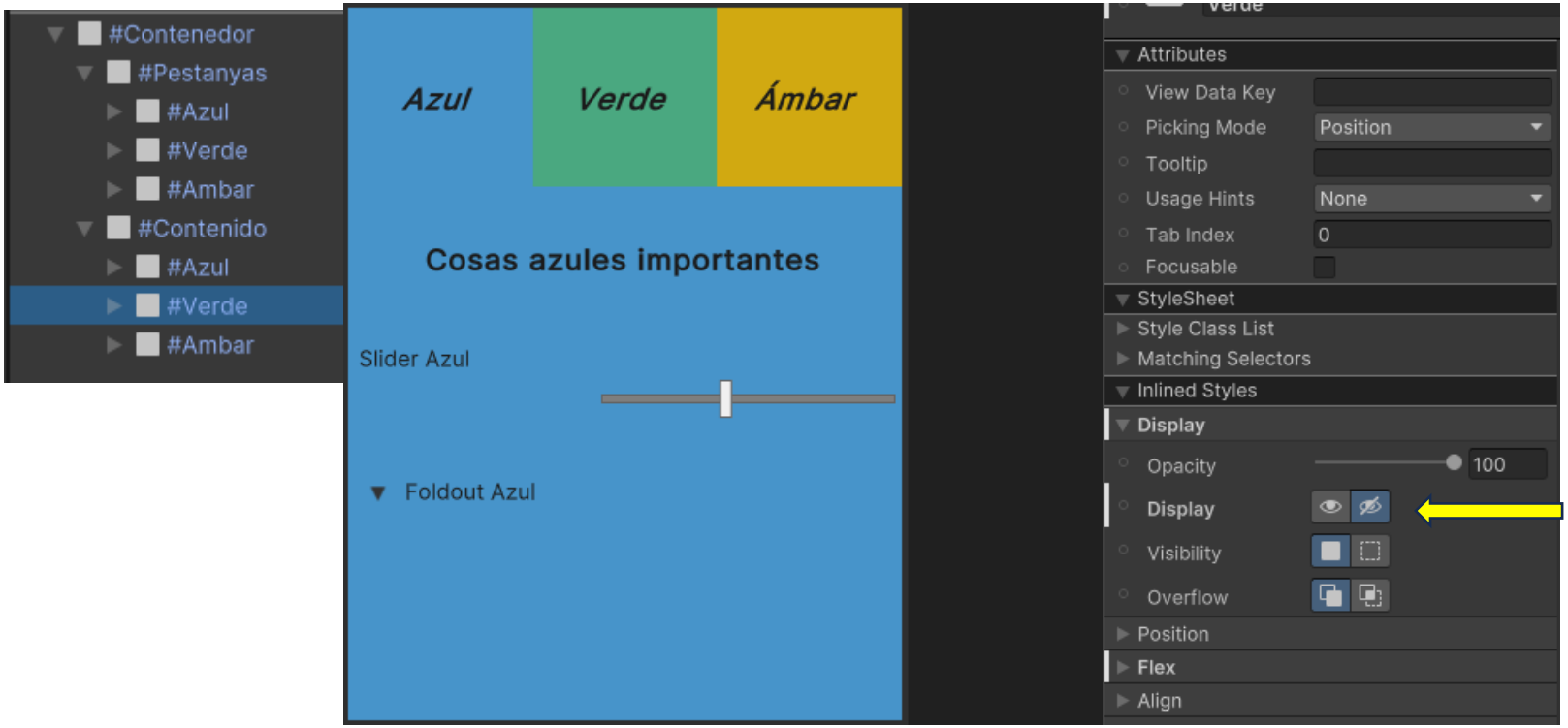
- Laboratorio 8
  - Tabbed Menu
  - Pseudo-classes
  - Texto enriquecido

- Proyecto
- Project Samples
  - Dragon Crashers
    - Custom Controls
      - Tabbed menú
      - Radial progress bar
  - Quizu
    - Demos
      - CustomControls
      - Manipulators
    - Quiz
      - Data
      - Scripts
        - Coroutines
        - ScriptablesObjects
        - Actions

Menú con pestañas muy sencillo



Menú con pestañas muy sencillo



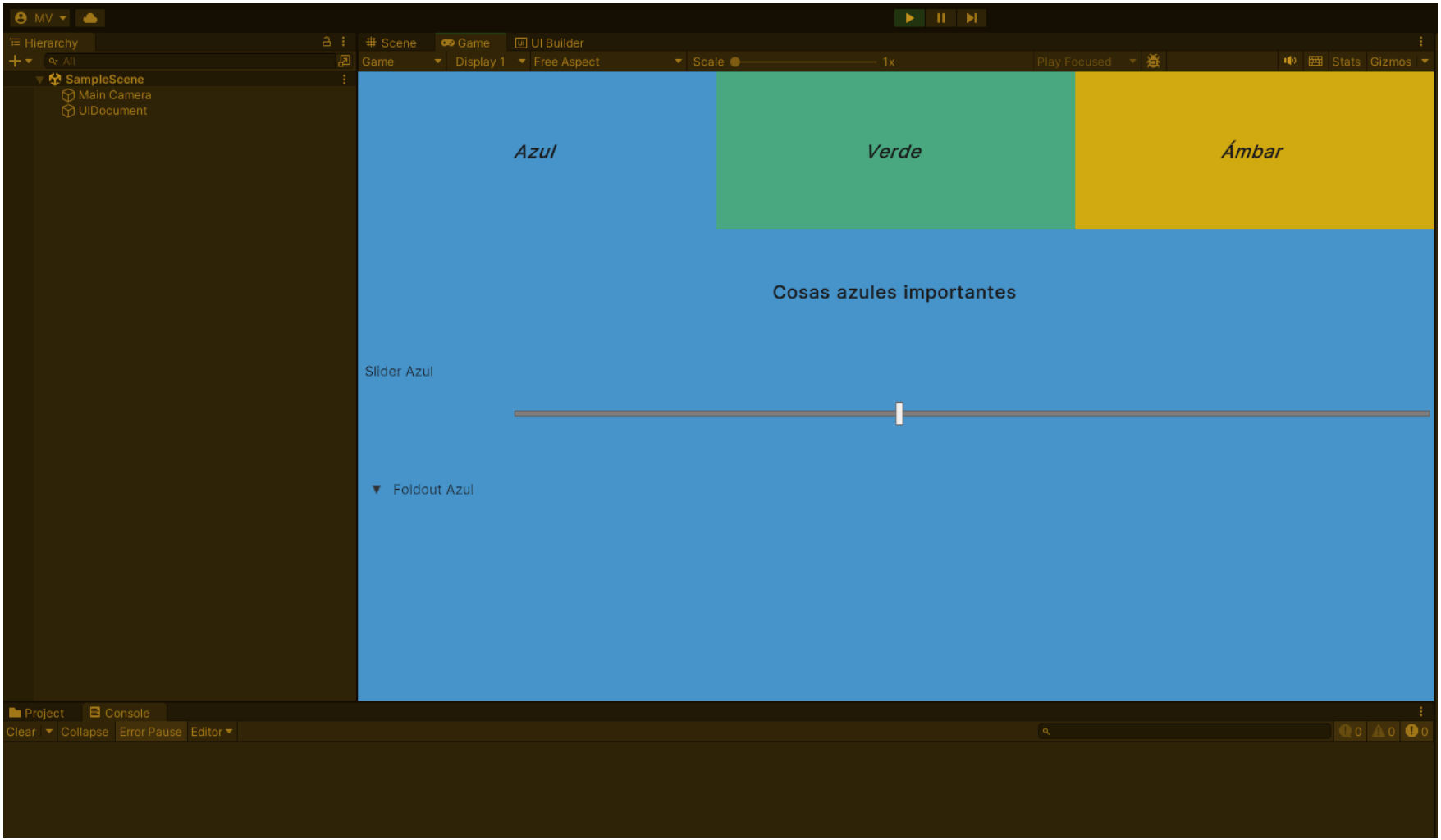
Menú con pestañas

```
1   using UnityEngine;
2   using UnityEngine.UIElements;
3
    0 references
4   public class Lab7 : MonoBehaviour
5   {
            3 references
6           VisualElement contenido_azul;
            3 references
7           VisualElement contenido_verde;
            3 references
8           VisualElement contenido_ambar;
            2 references
9           VisualElement pestanya_azul;
            2 references
10          VisualElement pestanya_verde;
            2 references
11          VisualElement pestanya_ambar;
        3 references
12      private void NoContenido(){
13          contenido_azul.style.display  = DisplayStyle.None;
14          contenido_verde.style.display = DisplayStyle.None;
15          contenido_ambar.style.display = DisplayStyle.None;
16      }
17
```

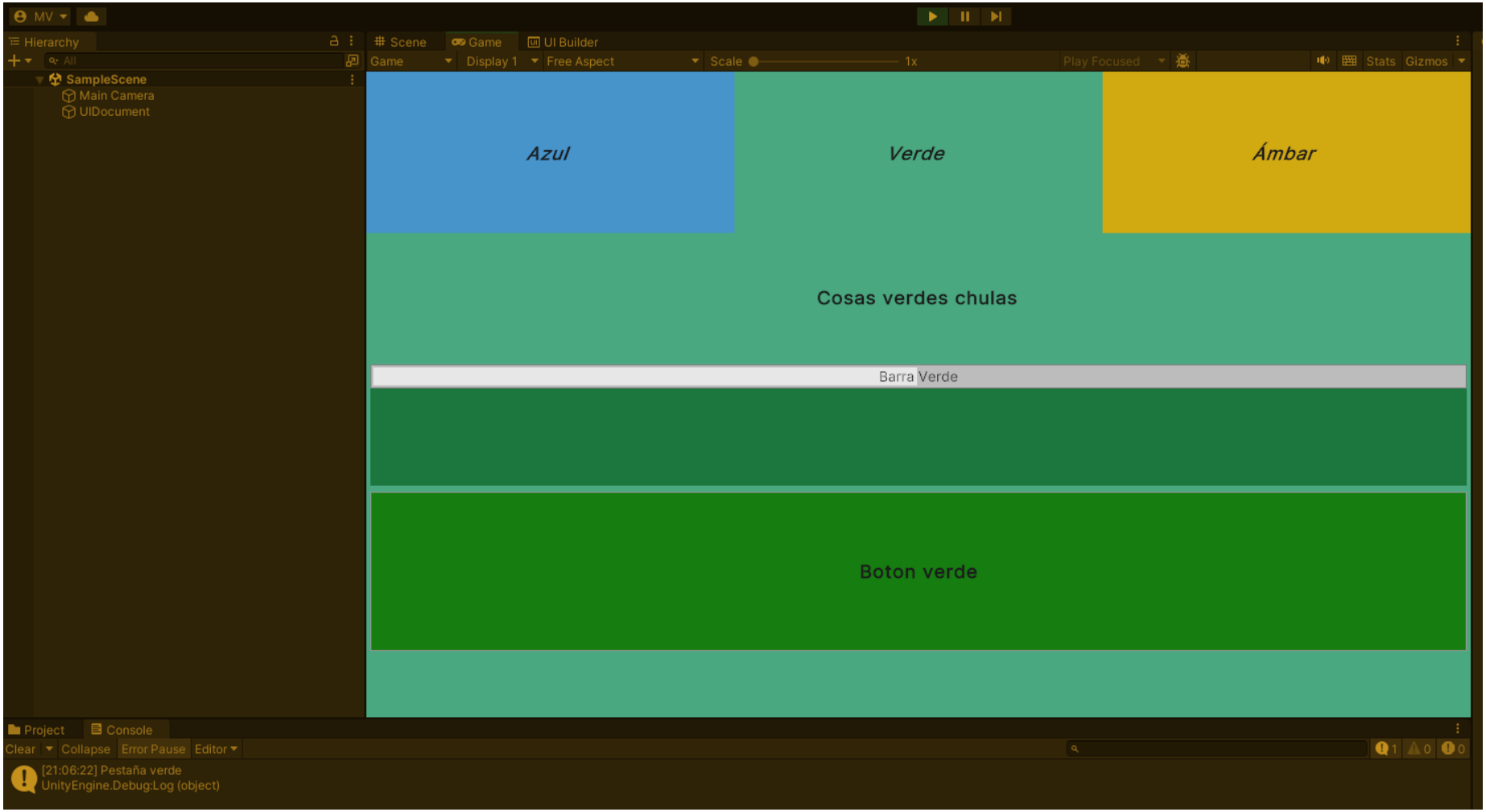
Menú con pestañas

```
0 references
18      private void OnEnable() {
19          UIDocument uidoc     = GetComponent<UIDocument>();
20          VisualElement rootve = uidoc.rootVisualElement;
21
22          VisualElement contenido = rootve.Q("Contenido");
23          VisualElement pestanyas = rootve.Q("Pestanyas");
24
25          pestanya_azul  = pestanyas.Q("Azul");
26          pestanya_verde = pestanyas.Q("Verde");
27          pestanya_ambar = pestanyas.Q("Ambar");
28
29          contenido_azul  = contenido.Q("Azul");
30          contenido_verde = contenido.Q("Verde");
31          contenido_ambar = contenido.Q("Ambar");
32
33
34          pestanya_azul.RegisterCallback<MouseDownEvent>(evt => {
35              Debug.Log("Pestaña azul");
36              NoContenido();
37              contenido_azul.style.display  = DisplayStyle.Flex;
38          });
39
40          pestanya_verde.RegisterCallback<MouseDownEvent>(evt => {
41                      Debug.Log("Pestaña verde");
42              NoContenido();
43              contenido_verde.style.display  = DisplayStyle.Flex;
44          });
45
46          pestanya_ambar.RegisterCallback<MouseDownEvent>(evt => {
47              Debug.Log("Pestaña ambar");
48              NoContenido();
49              contenido_ambar.style.display  = DisplayStyle.Flex;
50          });
51      }
52  }
```

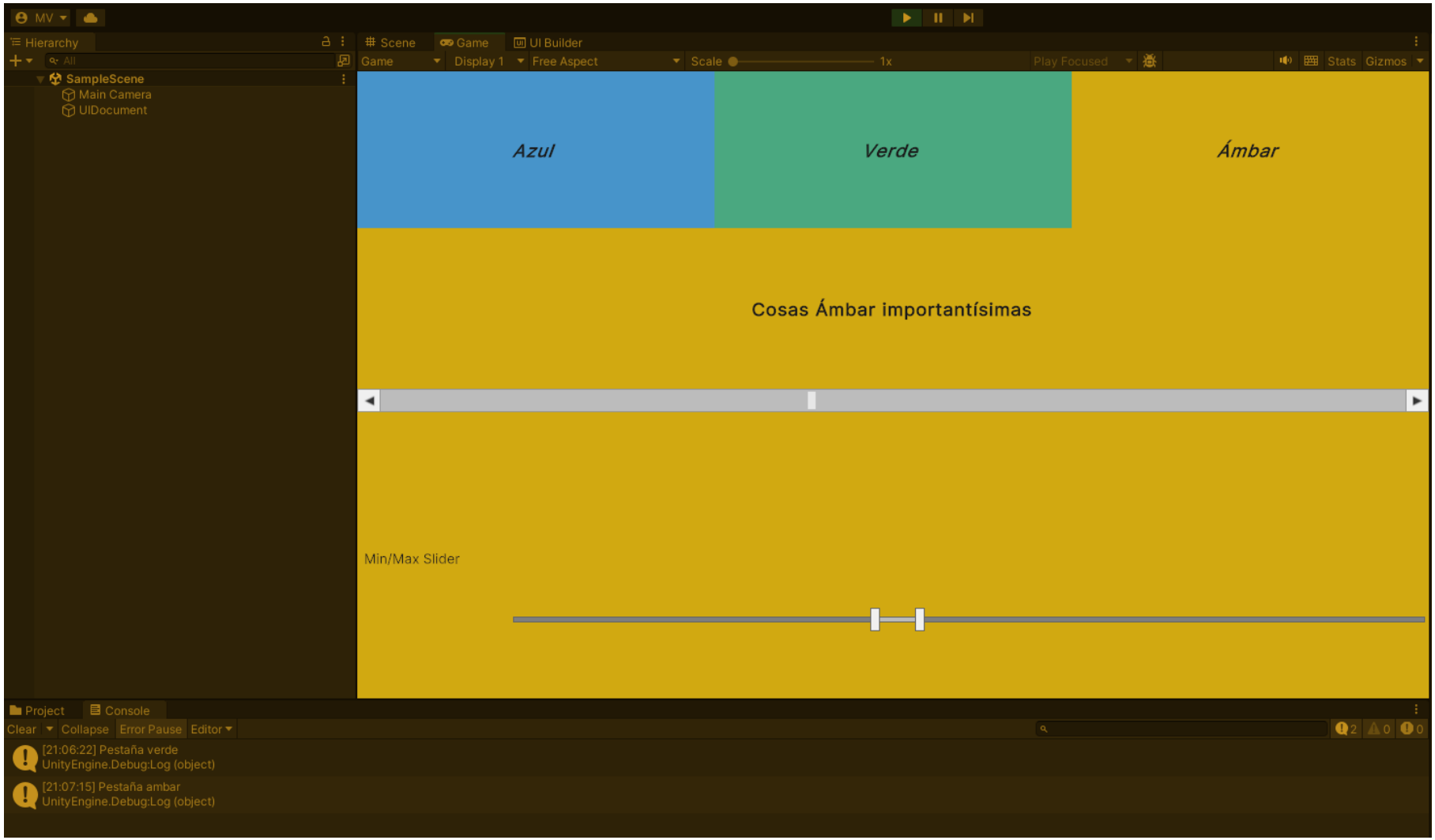
Menú con pestañas



Menú con pestañas

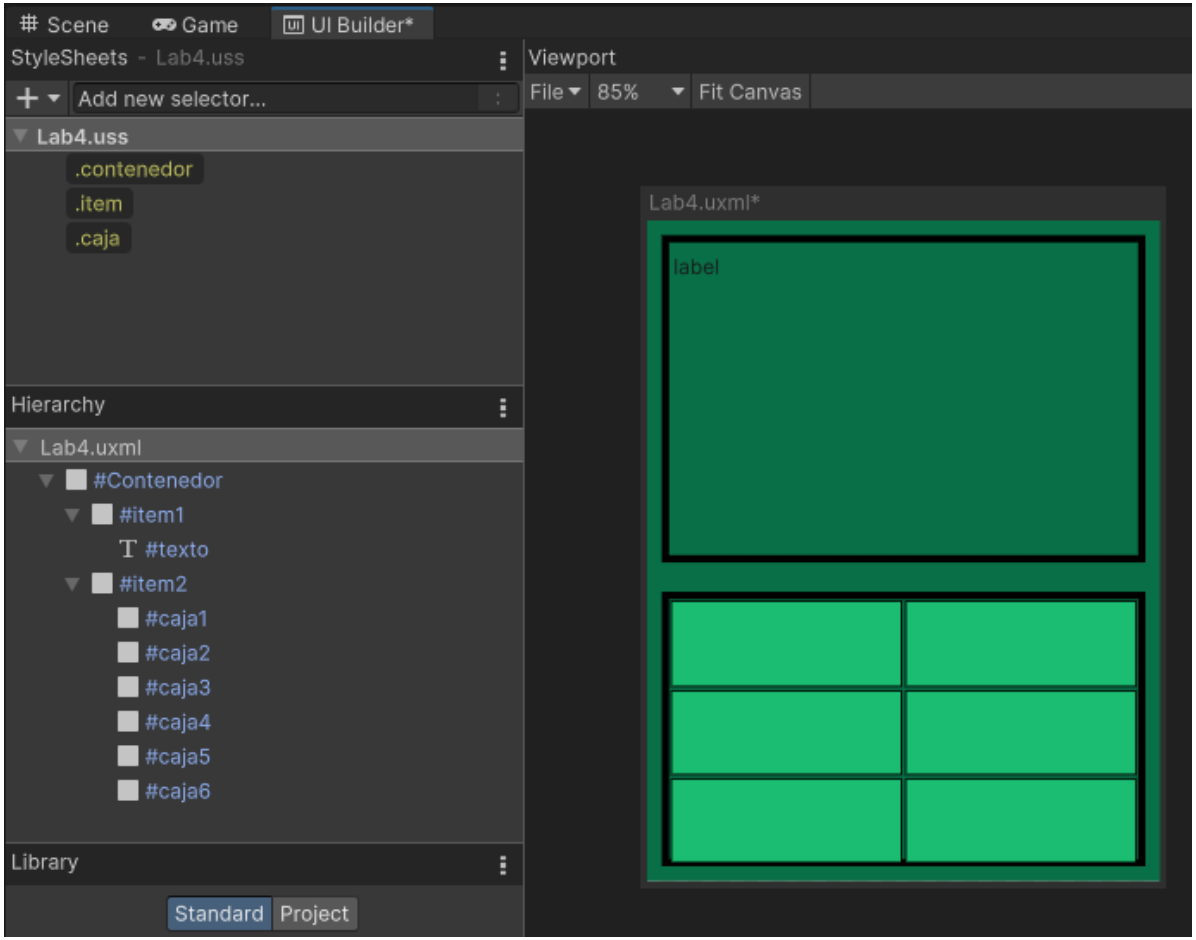


Menú con pestañas

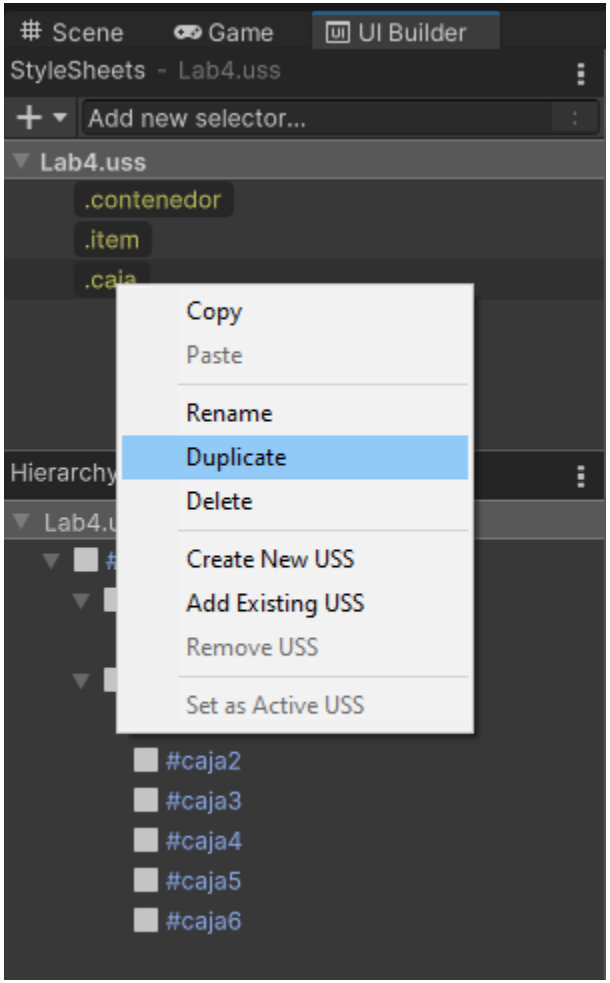


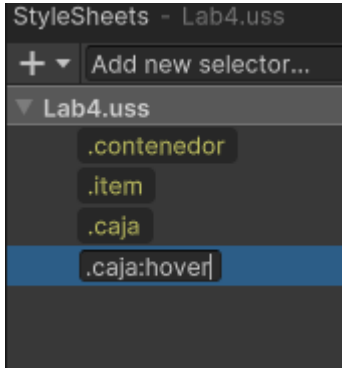
Menú con pestañas

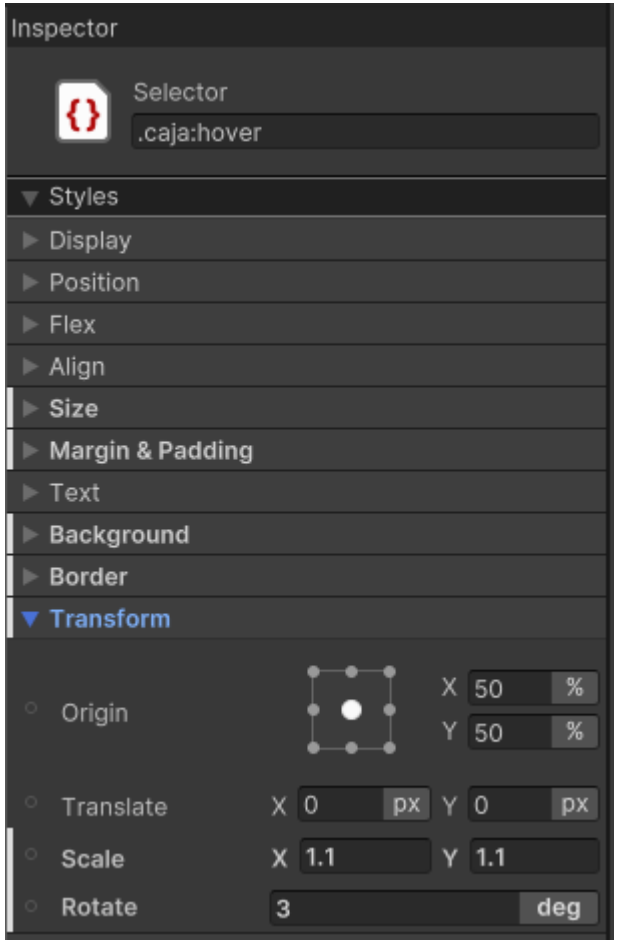
Pseudo Classes



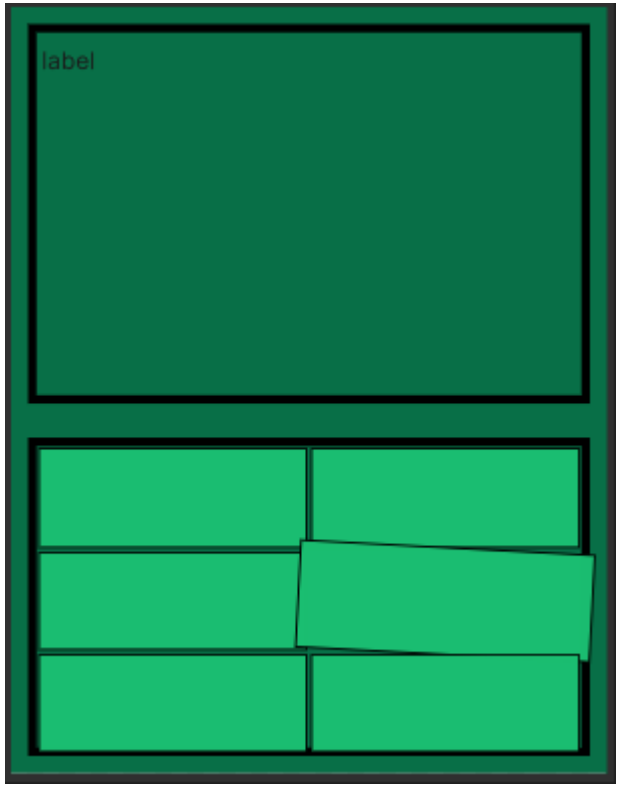
Pseudo Classes

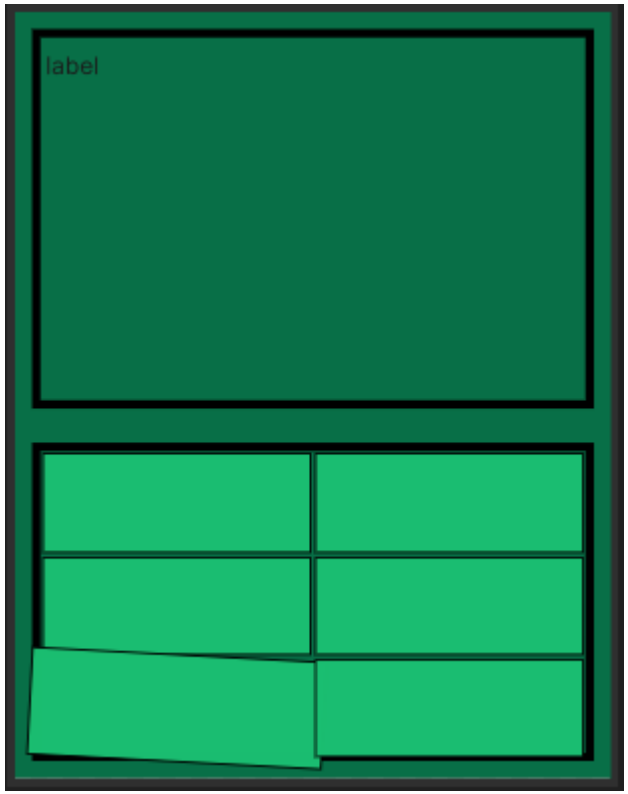




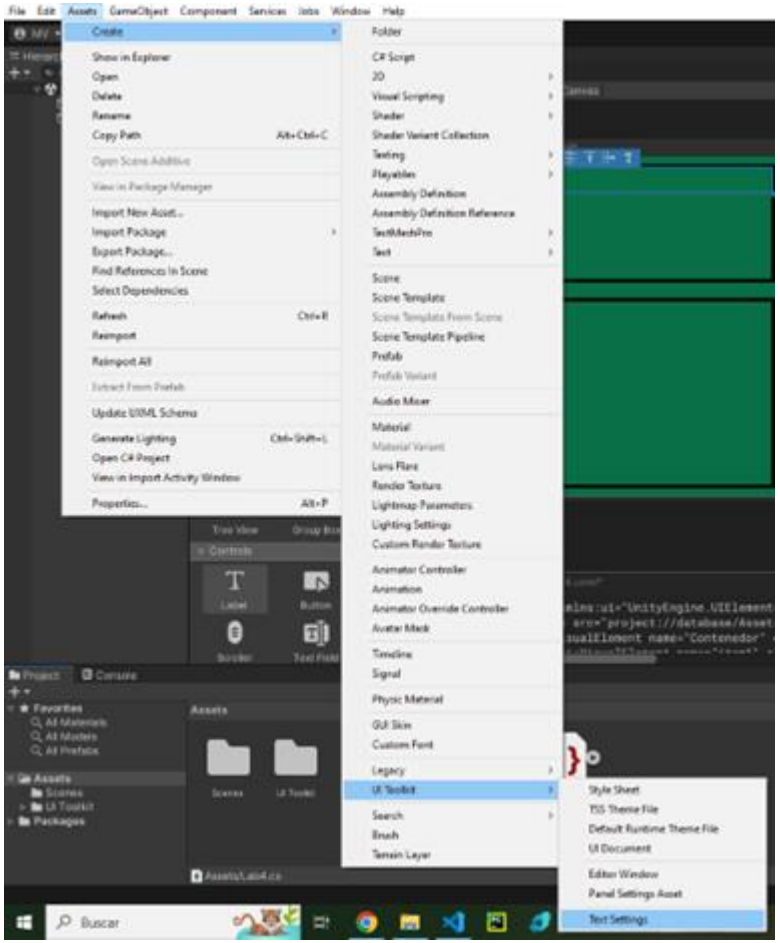


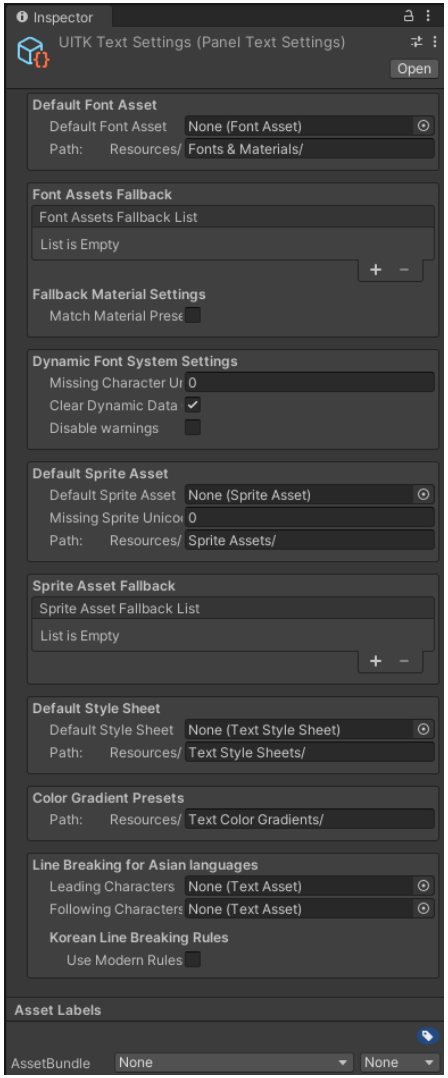
Pseudo Classes





Text


File Edit Assets GameObject Component Services Jobs Window Help
Create
Show in Explorer
Open
Delete
Rename
Copy Path
Import New Asset...
Import Package
Export Package...
Find References In Scene
Select Dependencies
Refresh
Reimport
Reimport All
Update UIML Schema
Generate Lighting
Open C# Project
View in Import Activity Window
Properties...
Folder
C# Script
2D
Visual Scripting
Shader
Shader Variant Collection
Testing
Playables
Assembly Definition
Assembly Definition Reference
TextMeshPro
Text
Scene
Scene Template
Scene Template Pipeline
Prefab
Audio Mixer
Material
Lens Flare
Render Texture
Lightmap Parameters
Lighting Settings
Custom Render Texture
Animator Controller
Animation
Animator Override Controller
Avatar Mask
Timeline
Signal
Physic Material
GUI Skin
Custom Font
Legacy
UI Toolkit
Search
Brush
Terrain Layer
Style Sheet
TSS Theme File
Default Runtime Theme File
UI Document
Editor Window
Panel Settings Asset
Text Settings
Buscar


Inspector
UITK Text Settings (Panel Text Settings)
Open
Default Font Asset
Default Font Asset None (Font Asset)
Path: Resources/ Fonts & Materials/
Font Assets Fallback
Font Assets Fallback List
List is Empty
Fallback Material Settings
Match Material Prese
Dynamic Font System Settings
Missing Character Ur 0
Clear Dynamic Data
Disable warnings
Default Sprite Asset
Default Sprite Asset None (Sprite Asset)
Missing Sprite Unico 0
Path: Resources/ Sprite Assets/
Sprite Asset Fallback
Sprite Asset Fallback List
List is Empty
Default Style Sheet
Default Style Sheet None (Text Style Sheet)
Path: Resources/ Text Style Sheets/
Color Gradient Presets
Path: Resources/ Text Color Gradients/
Line Breaking for Asian languages
Leading Characters None (Text Asset)
Following Characters None (Text Asset)
Korean Line Breaking Rules
Use Modern Rules
Asset Labels
AssetBundle None None

Texto enriquecido

Unity Documentation | Manual | Scripting API | Search manual... | unity.com

Version: 2022.3 | Language : English

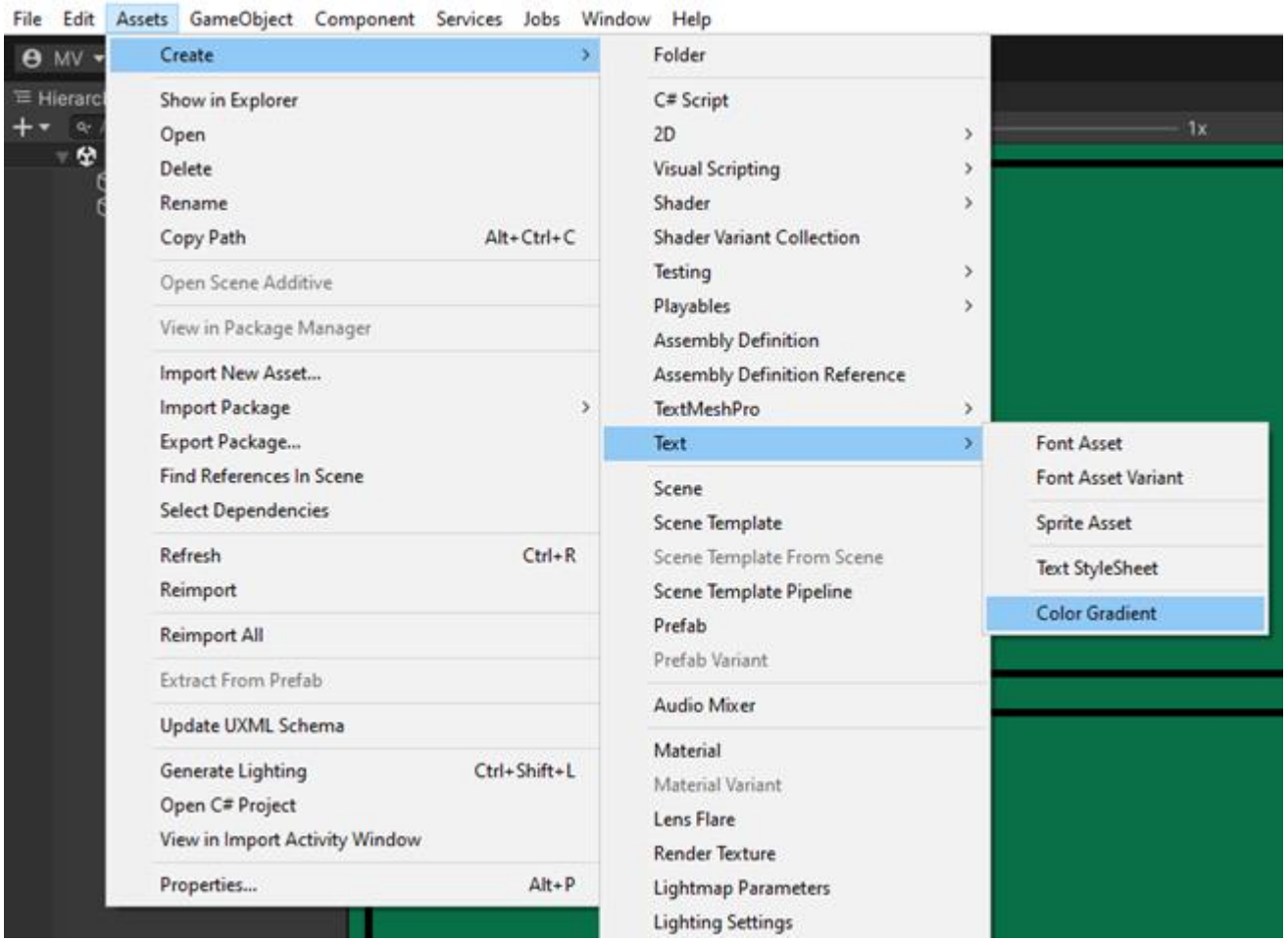
- UI Builder
- Structure UI
- Style UI
- UI Toolkit Debugger
- Control behavior with events
- UI Renderer
- Support for Editor UI
- Support for runtime UI
- Work with text
  - Get started with text
  - Style text with USS
  - Style text with rich text tags
  - Supported rich text tags
  - Font assets
  - Style sheets
  - Include sprites in text
  - Color gradients
  - Panel Text Settings assets
  - Fallback font
- Examples
- Migration guides
- Unity UI
- Immediate Mode GUI (IMGUI)
- Unity Services
- XR
- Unity's Asset Store
- Platform development
- Unity Search
- Glossary

Supported rich text tags

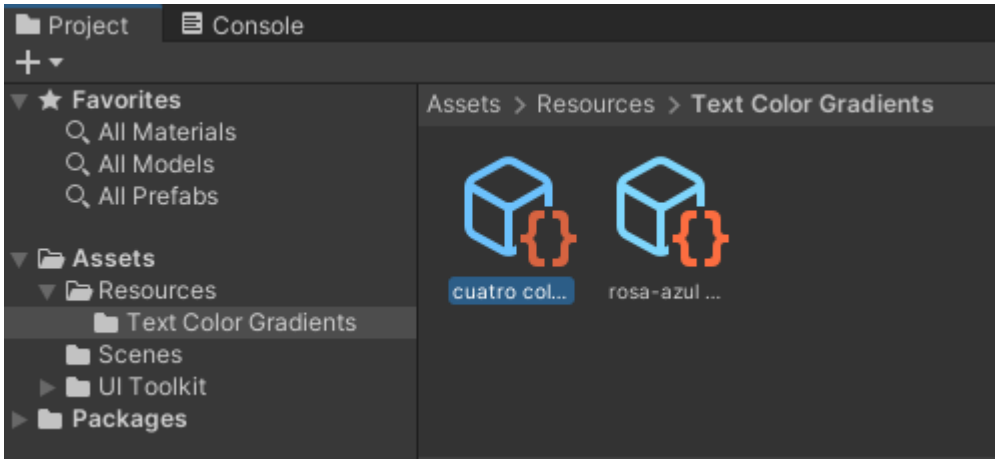
The following table lists all the supported rich text tags:

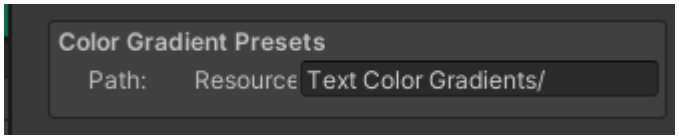
Tag	Description	Example
`<a>`	Define a hyperlink. Use the `href` attribute to define the hyperlink URL.	`<a href="https://www.unity.com">Visit Unity!</a>`
`<align>`	Change the text's horizontal alignment. The supported values are: • `left` • `center` • `right` • `justified` • `flush` If you put multiple alignment tags on the same line, the last one overrides the others.	`<align="left">Left-aligned</align>`
`<allcaps>`	Convert text to uppercase.	`<allcaps>Alice and Bob watched TV.</allcaps>`
`<alpha>`	Change text opacity. It works with hexadecimal values.	`<alpha=#FF>FF <alpha=#CC>CC <alpha=#AA>AA <alpha=#88>88 <alpha=#66>66 <alpha=#44>44 <alpha=#22>22 <alpha=#00>00`
`<b>`	Render text in boldface.	`The fox jumps over the <b>lazy dog</b>`
` `	Forces a line break in text.	`Break the line here   New line starts`
`<color>`	Change text color or color and opacity. It supports color names and hexadecimal values. If you apply successive tags in the same text, the last one takes precedence over the others until you either add another tag or use a closing tag to end the current color's scope.	`<color="red">Red <color=#005500>Dark Green <#0000FF>Blue <color=#FF000088>Semitransparent Red`
`<cspace>`	Change spacing between characters, either absolute or relative to the original font Asset. Use pixels or font units. Positive adjustments push the characters apart, negative adjustments pull them together.	`<cspace=1em>Spacing</cspace> is just as important as <cspace=-0.5em>timing.`
`<font>`	Change text font.	`Would you like <font="Impact SDF">a different font?</font>`
`<font-weight>`	Change the text's font weight to any of the weights defined in the font Asset. If you haven't defined any font weights, you can still use `400` for normal, `700` for bold.	`<font-weight="100">Thin</font-weight>`

Gradientes de color

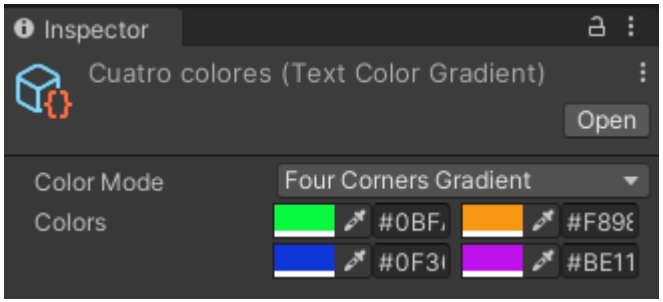


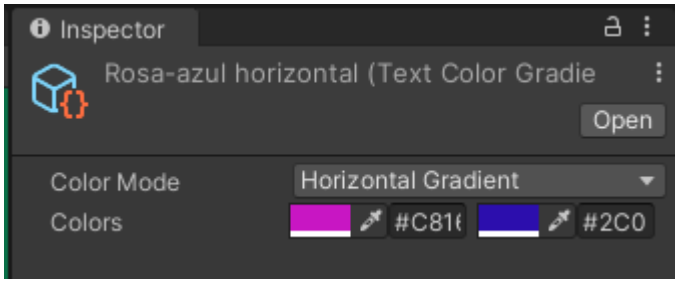
Gradientes de color





¡Importante! → Mantener el path.

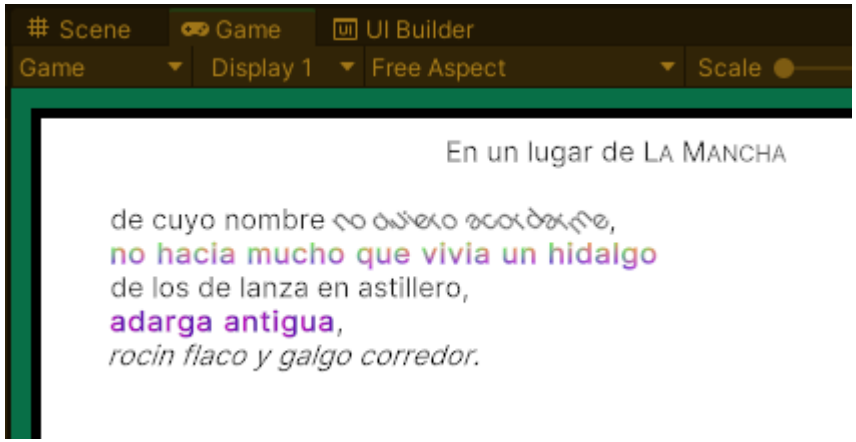




Texto enriquecido

```
Label texto = rootve.Q<Label>("texto");
item1.style.backgroundColor = Color.white;
texto.text = @"<line-indent=15%>En un lugar de <smallcaps>La Mancha</smallcaps> </line-indent><br>
de cuyo nombre <rotate=""45"">no quiero acordarme</rotate>,
<b><gradient=""cuatro colores"">no hacia mucho que vivia un hidalgo</gradient></b>
de los de lanza en astillero,
<b><color=""black""><gradient=""rosa-azul horizontal"">adarga antigua</gradient></b>,
<i>rocin flaco y galgo corredor.";
```

¡Importante! → Los nombres tienen que coincidir exactamente



Laboratorio 8

- En este laboratorio practicaremos:
  - A) Crear un menú con pestañas con múltiples elementos, controles, texto enriquecido y pseudo-classes.
  - B) Modificaremos el menú con pestañas para crear un sistema de menús independientes.

- Al campus subimos los ficheros .uxml, .uss y .cs.

Índice

- Laboratorio 8
  - Tabbed Menu
  - Pseudo-classes
  - Texto enriquecido

- Proyecto
- Project Samples
  - Dragon Crashers
    - Custom Controls
      - Tabbed menú
      - Radial progress bar
  - Quizu
    - Demos
      - CustomControls
      - Manipulators
    - Quiz
      - Data
      - Scripts
        - Coroutines
        - ScriptablesObjects
        - Actions

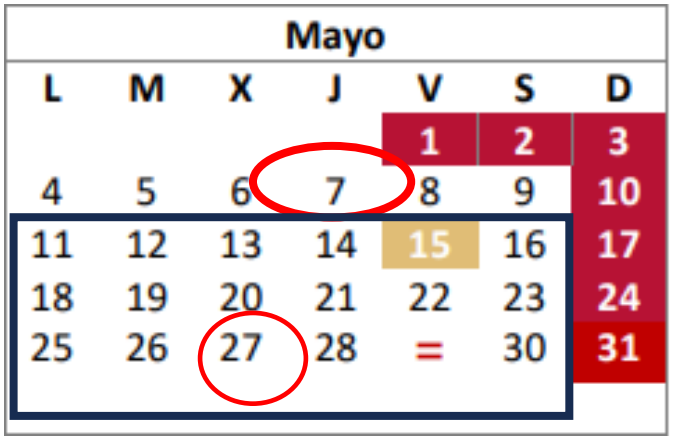
Proyecto

- El proyecto se hace individual o en grupos de 2-3 personas.
- Tiene que recoger más o menos todo lo que hemos ido viendo en las prácticas.
  - UI Toolkit: Uxml, USS, flexbox, classes, pseudo-classes.
  - Uquery
  - Eventos
  - Templates
  - Custom Controls
  - Manipulators
  - Data Binding
    - JSON

Proyecto

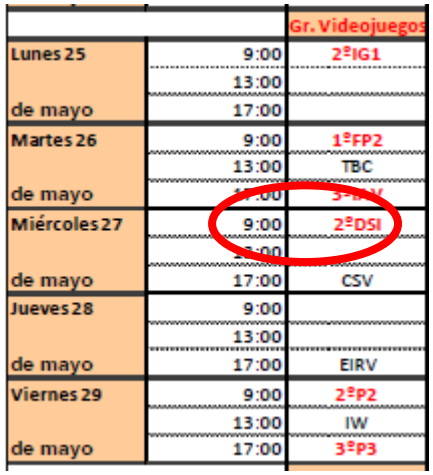
- El proyecto se hace individual o en grupos de 2-3 personas.
- Tiene que recoger más o menos todo lo que hemos ido viendo en las prácticas.
- Crearos una cuenta en Github (si no la tenéis ya) para guardar el proyecto completo.
- En el campus virtual hay dos enlaces:
  - 1. PDF con el prototipo en Figma.
  - 2. Fichero de texto con el enlace a github donde está el proyecto.

Último día presentación → Proyecto y prácticas



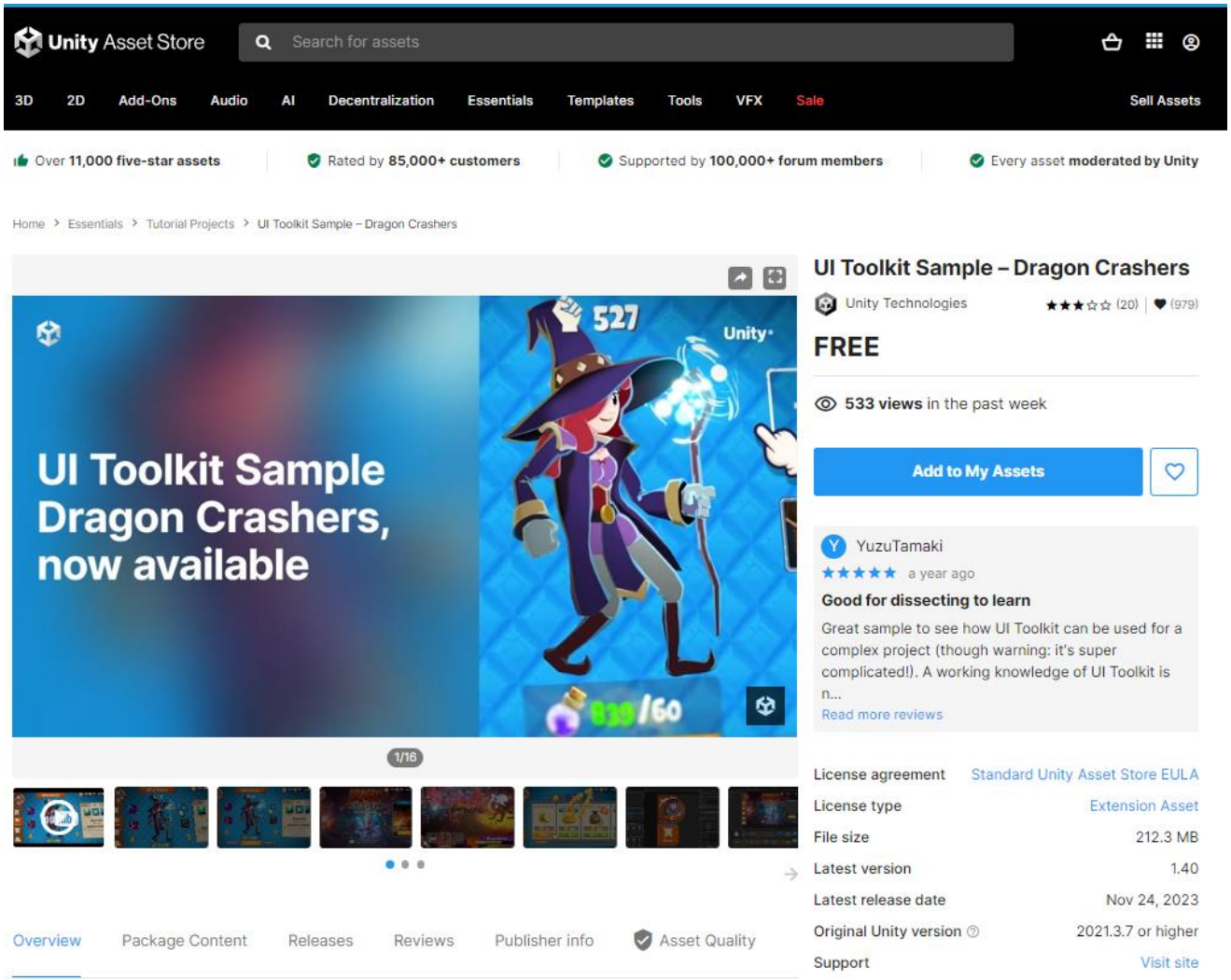
Mayo						
L	M	X	J	V	S	D
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	=	30	31

Examen

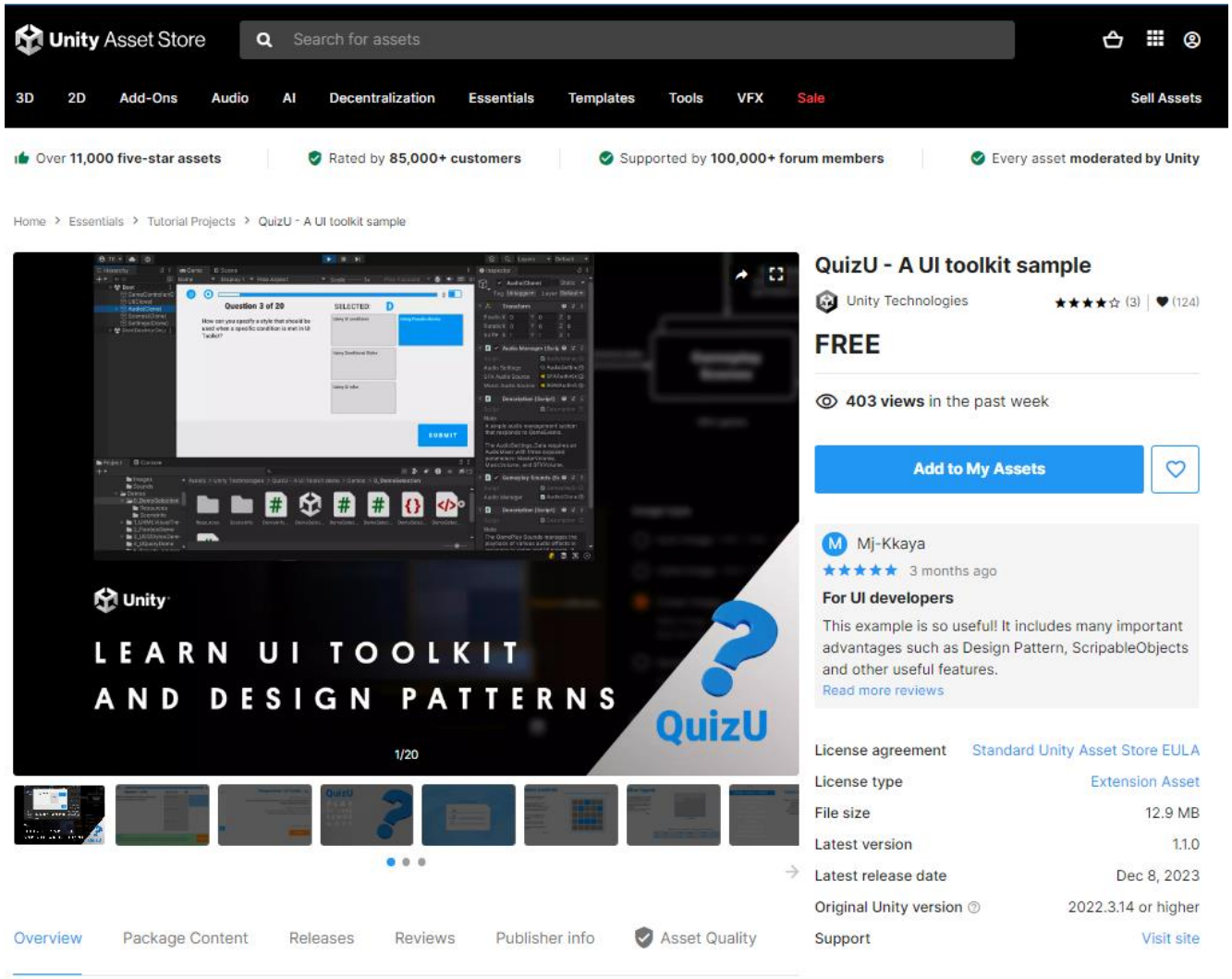


		Gr. Videojuegos
Lunes 25	9:00	2ºIG1
	13:00	
de mayo	17:00	
Martes 26	9:00	1ºFP2
	13:00	TBC
de mayo	[illegible]	[illegible]
Miércoles 27	9:00	2ºDSI
	[illegible]	
de mayo	17:00	CSV
Jueves 28	9:00	
	13:00	
de mayo	17:00	EIRV
Viernes 29	9:00	2ºP2
	13:00	IW
de mayo	17:00	3ºP3

Dragon Crashers



QuizU


Unity Asset Store
Search for assets
3D
2D
Add-Ons
Audio
AI
Decentralization
Essentials
Templates
Tools
VFX
Sale
Sell Assets
Over 11,000 five-star assets
Rated by 85,000+ customers
Supported by 100,000+ forum members
Every asset moderated by Unity
Home > Essentials > Tutorial Projects > QuizU - A UI toolkit sample
Unity
LEARN UI TOOLKIT
AND DESIGN PATTERNS
QuizU
1/20
QuizU - A UI toolkit sample
Unity Technologies
(3)
(124)
FREE
403 views in the past week
Add to My Assets
Mj-Kkaya
3 months ago
For UI developers
This example is so useful! It includes many important advantages such as Design Pattern, ScripableObjects and other useful features.
Read more reviews
License agreement
Standard Unity Asset Store EULA
License type
Extension Asset
File size
12.9 MB
Latest version
1.1.0
Latest release date
Dec 8, 2023
Original Unity version
2022.3.14 or higher
Support
Visit site
Overview
Package Content
Releases
Reviews
Publisher info
Asset Quality

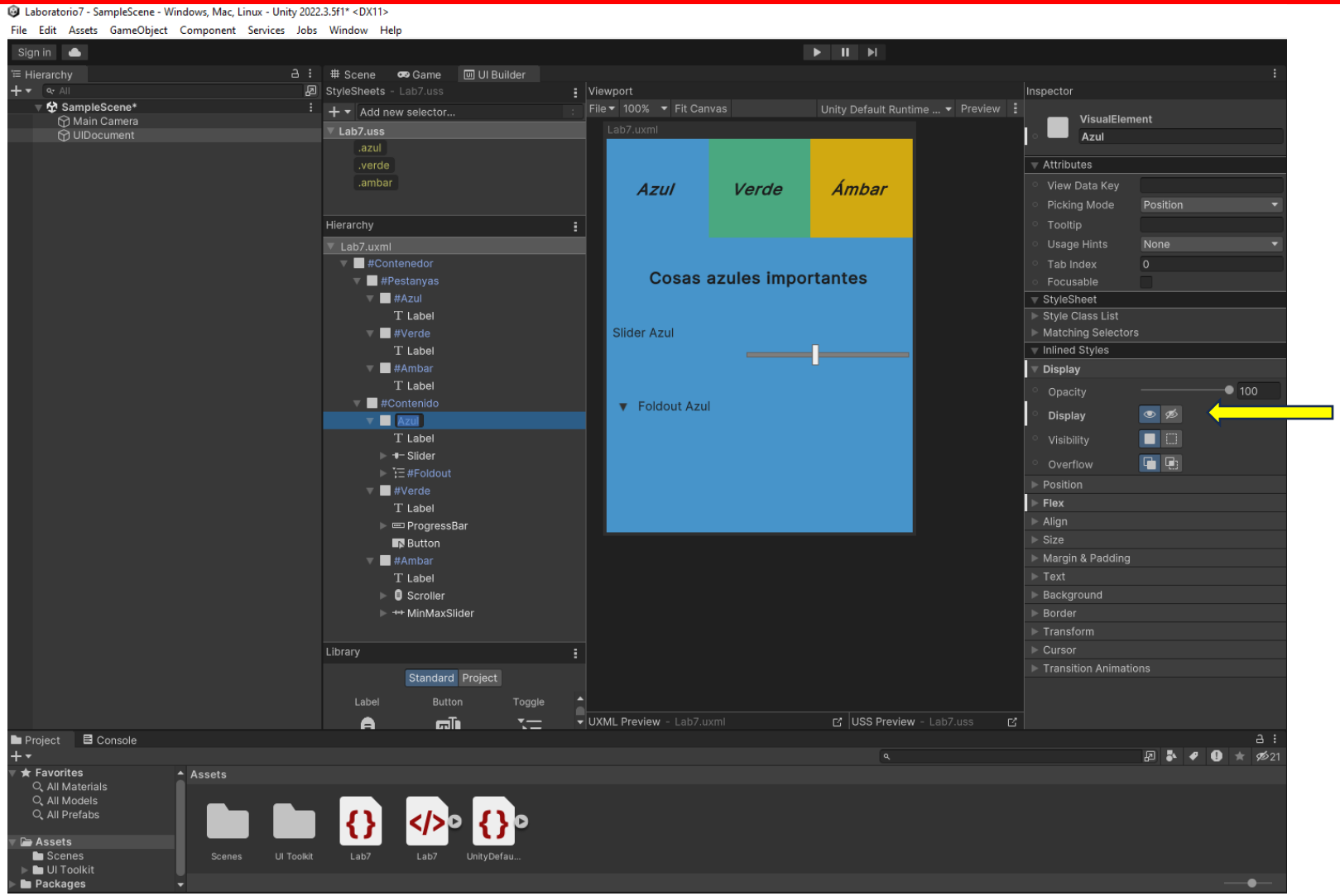
Índice

- Proyecto

- **Project Samples**
  - **Dragon Crashers**
    - **Custom Controls**
      - **Tabbed menú**
      - **Radial progress bar**
  - Quizu
    - Demos
      - CustomControls
      - Manipulators
    - Quiz
      - Data
      - Scripts
        - Coroutines
        - ScriptablesObjects
        - Actions

Dragon Crashers → Tabbed menú

Menú con pestañas muy sencillo



Menú con pestañas

Lab7.cs

Assets > Lab7.cs > Lab7 > OnEnable

```
using UnityEngine;
using UnityEngine.UIElements;

0 references
public class Lab7 : MonoBehaviour
{
        3 references
        VisualElement contenido_azul;
        3 references
        VisualElement contenido_verde;
        3 references
        VisualElement contenido_ambar;
        2 references
        VisualElement pestanya_azul;
        2 references
        VisualElement pestanya_verde;
        2 references
        VisualElement pestanya_ambar;
    3 references
    private void NoContenido(){
        contenido_azul.style.display  = DisplayStyle.None;
        contenido_verde.style.display = DisplayStyle.None;
        contenido_ambar.style.display = DisplayStyle.None;
    }

```

Menú con pestañas

```
0 references
18      private void OnEnable() {
19          UIDocument uidoc     = GetComponent<UIDocument>();
20          VisualElement rootve = uidoc.rootVisualElement;
21
22          VisualElement contenido = rootve.Q("Contenido");
23          VisualElement pestanyas = rootve.Q("Pestanyas");
24
25          pestanya_azul  = pestanyas.Q("Azul");
26          pestanya_verde = pestanyas.Q("Verde");
27          pestanya_ambar = pestanyas.Q("Ambar");
28
29          contenido_azul  = contenido.Q("Azul");
30          contenido_verde = contenido.Q("Verde");
31          contenido_ambar = contenido.Q("Ambar");
32
33
34          pestanya_azul.RegisterCallback<MouseDownEvent>(evt => {
35              Debug.Log("Pestaña azul");
36              NoContenido();
37              contenido_azul.style.display  = DisplayStyle.Flex;
38          });
39
40          pestanya_verde.RegisterCallback<MouseDownEvent>(evt => {
41                      Debug.Log("Pestaña verde");
42              NoContenido();
43              contenido_verde.style.display  = DisplayStyle.Flex;
44          });
45
46          pestanya_ambar.RegisterCallback<MouseDownEvent>(evt => {
47              Debug.Log("Pestaña ambar");
48              NoContenido();
49              contenido_ambar.style.display  = DisplayStyle.Flex;
50          });
51      }
52  }
```

Dragon Crashers → TabbedMenu (campus virtual)

TabbedMenuController.cs | TabbedMenu.cs ×

E: > PROYECTOS > UNITY > sample2 > Assets > Scripts > UI > Components > TabbedMenu.cs > ...

```
using UnityEngine;
using UnityEngine.UIElements;

namespace UIToolkitDemo
{
    // adapted from: https://docs.unity3d.com/2021.2/Documentation/Manual/UIE-create-tabbed-menu-for-runtime.html

    // This establishes a tabbed menu system for the UI document. This modifies the original to support multiple
    // tabbed menus within the same document.

    [System.Serializable]
    1 reference
    public struct TabbedMenuIDs
    {
        // UXML selector for a clickable tab
        2 references
        public string tabClassName;// = "tab";

        // UXML selector for currently selected tab
        2 references
        public string selectedTabClassName; //= "selected-tab";

        // UXML selector for content to hide
        2 references
        public string unselectedContentClassName; // = "unselected-content";

        // use a basename to pair a tab with its content, e.g. 'name1-tab' matches 'name1-content'

        // suffix naming convention for tabs
        2 references
        public string tabNameSuffix;// = "-tab";

        // suffix naming convention for content
        2 references
        public string contentNameSuffix;// = "-content";

    }
    0 references
```

Dragon Crashers → TabbedMenu (campus virtual)

```
32      public class TabbedMenu : MonoBehaviour
            0 references
47          private void OnEnable()
48          {
49              if (m_Document == null)
50                  m_Document = GetComponent<UIDocument>();
51
52              VisualElement root = m_Document.rootVisualElement;
53              m_MenuElement = root.Q(m_MenuElementName);
54
55              // create a new TabbedMenuController for a specific element (fall back to the entire tree if unspecified)
56              m_Controller = (string.IsNullOrEmpty(m_MenuElementName) || m_MenuElement == null) ?
57                  new TabbedMenuController(root, m_TabbedMenuStrings) : new TabbedMenuController(m_MenuElement, m_TabbedMenuStrings);
58
59              // set up the click events on the tab
60              m_Controller.RegisterTabCallbacks();
61          }
62
63          // fill in default names for convenience - make these unique for each tabbed menu/UI
            0 references
64          void OnValidate()
65          {
66              if (string.IsNullOrEmpty(m_TabbedMenuStrings.tabClassName))
67              {
68                  m_TabbedMenuStrings.tabClassName = "tab";
69              }
70
71              if (string.IsNullOrEmpty(m_TabbedMenuStrings.selectedTabClassName))
72              {
73                  m_TabbedMenuStrings.selectedTabClassName = "selected-tab";
74              }
75
76              if (string.IsNullOrEmpty(m_TabbedMenuStrings.unselectedContentClassName))
77              {
78                  m_TabbedMenuStrings.unselectedContentClassName = "unselected-content";
79              }
80
81              if (string.IsNullOrEmpty(m_TabbedMenuStrings.tabNameSuffix))
82              {
83                  m_TabbedMenuStrings.tabNameSuffix = "-tab";
84              }
85
86              if (string.IsNullOrEmpty(m_TabbedMenuStrings.contentNameSuffix))
87              {
88                  m_TabbedMenuStrings.contentNameSuffix = "-content";
89              }
90          }
```

Dragon Crashers → TabbedMenuController (campus virtual)

TabbedMenuController.cs ×

E: > PROYECTOS > UNITY > sample2 > Assets > Scripts > UI > Controllers > TabbedMenuController.cs > TabbedMenuController

```
using UnityEngine;
using UnityEngine.UIElements;
using System;

namespace UIToolkitDemo
{

    // adapted from: https://docs.unity3d.com/2021.2/Documentation/Manual/UIE-create-tabbed-menu-for-runtime.html

    1 reference
    public class TabbedMenuController
    {
        // event to notify other objects
        1 reference
        public static event Action TabSelected;

        // base VisualElement of the UI (e.g. MailScreen, CharScreen, ShopScreen)
        5 references
        readonly VisualElement m_Root;

        // strings to query for VisualElements
        12 references
        readonly TabbedMenuIDs m_IDs;

        // initialize the root Visual Element for reuse (constructor for non-Monobehaviour)
        0 references
        public TabbedMenuController(VisualElement root, TabbedMenuIDs ids)
        {
            m_Root = root;
            m_IDs = ids;
        }

        // set up click events for tab buttons
        0 references
        public void RegisterTabCallbacks()
        {
            // identify each tab
            UQueryBuilder<VisualElement> tabs = GetAllTabs();

            // register the ClickTab event handler for each Visual Element
            tabs.ForEach(
                (t) =>
                {
                    t.RegisterCallback<ClickEvent>(OnTabClick);
                });
        }
```

```
        // locate all VisualElements that have the tab class name
        4 references
        UQueryBuilder<VisualElement> GetAllTabs()
        {
            return m_Root.Query<VisualElement>(className: m_IDs.tabClassName);
        }
```

Dragon Crashers → TabbedMenuController

```
41
42      // process a click
        1 reference
43      void OnTabClick(ClickEvent evt)
44      {
45          VisualElement clickedTab = evt.currentTarget as VisualElement;
46
47          // if the clicked tab is not already selected, select the correct one
48          if (!IsTabSelected(clickedTab))
49          {
50              // de-select any other tabs that are currently active
51              GetAllTabs().Where(
52                  (tab) => tab != clickedTab && IsTabSelected(tab)
53                  ).ForEach(UnselectTab);
54
55              // select the clicked tab
56              SelectTab(clickedTab);
57              AudioManager.PlayDefaultButtonSound();
58          }
59      }
```

Nuestro Custom Control

- Ejemplos:
  - Defensa 3:

  - Ataque 2:

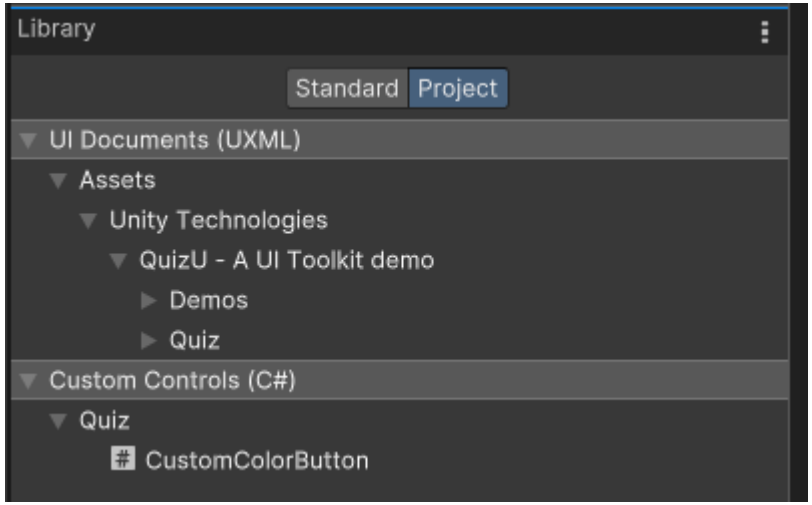
Dragon Crashers → radial progress bar

Índice

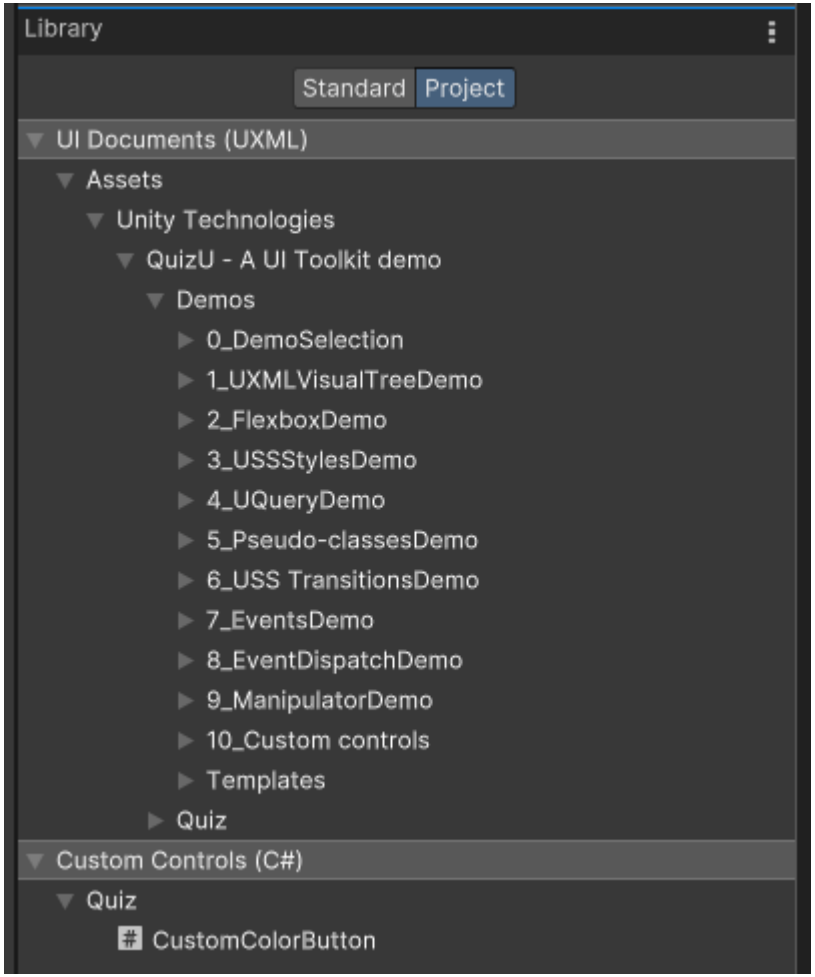
- Proyecto

- Project Samples
  - Dragon Crashers
    - Custom Controls
      - Tabbed menú
      - Radial progress bar
  - **Quizu**
    - **Demos**
      - **CustomControls**
      - **Manipulators**
    - Quiz
      - Data
      - Scripts
        - Coroutines
        - ScriptablesObjects
        - Actions

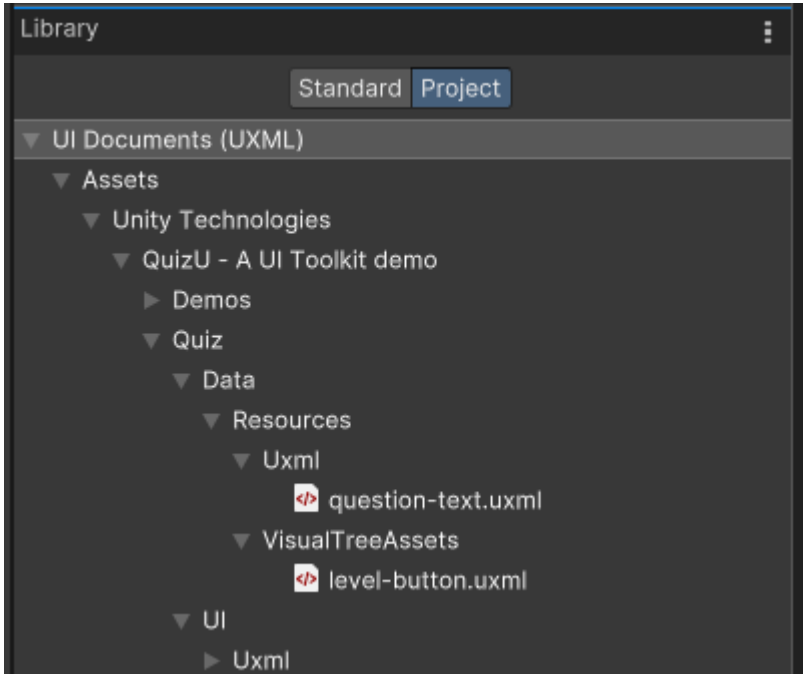
QuizU

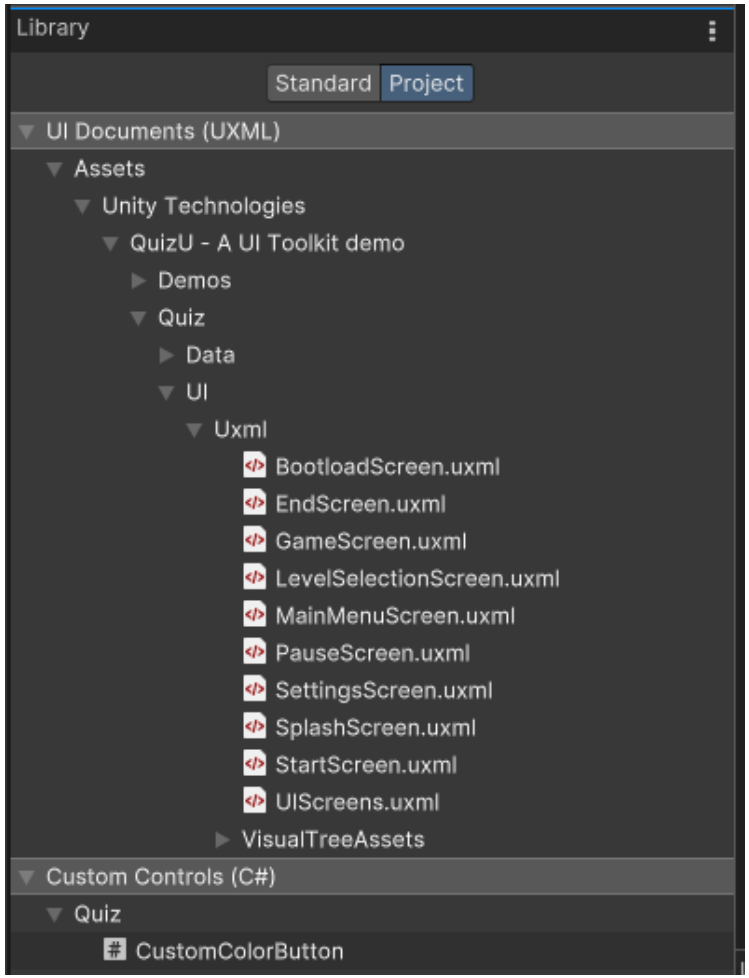

Library
Standard
Project
UI Documents (UXML)
Assets
Unity Technologies
QuizU - A UI Toolkit demo
Demos
Quiz
Custom Controls (C#)
Quiz
CustomColorButton

QuizU - Demos


Library
Standard
Project
UI Documents (UXML)
Assets
Unity Technologies
QuizU - A UI Toolkit demo
Demos
0_DemoSelection
1_UXMLVisualTreeDemo
2_FlexboxDemo
3_USSStylesDemo
4_UQueryDemo
5_Pseudo-classesDemo
6_USS TransitionsDemo
7_EventsDemo
8_EventDispatchDemo
9_ManipulatorDemo
10_Custom controls
Templates
Quiz
Custom Controls (C#)
Quiz
CustomColorButton

QuizU - Quiz


Library
Standard
Project
UI Documents (UXML)
Assets
Unity Technologies
QuizU - A UI Toolkit demo
Demos
Quiz
Data
Resources
Uxml
question-text.uxml
VisualTreeAssets
level-button.uxml
UI
Uxml


Library
Standard
Project
UI Documents (UXML)
Assets
Unity Technologies
QuizU - A UI Toolkit demo
Demos
Quiz
Data
UI
Uxml
BootloadScreen.uxml
EndScreen.uxml
GameScreen.uxml
LevelSelectionScreen.uxml
MainMenuScreen.uxml
PauseScreen.uxml
SettingsScreen.uxml
SplashScreen.uxml
StartScreen.uxml
UIScreens.uxml
VisualTreeAssets
Custom Controls (C#)
Quiz
CustomColorButton

CustomControl → C# - USS transition

```
using UnityEngine;
using UnityEngine.UIElements;

namespace Quiz
{
    /// <summary>
    /// This button toggles between an active and inactive color, showing a simple custom control
    /// available in the UI Builder.
    /// </summary>
    3 references
    public class CustomColorButton : Button
    {
        // Properties to hold the active and inactive colors
        2 references
        Color activeColor = new Color(0f, 0.5f, 0.85f, 1f);
        3 references
        Color inactiveColor = new Color(0.5f, 0.5f, 0.5f, 1f);

        3 references
        bool m_IsActive = false;

        1 reference
        public new class UxmlFactory : UxmlFactory<CustomColorButton, UxmlTraits> { }

        // This exposes attributes in the UI Builder
        1 reference
        public new class UxmlTraits : Button.UxmlTraits
        {
            1 reference
            UxmlColorAttributeDescription m_AttrActiveColor =
                new UxmlColorAttributeDescription
                {
                    name = "active-color",
                    defaultValue = new Color(0f, 0.5f, 0.85f, 1f)
                };

            1 reference
            UxmlColorAttributeDescription m_AttrInactiveColor =
                new UxmlColorAttributeDescription
                {
                    name = "inactive-color",
                    defaultValue = new Color(0.5f, 0.5f, 0.5f, 1f)
                };
```

CustomControl → C# - USS transition

```
        0 references
        public CustomColorButton()
        {
            this.style.minWidth = 100;
            this.style.minHeight = 100;

            //// Optional: this shows the syntax for setting up a basic USS transition using the C# syntax
            //// More information: https://docs.unity3d.com/Manual/UIE-Transitions.html

            // Omitted for simplicitiy. Uncomment this block to add some "built-in" USS Transitions:


            // TransitionDuration needs a List of TimeValues

            List<TimeValue> durations = new List<TimeValue>();
            durations.Add(new TimeValue(0.5f, TimeUnit.Second));
            this.style.transitionDuration = new StyleList<TimeValue>(durations);

            // easingFunctions also needs a List

            List<EasingFunction> easingFunctions = new List<EasingFunction>();
            easingFunctions.Add(new EasingFunction(EasingMode.EaseInOut));

            this.style.transitionTimingFunction = new StyleList<EasingFunction>(easingFunctions);


            // Use the Clickable Manipulator to toggle between active and inactive color.
            // The clicked property is a shorthand way to subscribe to click events (versus RegisterCallback).
            this.clicked += OnClick;

        }

        // Note: CustomColorButton instance is tightly bound to its OnClick handler; when the button is destroyed, the OnClick handler will be
        // be garbage collected alongside the button.
        1 reference
        private void OnClick()
        {
            m_IsActive = !m_IsActive;
            this.style.backgroundColor = m_IsActive ? activeColor : inactiveColor;
        }
    }
}
```

Manipulator → Dragger

```
21      public SimpleDragManipulator()
22      {
23          // This filter determines what condition starts the Manipulator (Left Mouse Button down)
24          activators.Add(new ManipulatorActivationFilter { button = MouseButton.LeftMouse });
25      }
26
27      // Event subscriptions
        0 references
28      protected override void RegisterCallbacksOnTarget()
29      {
30          target.RegisterCallback<MouseDownEvent>(OnMouseDown);
31          target.RegisterCallback<MouseMoveEvent>(OnMouseMove);
32          target.RegisterCallback<MouseUpEvent>(OnMouseUp);
33      }
34
35      // Event unsubscriptions
        0 references
36      protected override void UnregisterCallbacksFromTarget()
37      {
38          target.UnregisterCallback<MouseDownEvent>(OnMouseDown);
39          target.UnregisterCallback<MouseMoveEvent>(OnMouseMove);
40          target.UnregisterCallback<MouseUpEvent>(OnMouseUp);
41      }
42
43      // Check if we can click and drag an element and then initialize some values
        2 references
44      private void OnMouseDown(MouseDownEvent evt)
45      {
46          // Checks whether MouseEvent satisfies all of the ManipulatorActivationFilter requirements.
47          if (CanStartManipulation(evt))
48          {
49              // Save the mouse and element start positions
50              m_StartMousePosition = evt.mousePosition;
51              m_StartElementPosition = new Vector2(target.layout.x, target.layout.y);
52
53              // Flag that this is active and receive all mouse events, even if the mouse pointer leaves
54              m_IsDragging = true;
55              target.CaptureMouse();
56
57              // Limit the event to this element, do not send up or down the hierarchy
58              evt.StopPropagation();
59          }
60      }
```

Manipulator → Dragger

```
61
62      // Offset the dragged element based on the difference from the start position
        2 references
63      private void OnMouseMove(MouseMoveEvent evt)
64      {
65          // Checks whether the MouseEvent is related to this Manipulator and dragging is active
66          if (CanStopManipulation(evt) && m_IsDragging)
67          {
68              // Use the difference in mouse position to offset the element as well
69              Vector3 mouseDelta = evt.mousePosition - m_StartMousePosition;
70
71              // Convert the pixel offset into a new left/top StyleLength
72              target.style.left = CreatePixelLength(m_StartElementPosition.x + mouseDelta.x);
73              target.style.top = CreatePixelLength(m_StartElementPosition.y + mouseDelta.y);
74
75              // Limit the event to this element, do not send up or down the hierarchy
76              evt.StopPropagation();
77          }
78      }
79
80      // Creates a new StyleLength representing a length in pixels.
        2 references
81      private StyleLength CreatePixelLength(float value)
82      {
83          return new StyleLength(new Length(value, LengthUnit.Pixel));
84      }
```

Diferencias con el nuestro

```
21      public SimpleDragManipulator()
22      {
23          // This filter determines what condition starts the Manipulator (Left Mouse Button down)
24          activators.Add(new ManipulatorActivationFilter { button = MouseButton.LeftMouse });
25      }
26
27      // Event subscriptions
        0 references
28      protected override void RegisterCallbacksOnTarget()
29      {
30          target.RegisterCallback<MouseDownEvent>(OnMouseDown);
31          target.RegisterCallback<MouseMoveEvent>(OnMouseMove);
32          target.RegisterCallback<MouseUpEvent>(OnMouseUp);
33      }
34
35      // Event unsubscriptions
        0 references
36      protected override void UnregisterCallbacksFromTarget()
37      {
38          target.UnregisterCallback<MouseDownEvent>(OnMouseDown);
39          target.UnregisterCallback<MouseMoveEvent>(OnMouseMove);
40          target.UnregisterCallback<MouseUpEvent>(OnMouseUp);
41      }
42
43      // Check if we can click and drag an element and then initialize some values
        2 references
44      private void OnMouseDown(MouseDownEvent evt)
45      {
46          // Checks whether MouseEvent satisfies all of the ManipulatorActivationFilter requirements.
47          if (CanStartManipulation(evt))
48          {
49              // Save the mouse and element start positions
50              m_StartMousePosition = evt.mousePosition;
51              m_StartElementPosition = new Vector2(target.layout.x, target.layout.y);
52
53              // Flag that this is active and receive all mouse events, even if the mouse pointer leaves
54              m_IsDragging = true;
55              target.CaptureMouse();
56
57              // Limit the event to this element, do not send up or down the hierarchy
58              evt.StopPropagation();
59          }
60      }
```

```
using UnityEngine;
using UnityEngine.UIElements;

public class ExampleDragger : PointerManipulator
{
    private Vector3 m_Start;
    protected bool m_Active;
    private int m_PointerId;
    private Vector2 m_StartSize;

    public ExampleDragger()
    {
        m_PointerId = -1;
        activators.Add(new ManipulatorActivationFilter { button = MouseButton.LeftMouse });
        m_Active = false;
    }

    protected override void RegisterCallbacksOnTarget()
    {
        target.RegisterCallback<PointerDownEvent>(OnPointerDown);
        target.RegisterCallback<PointerMoveEvent>(OnPointerMove);
        target.RegisterCallback<PointerUpEvent>(OnPointerUp);
    }

    protected override void UnregisterCallbacksFromTarget()
    {
        target.UnregisterCallback<PointerDownEvent>(OnPointerDown);
        target.UnregisterCallback<PointerMoveEvent>(OnPointerMove);
        target.UnregisterCallback<PointerUpEvent>(OnPointerUp);
    }
```

```
protected void OnPointerDown(PointerDownEvent e)
{
    if (m_Active)
    {
        e.StopImmediatePropagation();
        return;
    }

    if (CanStartManipulation(e))
    {
        m_Start = e.localPosition;
        m_PointerId = e.pointerId;

        m_Active = true;
        target.CapturePointer(m_PointerId);
        e.StopPropagation();
    }
}
```

Diferencias con el nuestro

```
61
62      // Offset the dragged element based on the difference from the start position
        2 references
63      private void OnMouseMove(MouseMoveEvent evt)
64      {
65          // Checks whether the MouseEvent is related to this Manipulator and dragging is active
66          if (CanStopManipulation(evt) && m_IsDragging)
67          {
68              // Use the difference in mouse position to offset the element as well
69              Vector3 mouseDelta = evt.mousePosition - m_StartMousePosition;
70
71              // Convert the pixel offset into a new left/top StyleLength
72              target.style.left = CreatePixelLength(m_StartElementPosition.x + mouseDelta.x);
73              target.style.top = CreatePixelLength(m_StartElementPosition.y + mouseDelta.y);
74
75              // Limit the event to this element, do not send up or down
76              evt.StopPropagation();
77          }
78      }
79
80      // Creates a new StyleLength representing a length in pixels.
        2 references
81      private StyleLength CreatePixelLength(float value)
82      {
83          return new StyleLength(new Length(value, LengthUnit.Pixel));
84      }
```

```
    protected void OnPointerMove(PointerMoveEvent e)
    {
        if (!m_Active || !target.HasPointerCapture(m_PointerId))
            return;

        Vector2 diff = e.localPosition - m_Start;

        target.style.top = target.layout.y + diff.y;
        target.style.left = target.layout.x + diff.x;

        e.StopPropagation();
    }

    protected void OnPointerUp(PointerUpEvent e)
    {
        if (!m_Active || !target.HasPointerCapture(m_PointerId) || !CanStopManipulation(e))
            return;

        m_Active = false;
        target.ReleaseMouse();
        e.StopPropagation();
    }
}
```

Campus Virtual → Documentation

QuizU documentation

Documentation for the QuizU - an official Unity sample demonstrating MVP, state pattern, managing menu screens and much more using UI Toolkit available on the Unity Asset Store.

Version 1.01 Nov 1st 2023

Table of contents

Welcome to the new sample project QuizU **4**
- Key features 4
- UI Toolkit demos 6
- The mini-game 7
- QuizU: Exploring the demo scenes 9
  - Demo selection 9
  - UXML and visual trees 10
- GroupBox versus VisualElement 11
  - Flexbox rules and layout 11
  - USS styles 12
  - UQuery 13
  - Pseudo-classes 14
  - USS Transitions 15
  - Events 16
  - Event dispatch and propagation 17
  - Manipulators 18
  - Custom controls 19
- Continuing with UI Toolkit 20

Índice

- Proyecto

- Project Samples
  - Dragon Crashers
    - Custom Controls
      - Tabbed menú
      - Radial progress bar
  - Quizu
    - Demos
      - CustomControls
      - Manipulators
    - **Quiz**
      - **Data**
      - **Scripts**
        - **Coroutines**
        - **ScriptablesObjects**
        - **Actions**

Quiz

UNITY › sample1 › Assets › Unity Technologies › QuizU - A UI Toolkit demo › Quiz ›

Nombre	Fecha de modificación	Tipo	Tamaño
Data	18/03/2024 17:46	Carpeta de archivos	
Prefabs	18/03/2024 17:46	Carpeta de archivos	
Scripts	18/03/2024 17:46	Carpeta de archivos	
Textures	18/03/2024 17:46	Carpeta de archivos	
TutorialInfo	18/03/2024 17:46	Carpeta de archivos	
UI	18/03/2024 17:46	Carpeta de archivos	
Data.meta	06/12/2023 14:48	Archivo META	1 KB
Prefabs.meta	06/12/2023 14:48	Archivo META	1 KB
Scripts.meta	06/12/2023 14:48	Archivo META	1 KB
Textures.meta	06/12/2023 14:48	Archivo META	1 KB
TutorialInfo.meta	06/12/2023 14:49	Archivo META	1 KB
UI.meta	06/12/2023 14:50	Archivo META	1 KB

Data → JSON

> UNITY › sample1 › Assets › Unity Technologies › QuizU - A UI Toolkit demo › Quiz › Data ›

Nombre	Fecha de modificación	Tipo	Tamaño
JSON	18/03/2024 17:46	Carpeta de archivos	
Questions	18/03/2024 17:46	Carpeta de archivos	
Resources	18/03/2024 17:46	Carpeta de archivos	
Unused	18/03/2024 17:46	Carpeta de archivos	
JSON.meta	06/12/2023 14:49	Archivo META	1 KB
Questions.meta	06/12/2023 14:49	Archivo META	1 KB
Resources.meta	06/12/2023 14:48	Archivo META	1 KB
Unused.meta	06/12/2023 14:50	Archivo META	1 KB

```
{
    "text": "How many audio listeners should be present in a Unity scene?",
    "skills": "Implement audio in Unity, Produce customized results by correctly configuring audio in a scene",
    "shuffleOptions": true,
    "options": [
        {
            "text": "As many as there are sound producing objects",
            "isCorrect": false,
            "feedback": "An audio listener plays any audio sources that are within range in a scene. There is only ever one audio listener, generally on the main camera."
        },
        {
            "text": "Two",
            "isCorrect": false,
            "feedback": "An audio listener plays any audio sources that are within range in a scene. There is only ever one audio listener, generally on the main camera."
        },
        {
            "text": "One for each directly interactable object",
            "isCorrect": false,
            "feedback": "An audio listener plays any audio sources that are within range in a scene. There is only ever one audio listener, generally on the main camera."
        },
        {
            "text": "One",
            "isCorrect": true,
            "feedback": "There is only ever one audio listener, generally on the main camera."
        }
    ],
    "feedbackCorrect": "There is only ever one audio listener, generally on the main camera.",
    "feedbackIncorrect": ""
}
```

Scripts

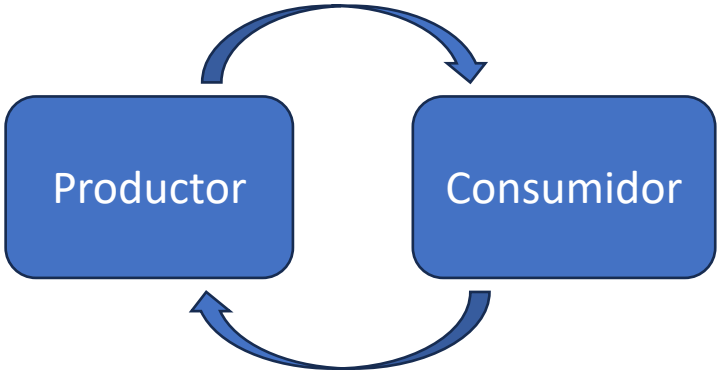
> UNITY > sample1 > Assets > Unity Technologies > QuizU - A UI Toolkit demo > Quiz > Scripts >

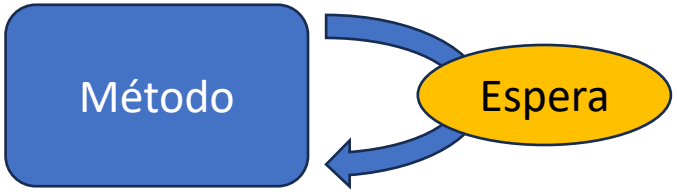
Nombre	Fecha de modificación	Tipo	Tamaño
Editor	18/03/2024 17:46	Carpeta de archivos	
Events	18/03/2024 17:46	Carpeta de archivos	
Gameplay	18/03/2024 17:46	Carpeta de archivos	
Managers	18/03/2024 17:46	Carpeta de archivos	
ScriptableObjects	18/03/2024 17:46	Carpeta de archivos	
StateMachine	18/03/2024 17:46	Carpeta de archivos	
UI	18/03/2024 17:46	Carpeta de archivos	
Utilities	18/03/2024 17:46	Carpeta de archivos	
Editor.meta	06/12/2023 14:49	Archivo META	1 KB
Events.meta	06/12/2023 14:48	Archivo META	1 KB
Gameplay.meta	06/12/2023 14:49	Archivo META	1 KB
Managers.meta	06/12/2023 14:48	Archivo META	1 KB
ScriptableObjects.meta	06/12/2023 14:49	Archivo META	1 KB
StateMachine.meta	06/12/2023 14:48	Archivo META	1 KB
UI.meta	06/12/2023 14:49	Archivo META	1 KB
Utilities.meta	06/12/2023 14:49	Archivo META	1 KB

State Machine → Coroutines

```
var q := new queue

coroutine produce
    loop
        while q is not full
            create some new items
            add the items to q
        yield to consume

coroutine consume
    loop
        while q is not empty
            remove some items from q
            use the items
        yield to produce

call produce
```

```
IEnumerator Fade()
{
    Color c = renderer.material.color;
    for (float alpha = 1f; alpha >= 0; alpha -= 0.1f)
    {
        c.a = alpha;
        renderer.material.color = c;
        yield return new WaitForSeconds(.1f);
    }
}
```





State Machine → Coroutines

```
1 reference
        protected virtual IEnumerator Loop()
        {
            while (true)
            {
                if (CurrentState != null && m_CurrentPlayCoroutine == null) //current state is done playing
                {
                    if (CurrentState.ValidateLinks(out var nextState))
                    {
                        if (m_PlayLock)
                        {
                            //finalize current state
                            CurrentState.Exit();
                            m_PlayLock = false;
                        }

                        CurrentState.DisableLinks();
                        SetCurrentState(nextState);
                        CurrentState.EnableLinks();
                    }
                }

                yield return null;
            }
        }

0 references
        public bool IsRunning => m_LoopCoroutine != null;
    }
}
```

ScriptableObjects

- Objetos que son instancias de clases que heredan de MonoBehaviour (“MonoBehaviourObjects”) son almacenados en memoria y existen solo mientras se juega el juego.
- **ScriptableObjects** → son almacenados en el disco duro en forma serializada (existen durante y después del juego). Son usados como un contenedor/plantilla para almacenar datos.

UNITY › sample1 › Assets › Unity Technologies › QuizU - A UI Toolkit demo › Quiz › Scripts › ScriptableObjects

Nombre	Fecha de modificación	Tipo	Tamaño
AudioSettingsSO.cs	06/12/2023 14:50	Archivo de origen ...	6 KB
DescriptionSO.cs	06/12/2023 14:48	Archivo de origen ...	1 KB
QuestionSO.cs	06/12/2023 14:49	Archivo de origen ...	5 KB
QuizSO.cs	06/12/2023 14:49	Archivo de origen ...	3 KB
AudioSettingsSO.cs.meta	06/12/2023 14:50	Archivo META	1 KB
DescriptionSO.cs.meta	06/12/2023 14:48	Archivo META	1 KB
QuestionSO.cs.meta	06/12/2023 14:49	Archivo META	1 KB
QuizSO.cs.meta	06/12/2023 14:49	Archivo META	1 KB

AudioSettingsSO

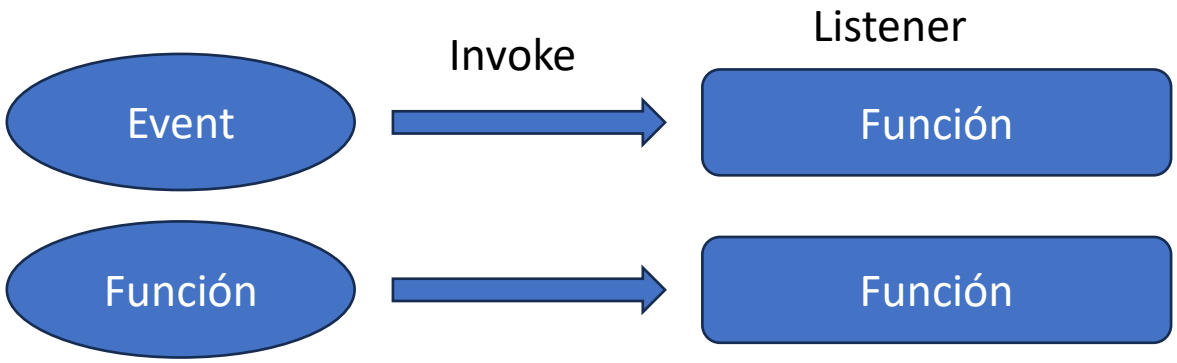
```
14      [CreateAssetMenu(fileName = "AudioSettings", menuName = "Quiz/AudioSettings", order = 1)]
        0 references
15      public class AudioSettingsSO : DescriptionSO
16      {
17          // Default ScriptableObject data
            1 reference
18          const float k_DefaultMasterVolume = 1f;
            1 reference
19          const float k_DefaultSFXVolume = 1f;
            1 reference
20          const float k_DefaultMusicVolume = 0f;
21
22          [Header("Mixer")]
23          [Tooltip("The AudioMixer that controls the audio levels for the game")]
            1 reference
24          [SerializeField] private AudioMixer m_AudioMixer;
25
26          [Header("Volume Settings")]
27          [Tooltip("The master volume level (0 to 1)")]
            2 references
28          [SerializeField] private float m_MasterVolume = k_DefaultMasterVolume;
29
30          [Tooltip("The sound effects volume level (0 to 1)")]
            2 references
31          [SerializeField] private float m_SoundEffectsVolume = k_DefaultSFXVolume;
32
33          [Tooltip("The music volume level (0 to 1)")]
            2 references
34          [SerializeField] private float m_MusicVolume = k_DefaultMusicVolume;
35
36          // Convert bool to 1 and 0 and multiply by MixerGroup (unused in this project, here for demo purposes)
37          [Header("Mute Settings")]
38          [Tooltip("Mute or unmute the master volume")]
            2 references
39          [SerializeField] private bool m_IsMasterMuted = false;
40
41          [Tooltip("Mute or unmute the sound effects volume")]
            2 references
42          [SerializeField] private bool m_IsSoundEffectsMuted = false;
43
44          [Tooltip("Mute or unmute the music volume")]
            2 references
45          [SerializeField] private bool m_IsMusicMuted = false;
46
```

QuestionSO

```
33      public class QuestionSO : ScriptableObject
34      {
35          [Tooltip("List of question text, supporting different font sizes.")]
            2 references
36          [SerializeField] List<QuestionText> m_QuestionText;
37          [Tooltip("Optional image associated with the question.")]
            1 reference
38          [SerializeField] [Optional] Sprite m_QuestionImage;
39          [TextArea(3, 5)]
40          [Tooltip("Skills tested or used in the question.")]
            2 references
41          [SerializeField] string m_Skills;
42          [Tooltip("Should the answers should be shuffled?")]
            2 references
43          [SerializeField] bool m_ShuffleAnswers;
44          [Tooltip("Additional information about the correct/incorrect responses")]
            6 references
45          [SerializeField] List<Answer> m_Answers;
46          [TextArea(3, 5)]
47          [Tooltip("Additional information provided when the correct answer is chosen.")]
            2 references
48          [SerializeField] string m_FeedbackCorrect;
49          [TextArea(3, 5)]
50          [Tooltip("Additional information provided when an incorrect answer is chosen.")]
            2 references
51          [SerializeField] string m_FeedbackIncorrect;
52          [Tooltip("Optional link to the documentation for more information.")]
            0 references
53          [SerializeField][Optional] string m_FeedbackLink;
54
```

Actions

- En clase hemos visto Delegates, events y callbacks. Resumiendo:
  - Delegates en C# son punteros a funciones. Los Delegates nos permiten usar las funciones como parámetros de entrada de otras funciones. Esto nos permite crear Callbacks.
  - En los Callbacks, un evento de sistema dispara una función.
  - En las Actions, el final de una función dispara otra función.
  - Observer Pattern:



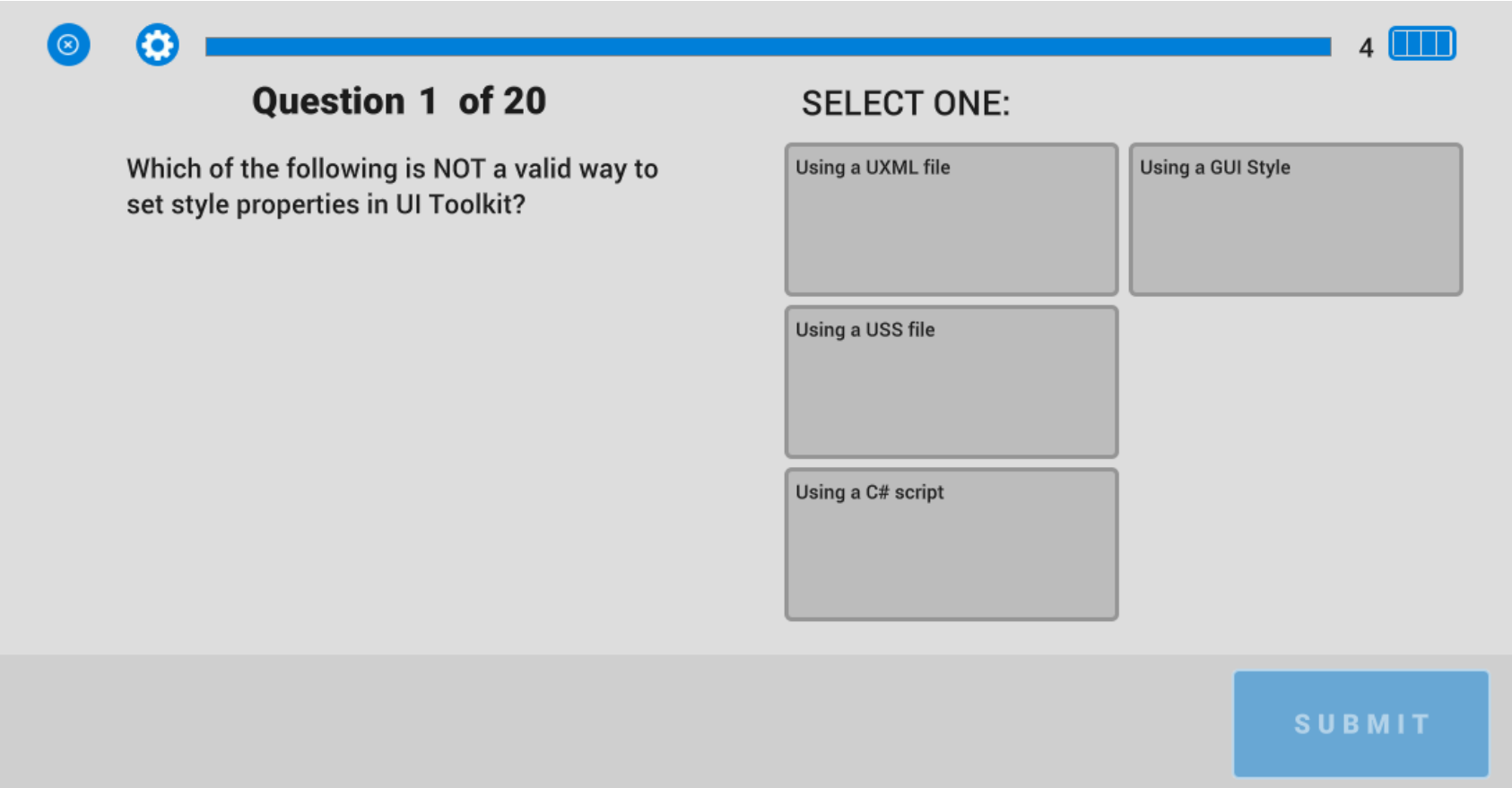
Events

```
15      public class AudioSettingsSO : DescriptionSO
76
77          // Event subscriptions
            0 references
78          private void OnEnable()
79          {
80              SettingsEvents.MasterVolumeChanged += SettingsEvents_MasterVolumeChanged;
81              SettingsEvents.MusicVolumeChanged += SettingsEvents_MusicVolumeChanged;
82              SettingsEvents.SFXVolumeChanged += SettingsEvents_SFXVolumeChanged;
83
84              // Note: we disable validating the Sound Effects AudioClips here, instead we
85              // validate it from the GameplaySounds class
86
87              // NullRefChecker.Validate(this);
88          }
89
90          // Event unsubscriptions
            0 references
91          private void OnDisable()
92          {
93              SettingsEvents.MasterVolumeChanged -= SettingsEvents_MasterVolumeChanged;
94              SettingsEvents.MusicVolumeChanged -= SettingsEvents_MusicVolumeChanged;
95              SettingsEvents.SFXVolumeChanged -= SettingsEvents_SFXVolumeChanged;
96          }
97
```

Game events → Actions

```
0 references
public static class GameEvents
{
    #region Gameplay events

    0 references
    public static Action<int> IndexUpdated;

    // Refreshed the current Question with new ScriptableObject data
    0 references
    public static Action<QuestionSO> QuestionUpdated;

    // Selected one or more user selected responses
    0 references
    public static Action<List<string>> AnswerSelected;

    // Submitted the selected answer
    0 references
    public static Action AnswerSubmitted;

    // Passes feedback for correct/incorrect responses
    0 references
    public static Action<string> CorrectlyAnswered;
    0 references
    public static Action<string> IncorrectlyAnswered;
```

Quiz → UI Toolkit



Quiz → UI Toolkit

4

Question 1 of 20

SELECTED: **D**

Which of the following is NOT a valid way to set style properties in UI Toolkit?

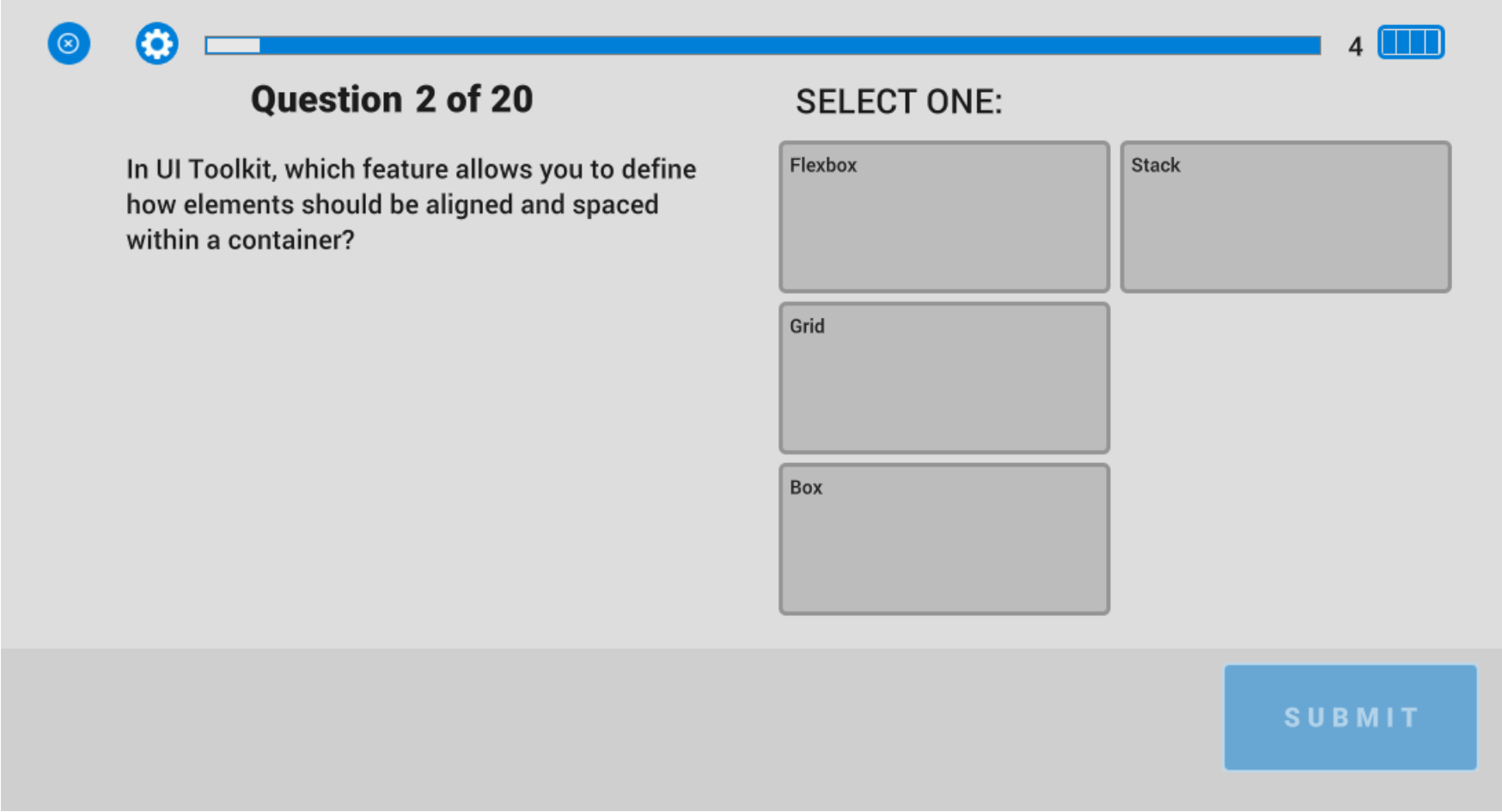
- Using a UXML file
- Using a GUI Style
- Using a USS file
- Using a C# script

Correct

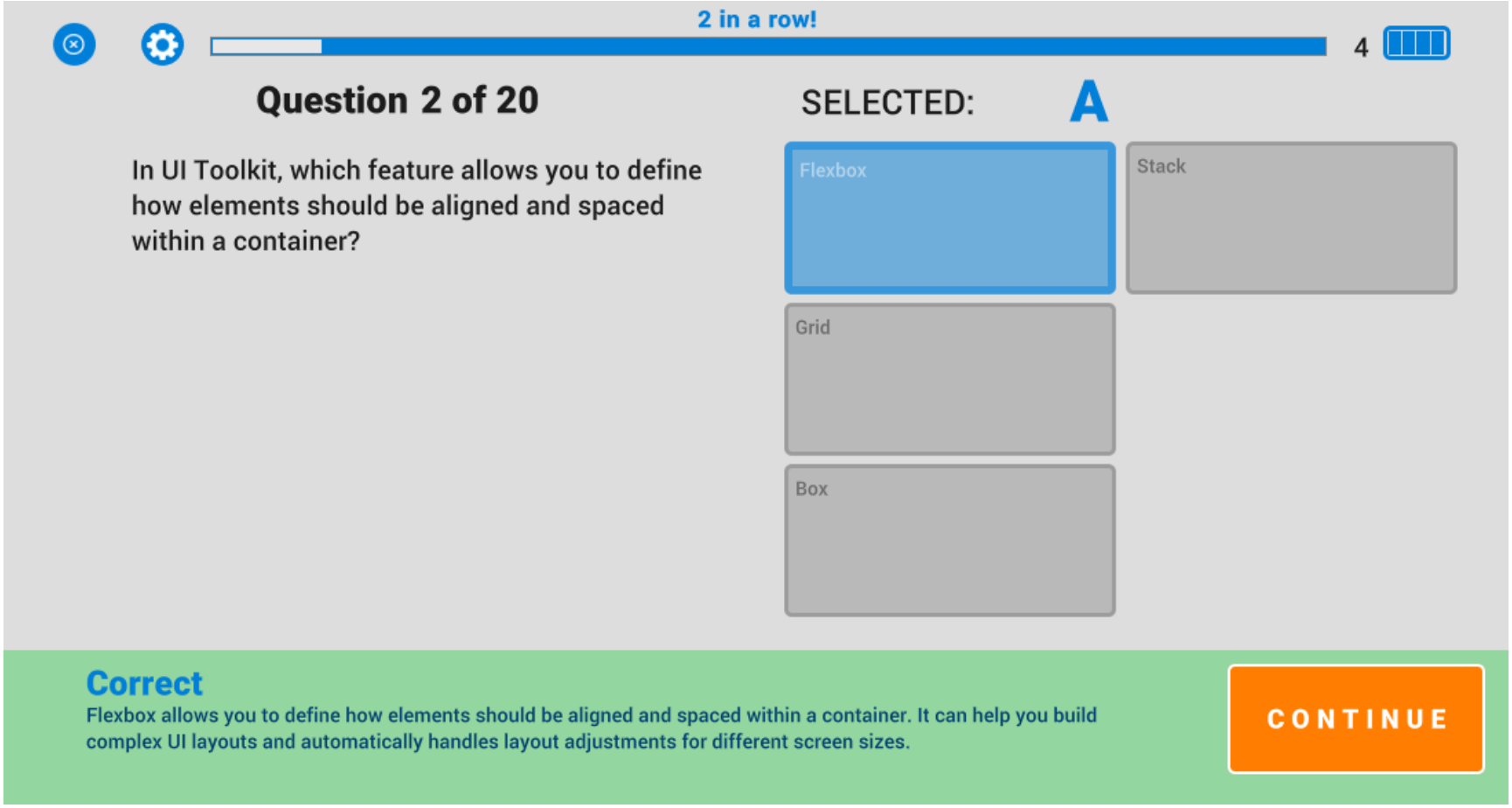
In UI Toolkit, you can set style properties using a UXML file, a USS file, or a C# script. However, UI Toolkit does not support setting style properties using a GUI Style (IMGUI), as it uses a separate system for styling UI elements.

CONTINUE

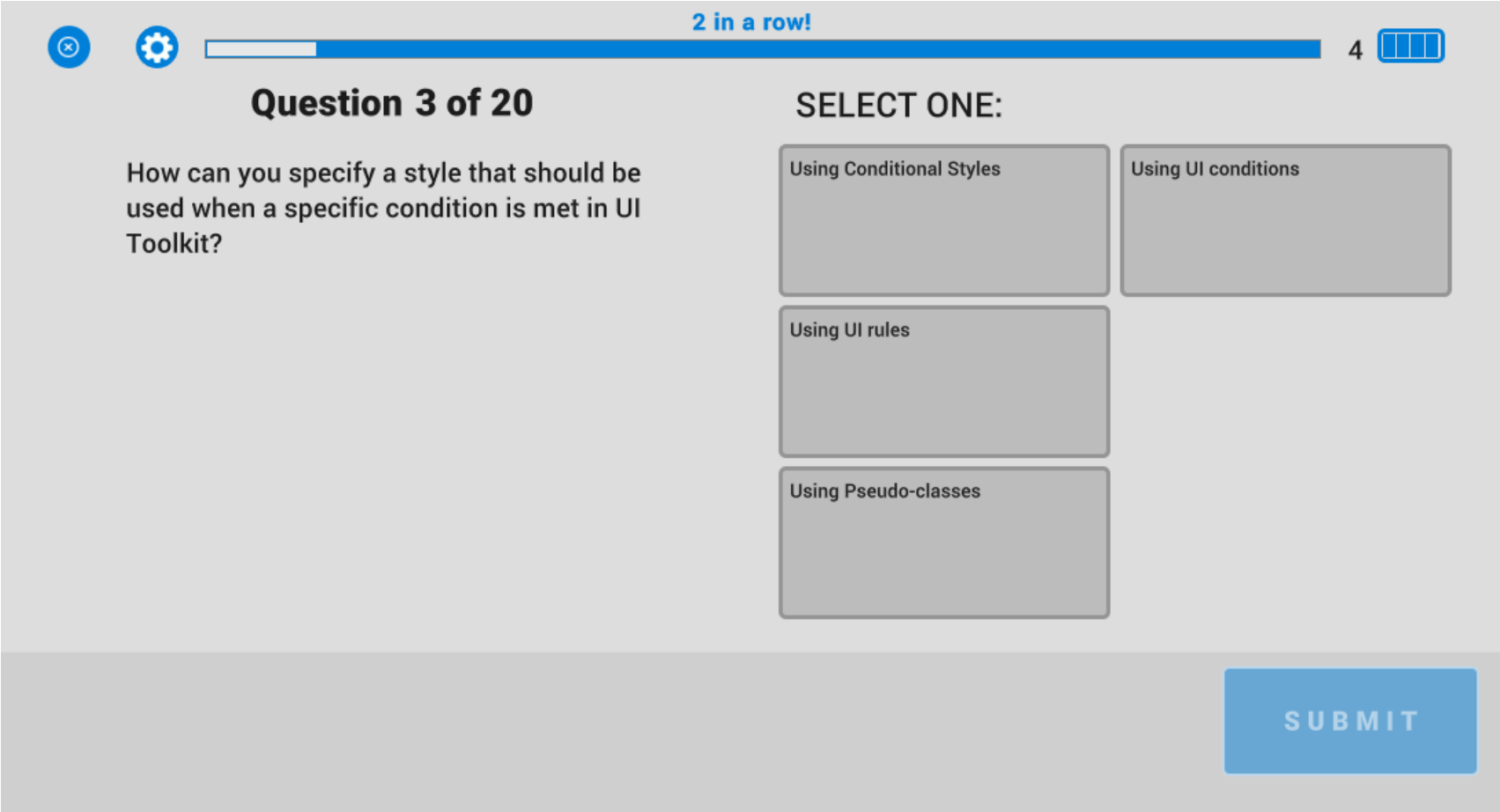
Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit

3 in a row!

4

Question 3 of 20

SELECTED: C

How can you specify a style that should be used when a specific condition is met in UI Toolkit?

- Using Conditional Styles
- Using UI conditions
- Using UI rules
- Using Pseudo-classes

Correct

In UI Toolkit, **Pseudo-classes** are used to specify a style that should be used when a specific condition is met. This allows you to create dynamic, responsive designs.

CONTINUE

Quiz → UI Toolkit

3 in a row!

4

Question 4 of 20

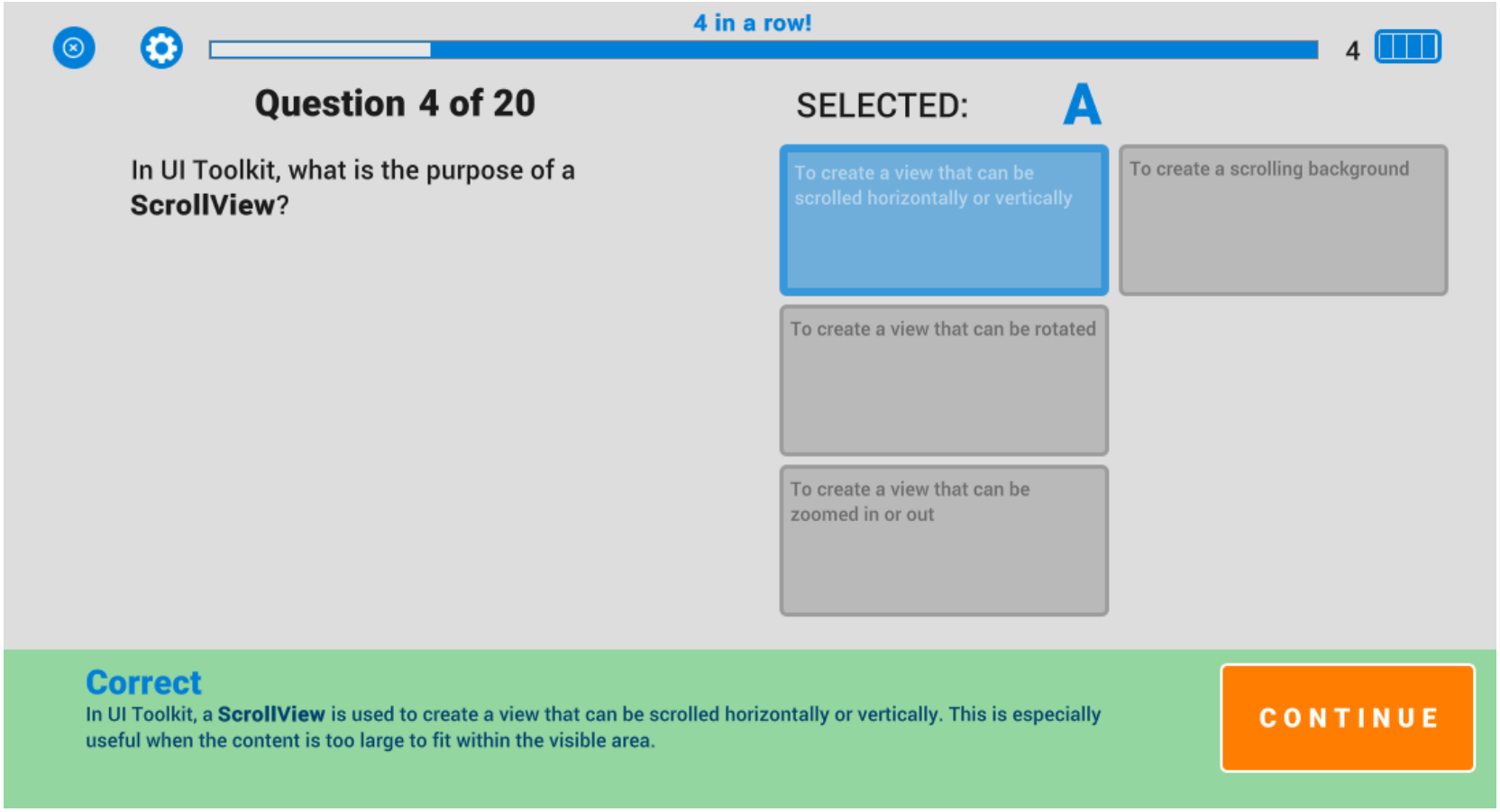
In UI Toolkit, what is the purpose of a **ScrollView**?

SELECT ONE:

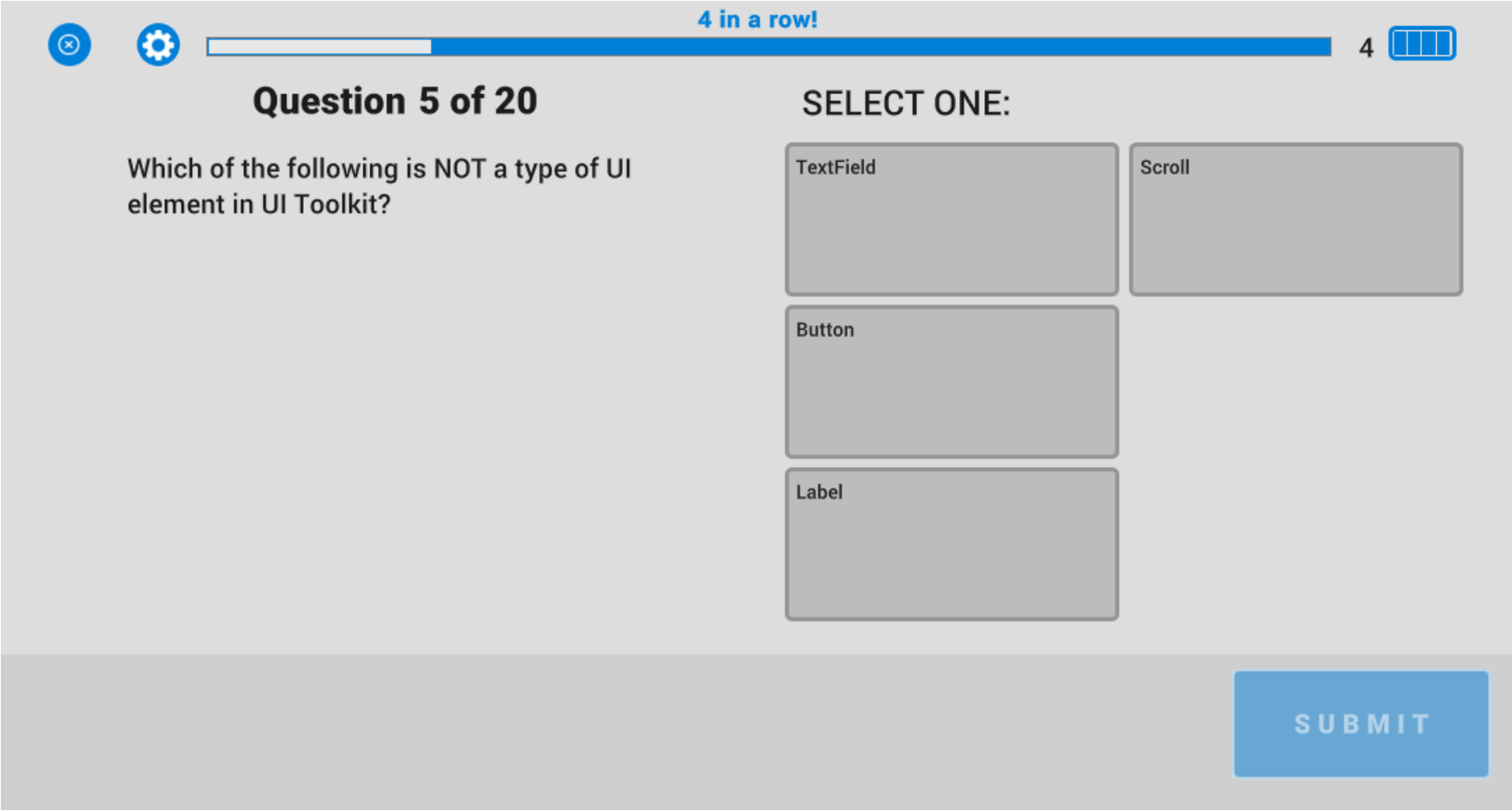
- To create a view that can be scrolled horizontally or vertically
- To create a scrolling background
- To create a view that can be rotated
- To create a view that can be zoomed in or out

SUBMIT

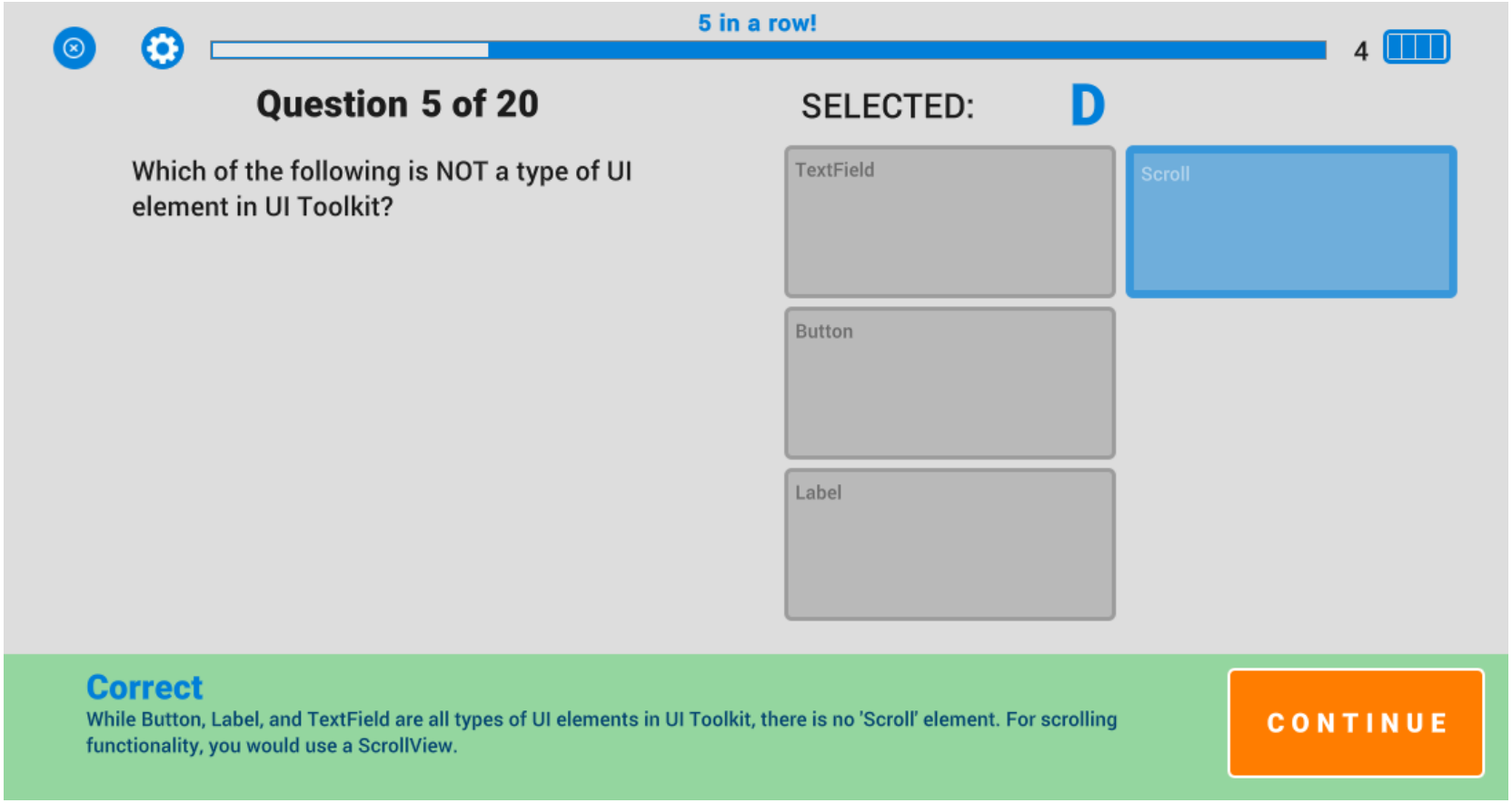
Quiz → UI Toolkit



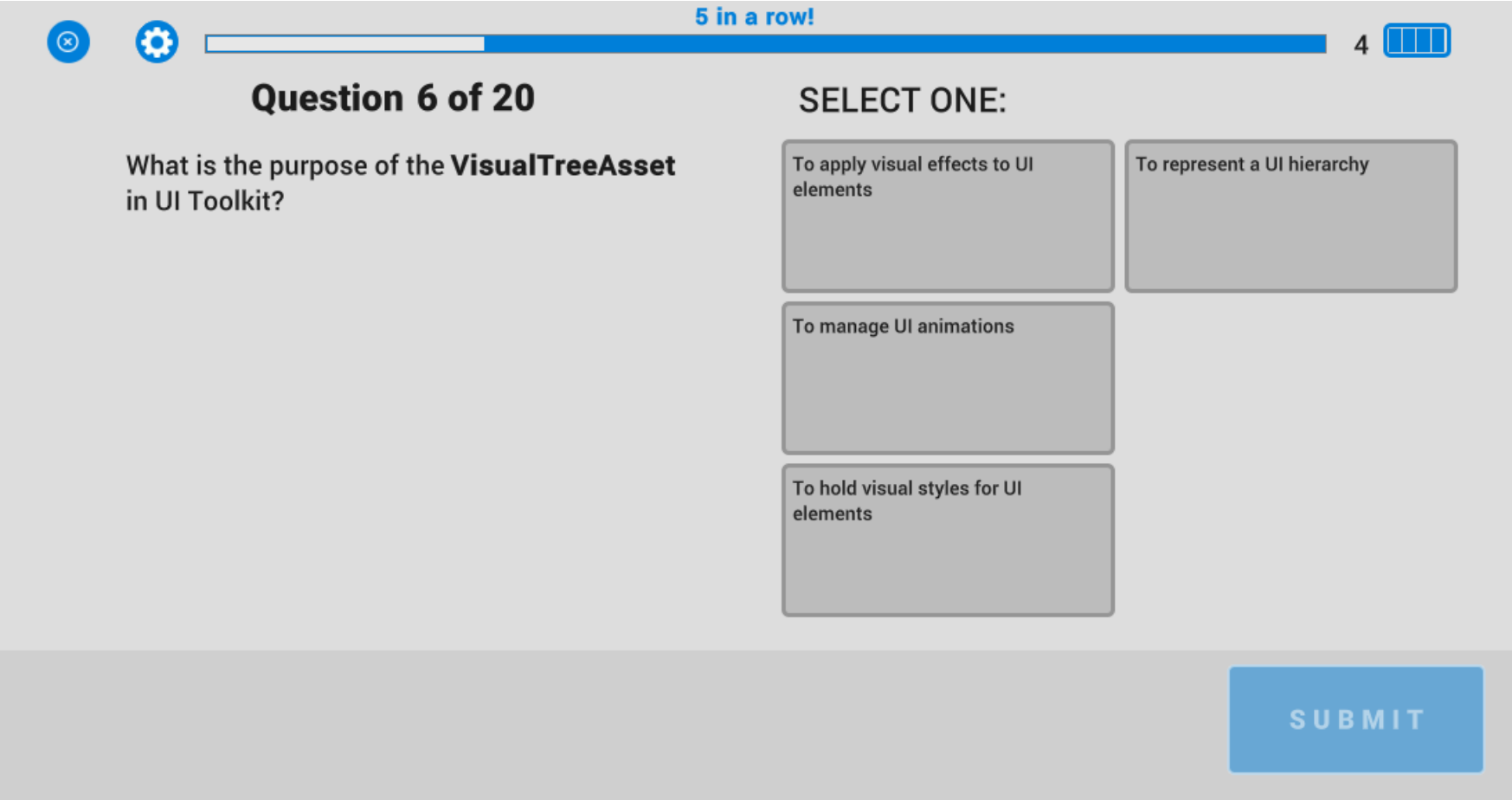
Quiz → UI Toolkit



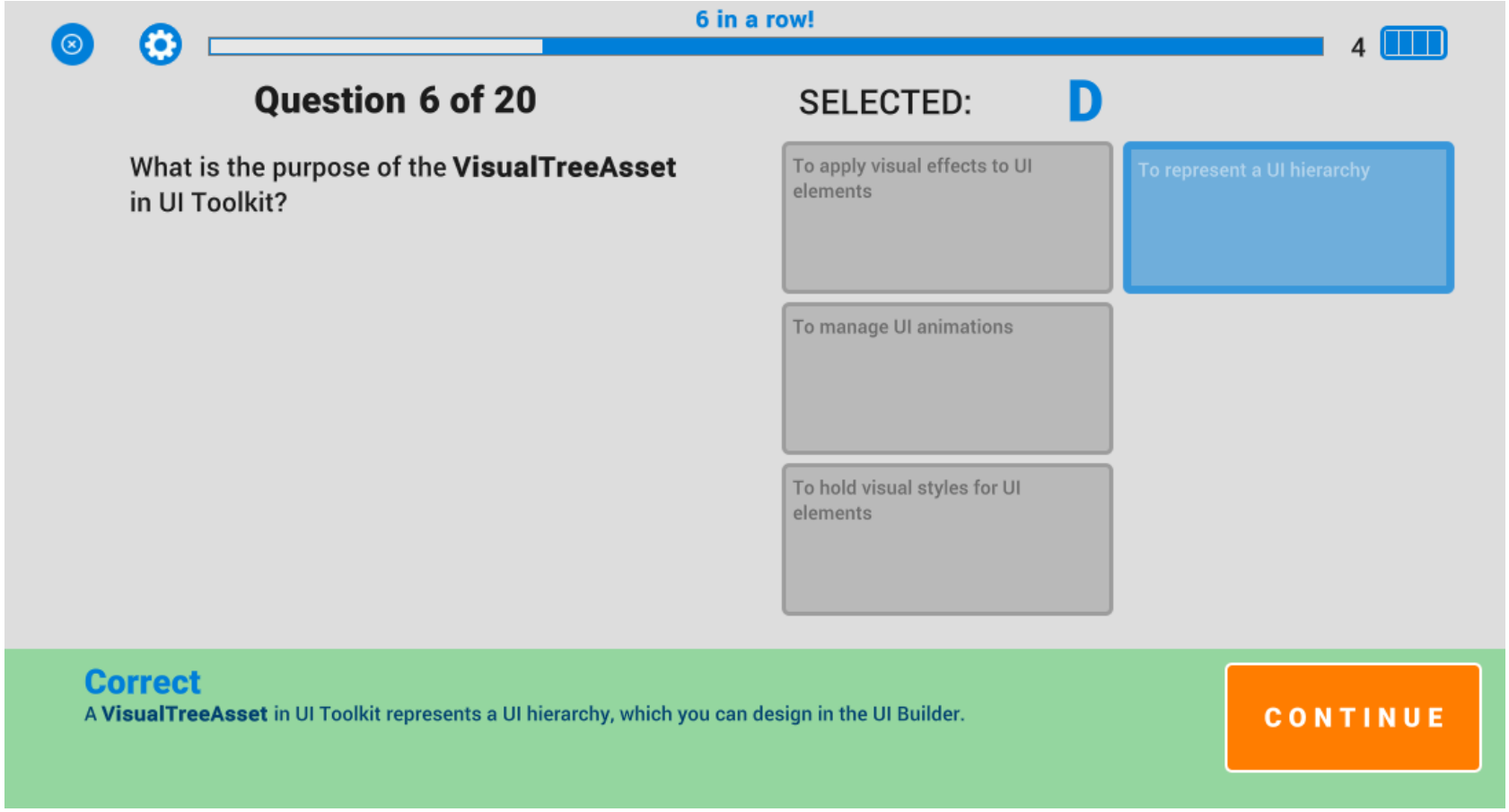
Quiz → UI Toolkit



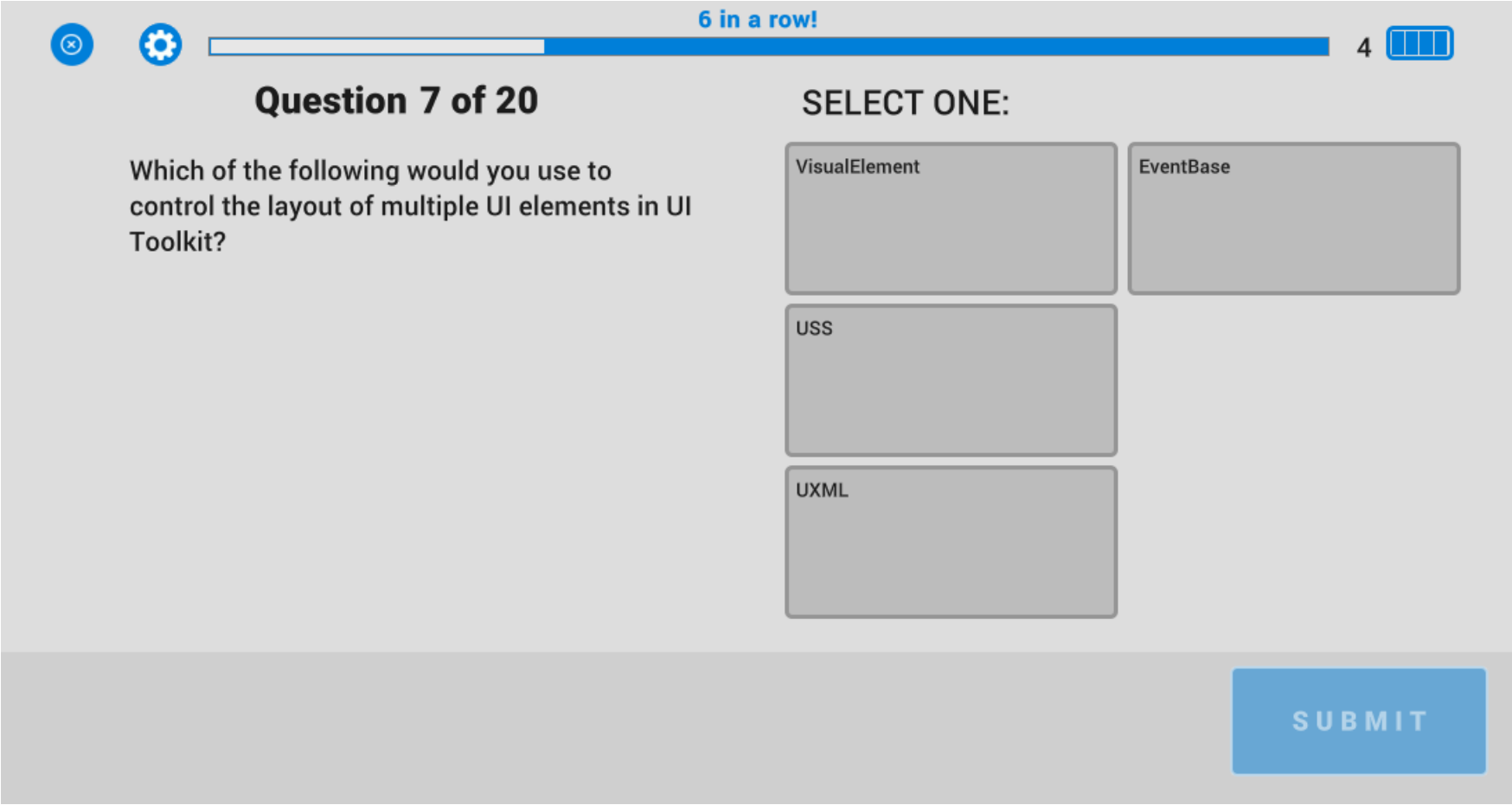
Quiz → UI Toolkit



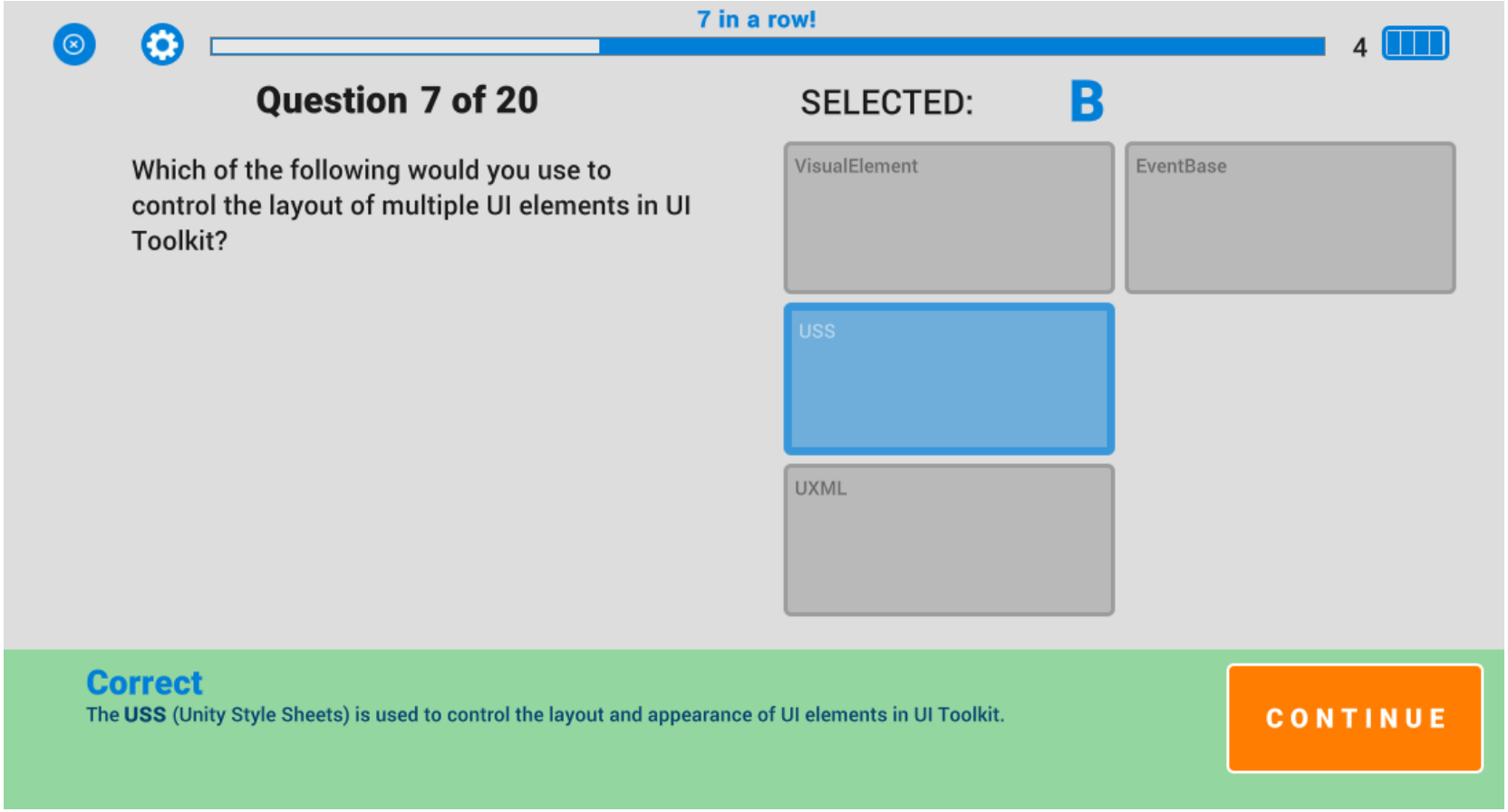
Quiz → UI Toolkit



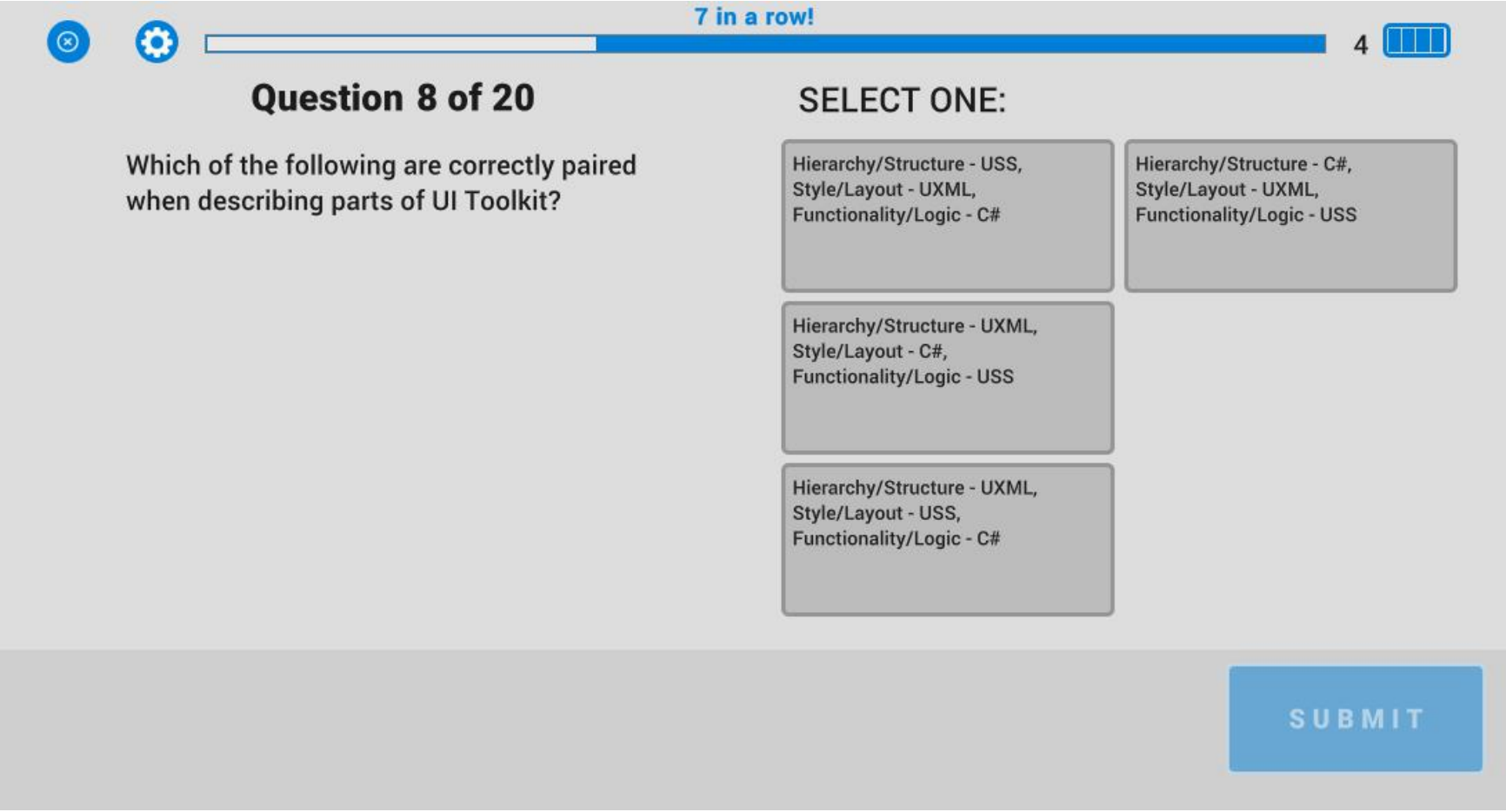
Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit

8 in a row!

4

Question 8 of 20

SELECTED: C

Which of the following are correctly paired when describing parts of UI Toolkit?

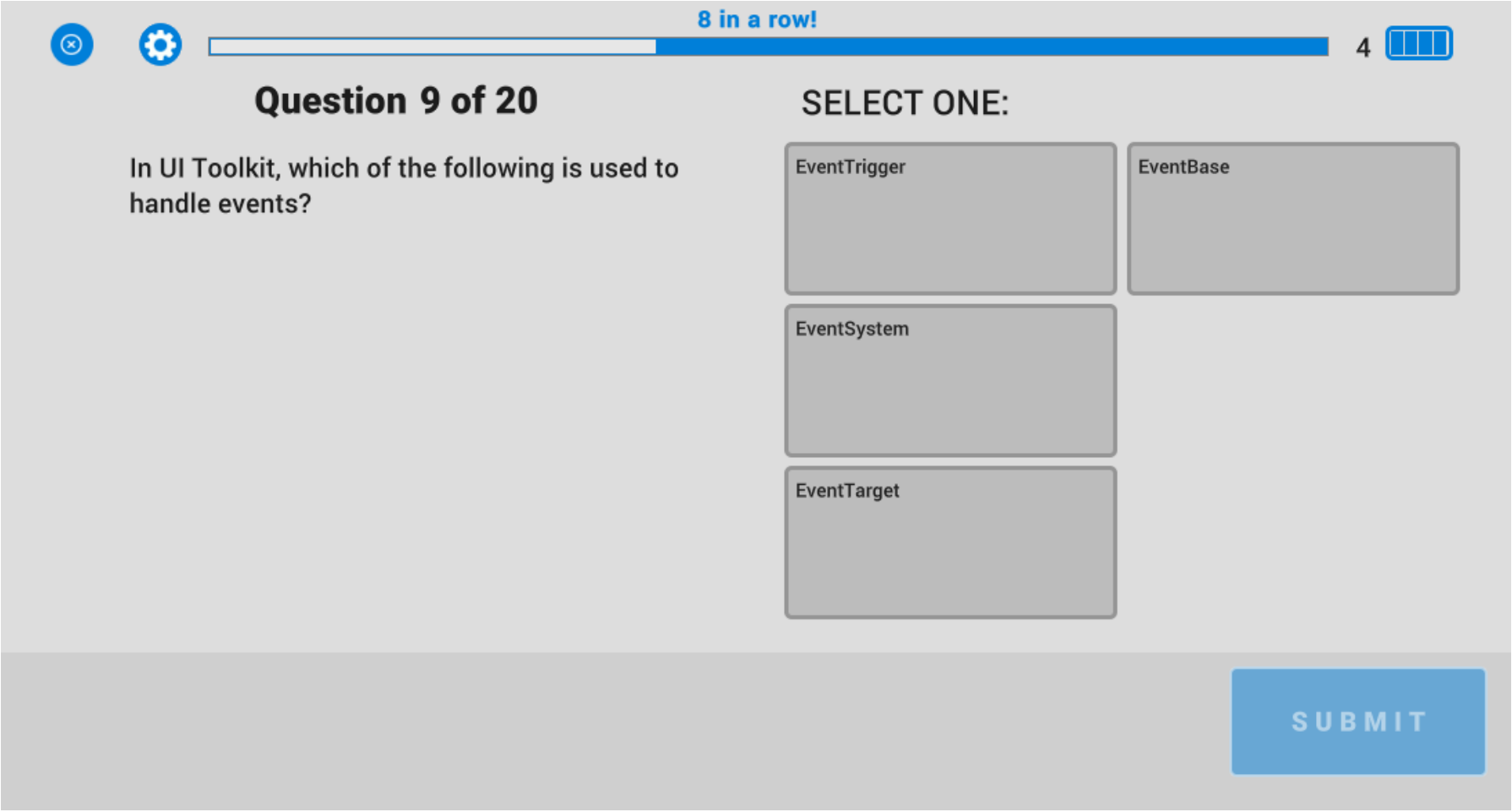
- Hierarchy/Structure - USS, Style/Layout - UXML, Functionality/Logic - C#
- Hierarchy/Structure - C#, Style/Layout - UXML, Functionality/Logic - USS
- Hierarchy/Structure - UXML, Style/Layout - C#, Functionality/Logic - USS
- Hierarchy/Structure - UXML, Style/Layout - USS, Functionality/Logic - C#

Correct

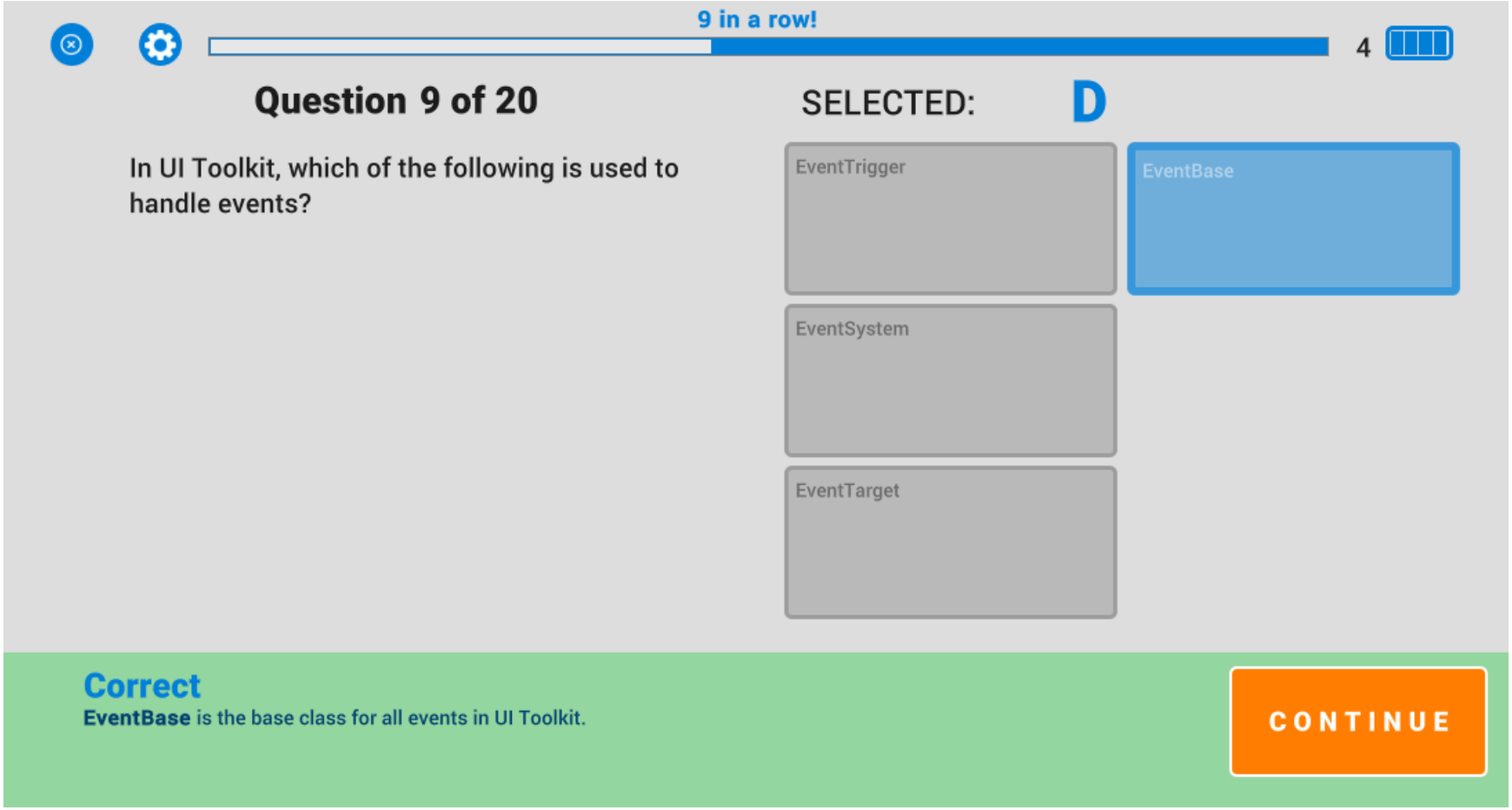
In UI Toolkit, UXML is used for declaring the hierarchy/structure of the UI, USS is used for styling and layout, and C# is used for implementing functionality and logic.

CONTINUE

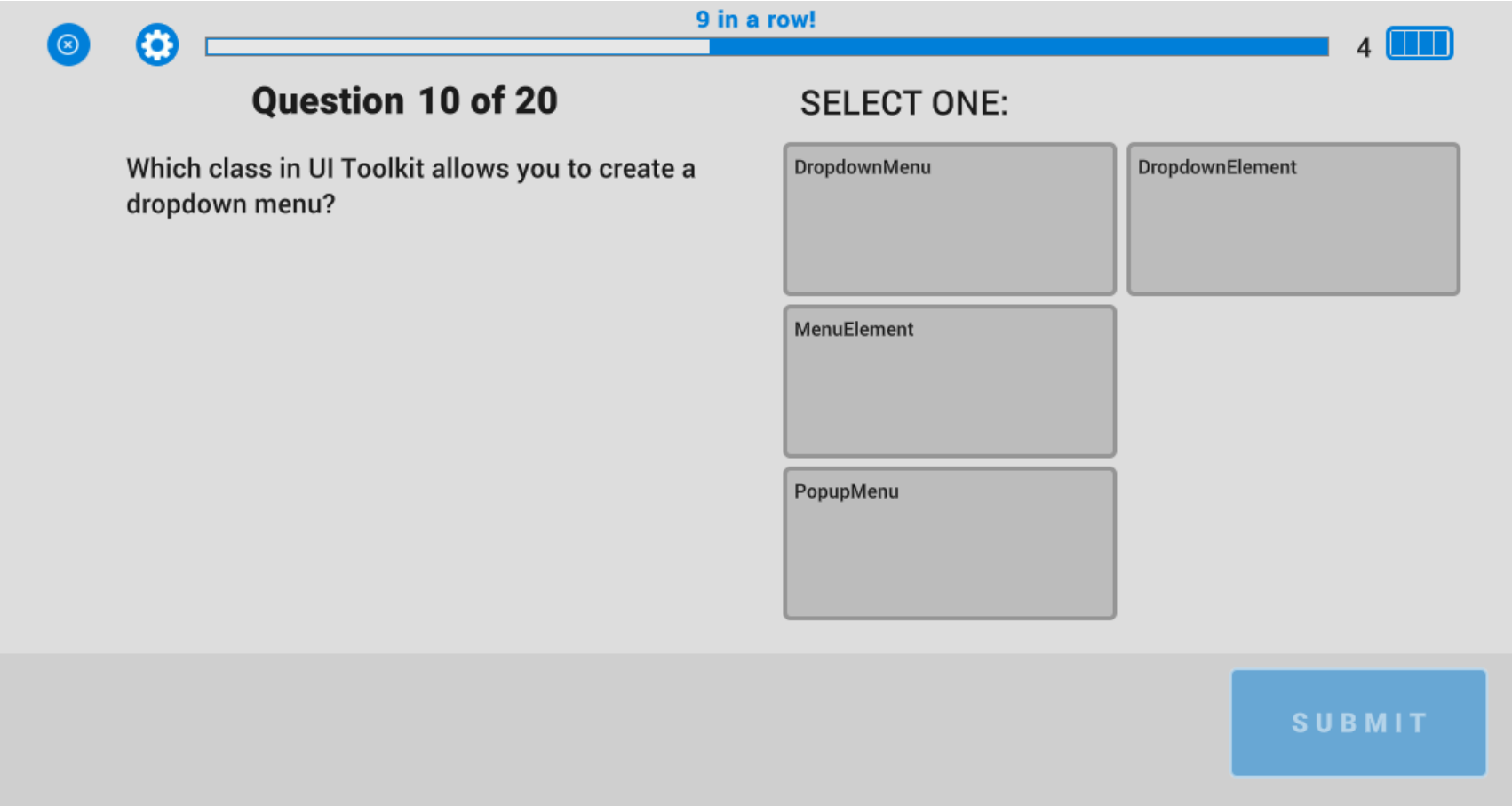
Quiz → UI Toolkit



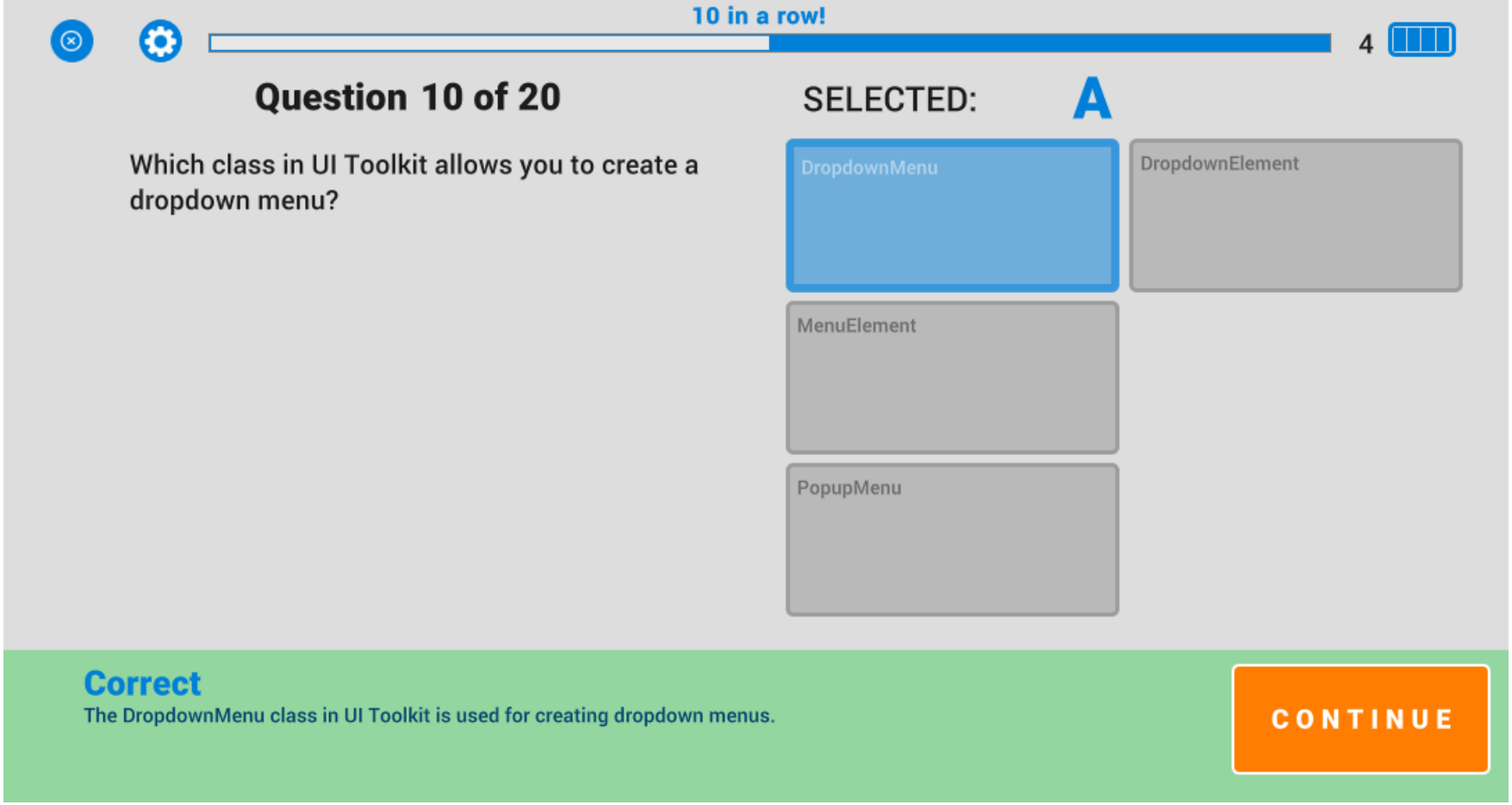
Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit

10 in a row!

4

Question 11 of 20

In UI Toolkit, how can you specify that a style should only apply when an element is hovered over by the mouse?

SELECT ONE:

- Using **:hover** pseudo-class
- Using **HoverHandler** class
- Using **hover** attribute
- Using **onMouseHover** event

SUBMIT

Quiz → UI Toolkit

11 in a row!

4

Question 11 of 20

SELECTED: A

In UI Toolkit, how can you specify that a style should only apply when an element is hovered over by the mouse?

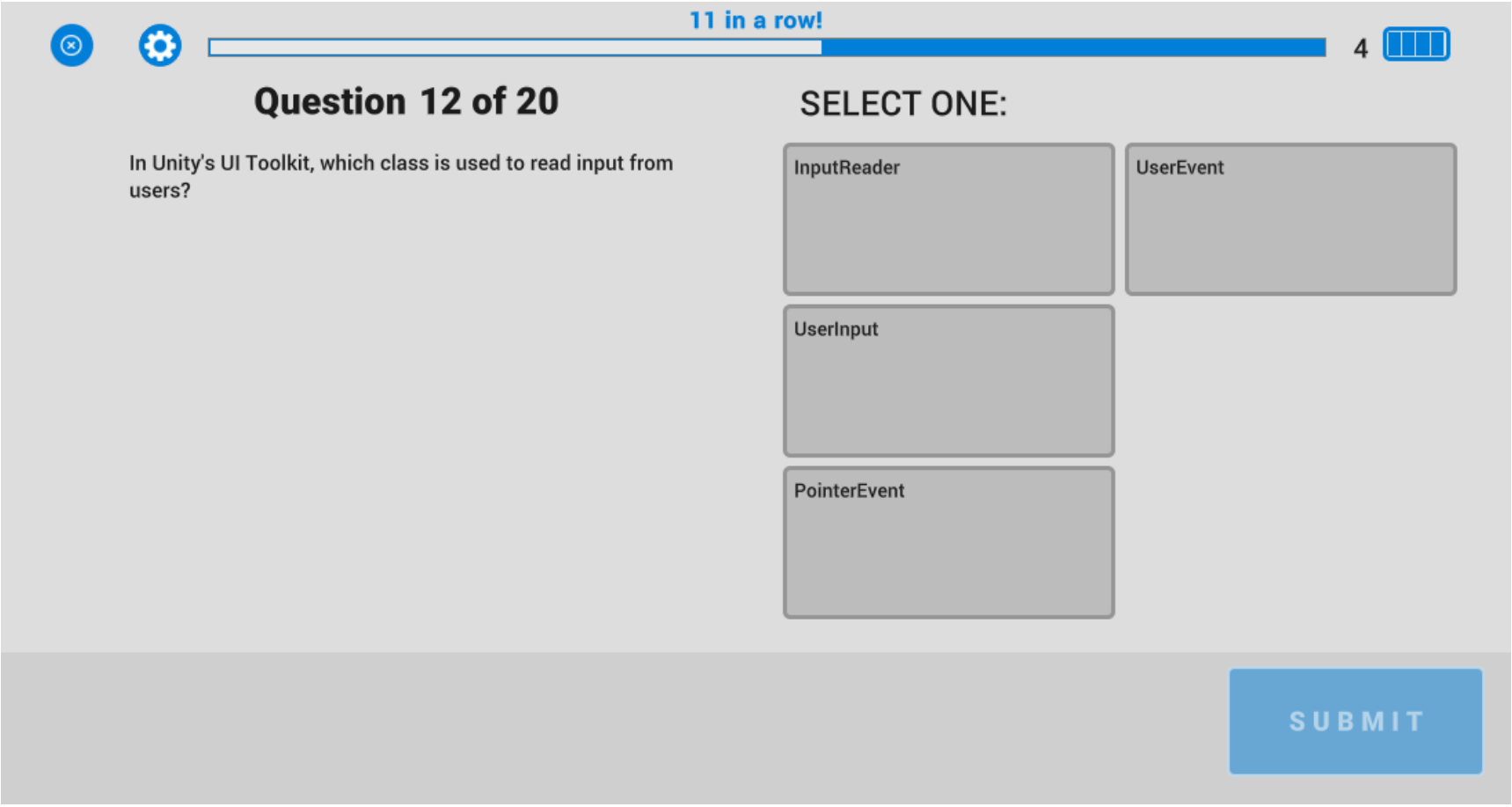
- Using **:hover** pseudo-class
- Using **HoverHandler** class
- Using **hover** attribute
- Using **onMouseHover** event

Correct

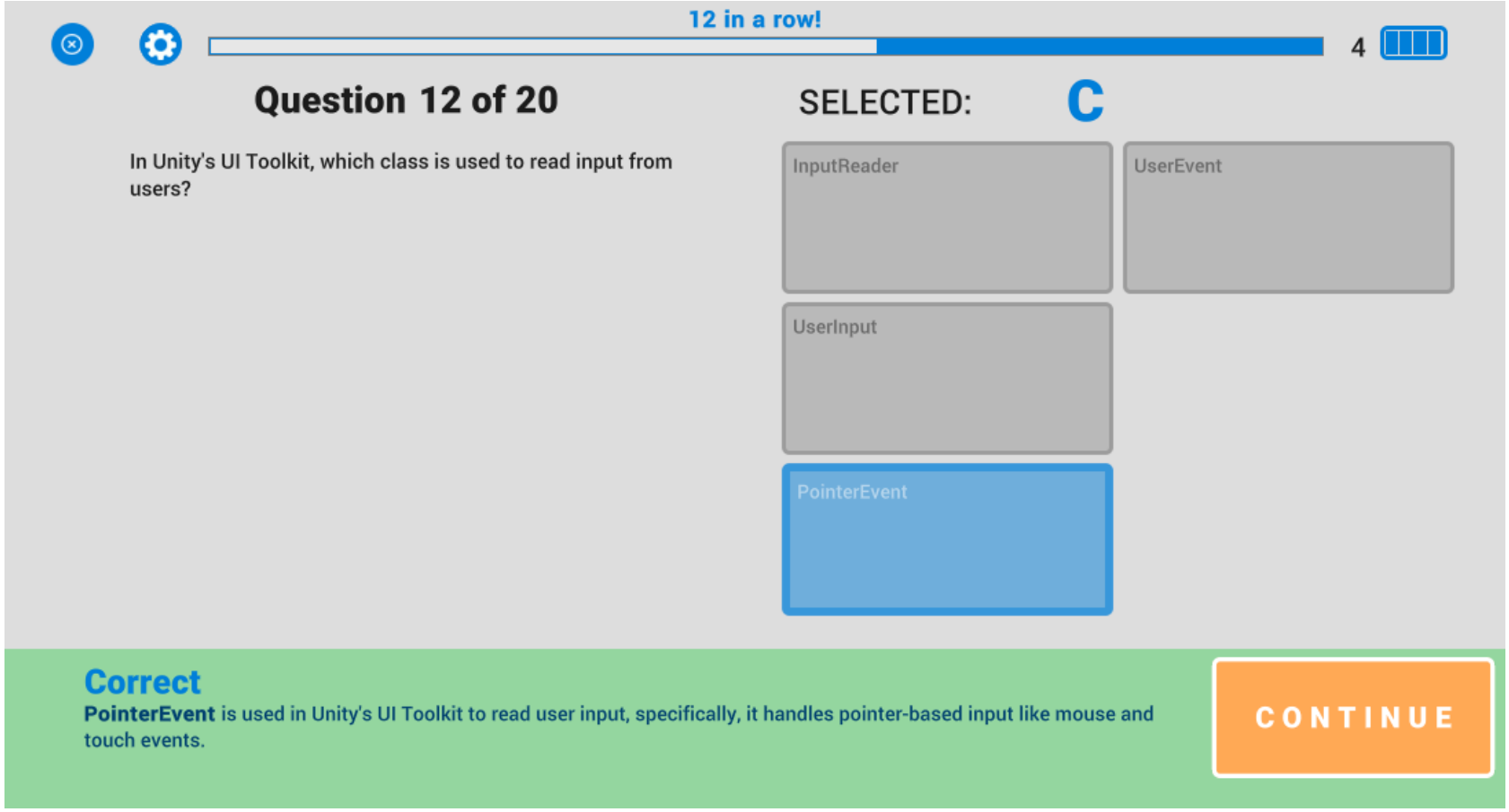
In Unity's UI Toolkit, the **:hover** pseudo-class is used to apply styles when an element is hovered over by the mouse. This allows you to create interactive and responsive designs in your user interface.

CONTINUE

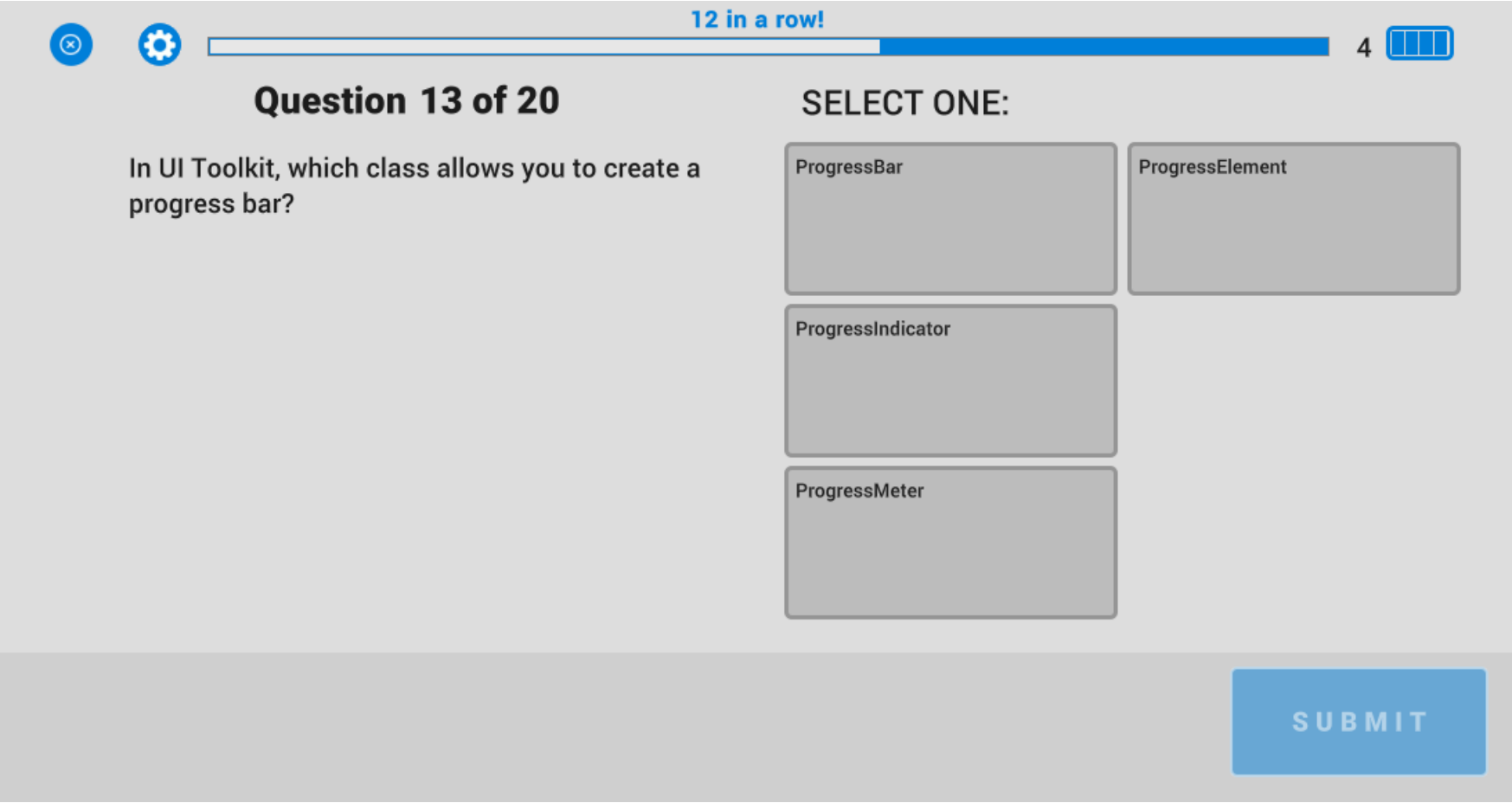
Quiz → UI Toolkit



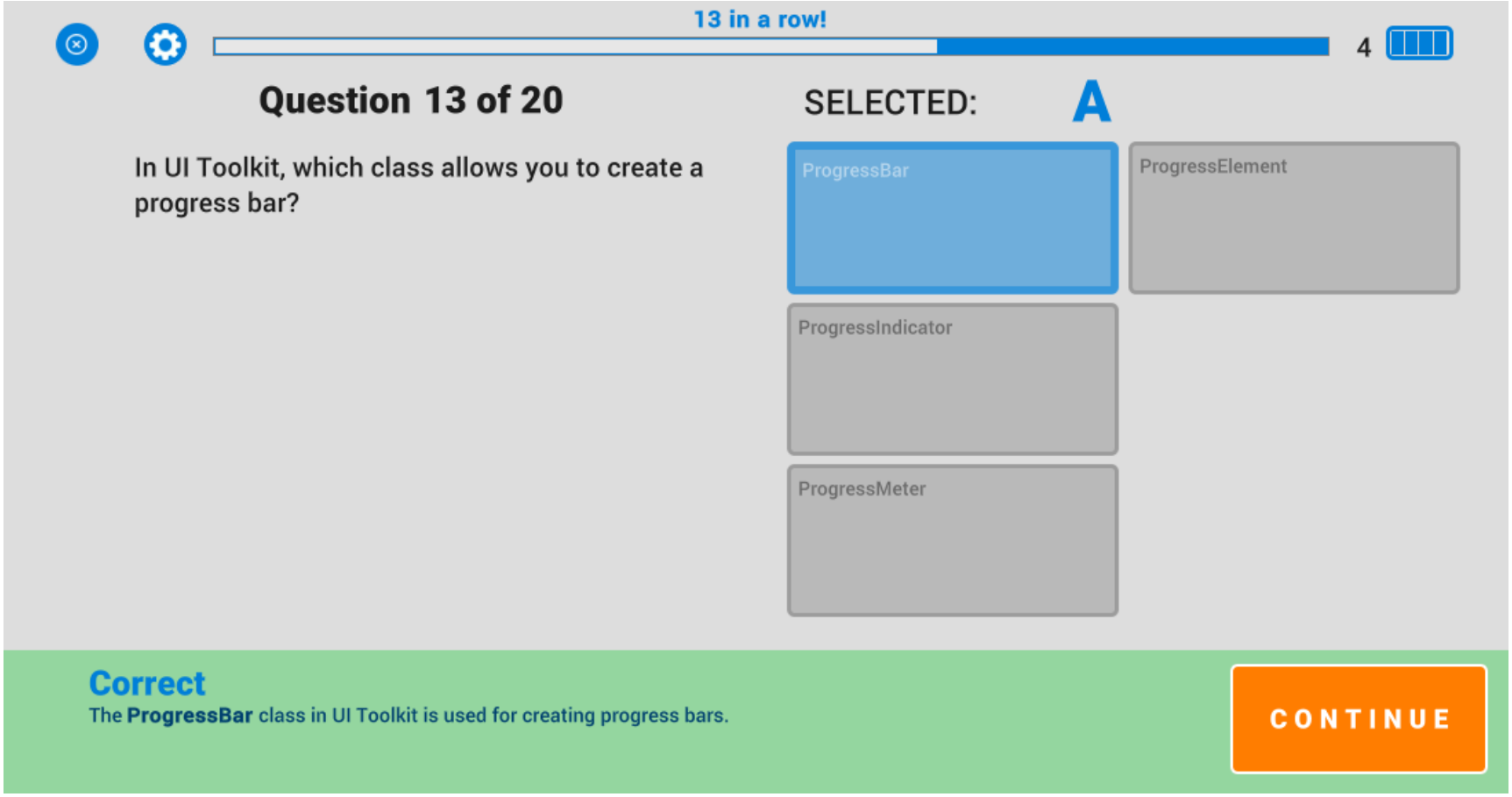
Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit

13 in a row!

4

Question 14 of 20

In UI Toolkit, what is the purpose of **BindableElement**?

SELECT ONE:

- It allows elements to be bound to data sources
- It allows elements to be bound to animations
- It is a base class for all interactive UI elements
- It allows elements to be bound to event handlers

SUBMIT

Quiz → UI Toolkit

14 in a row!

4

Question 14 of 20

SELECTED: A

In UI Toolkit, what is the purpose of **BindableElement**?

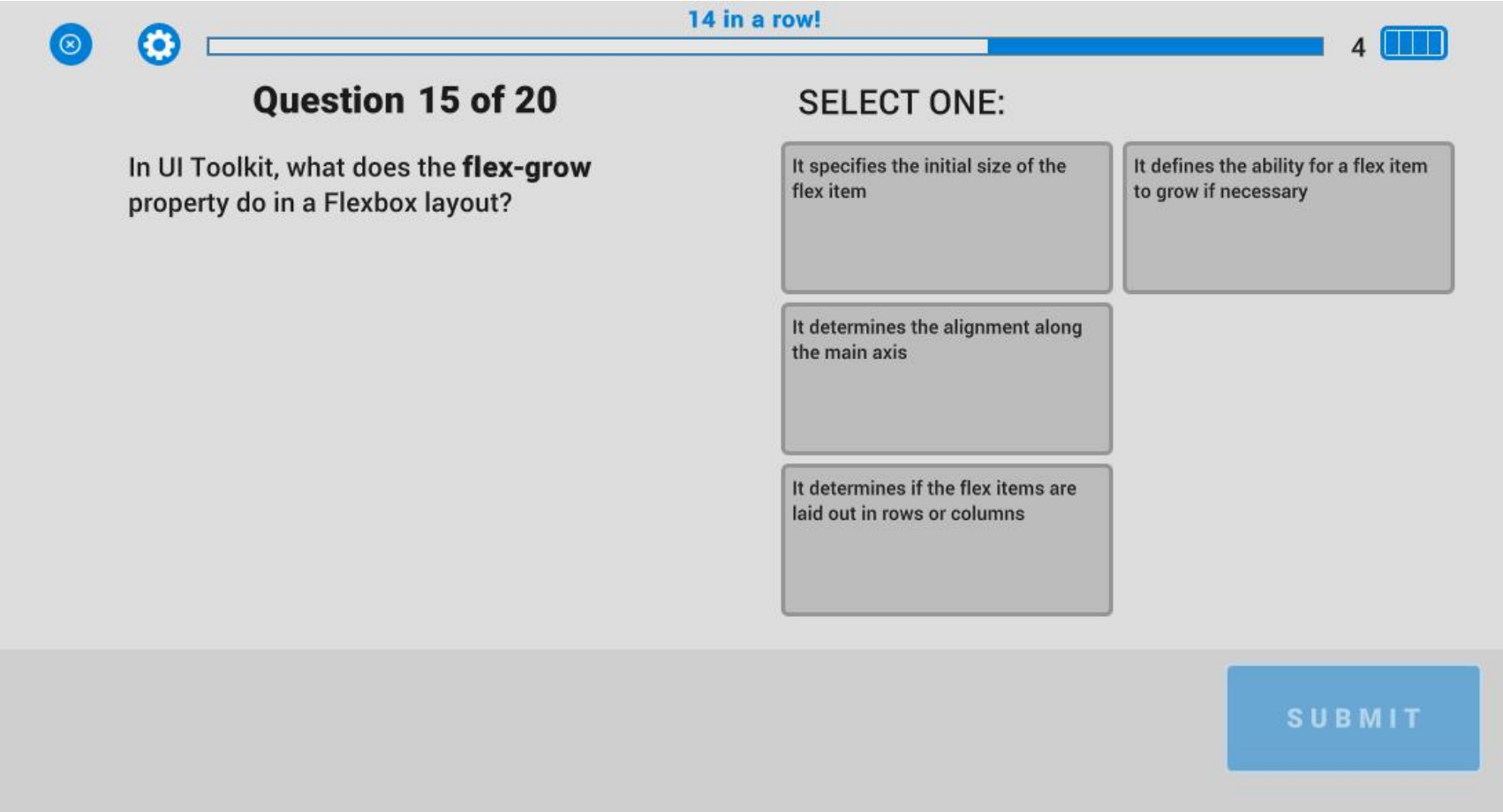
- It allows elements to be bound to data sources
- It allows elements to be bound to animations
- It is a base class for all interactive UI elements
- It allows elements to be bound to event handlers

Correct

The **BindableElement** class in UI Toolkit allows elements to be bound to data sources, such as variables or properties in your scripts, allowing them to automatically update to reflect changes in the underlying data.

CONTINUE

Quiz → UI Toolkit



Quiz → UI Toolkit

15 in a row!

4

Question 15 of 20

SELECTED: D

In UI Toolkit, what does the **flex-grow** property do in a Flexbox layout?

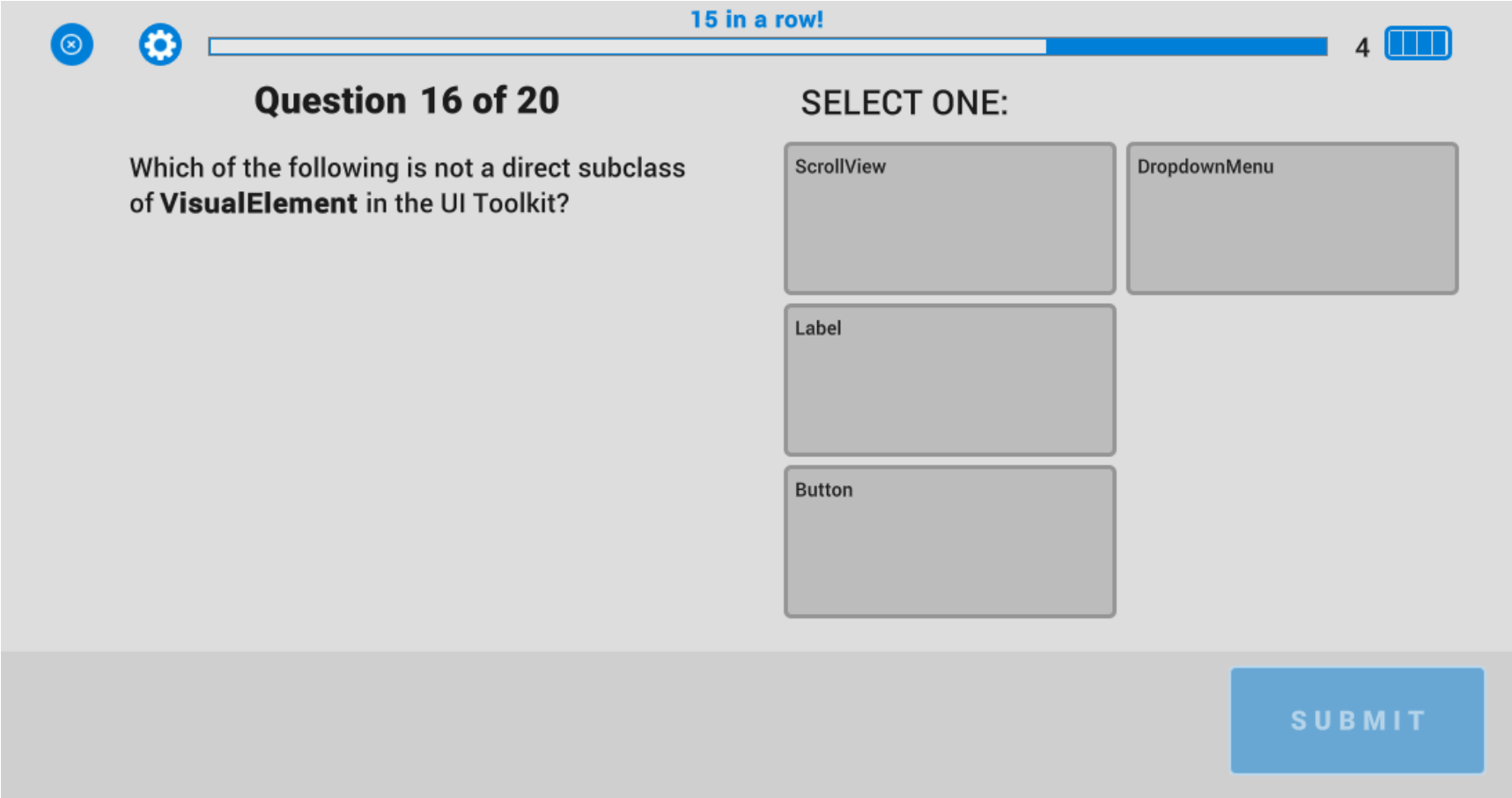
- It specifies the initial size of the flex item
- It defines the ability for a flex item to grow if necessary
- It determines the alignment along the main axis
- It determines if the flex items are laid out in rows or columns

Correct

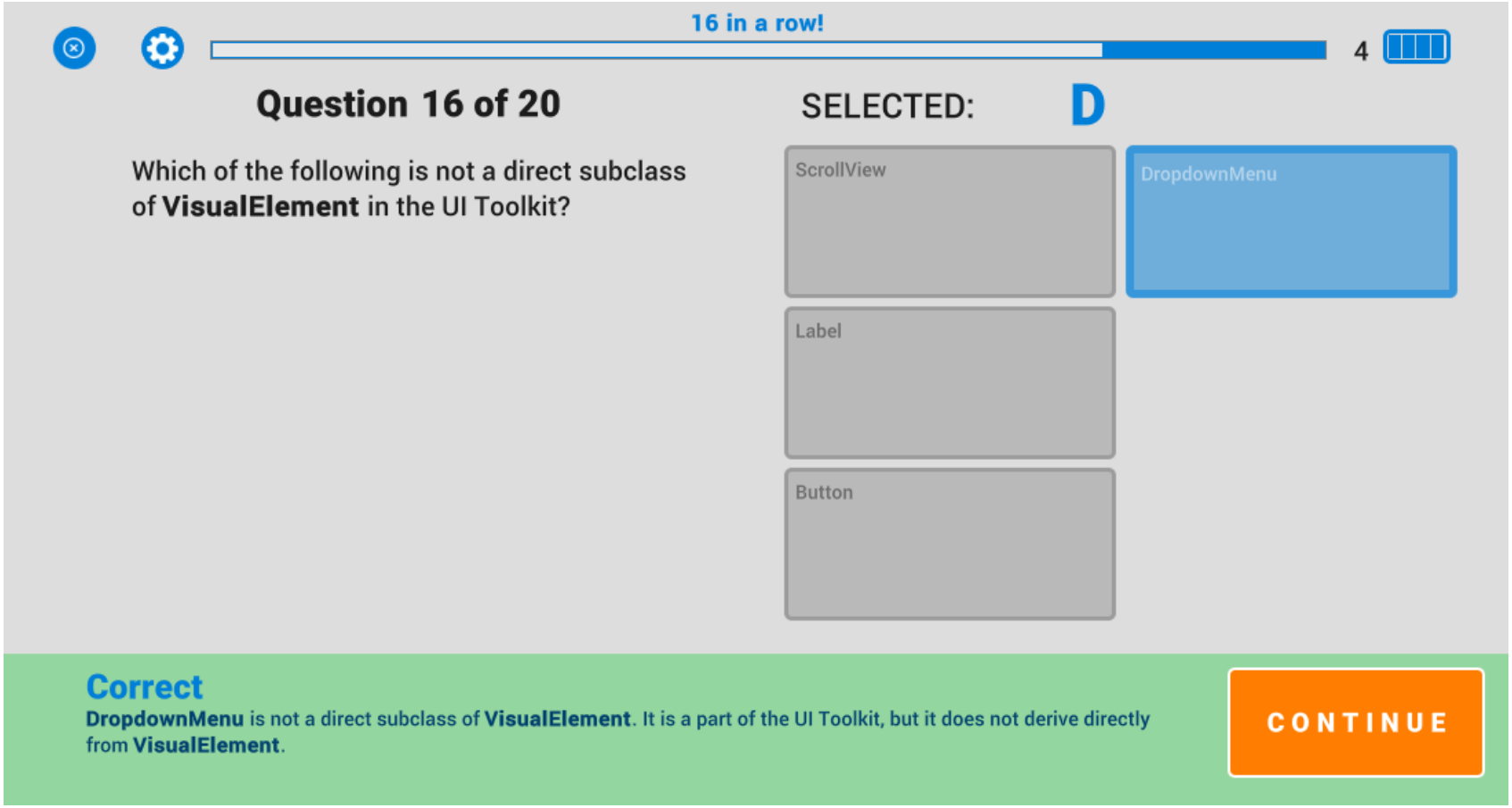
The **flex-grow** property in a Flexbox layout defines the ability for a flex item to grow if necessary. It accepts a unitless value that serves as a proportion, dictating the amount of space inside the flex container the item should take up.

CONTINUE

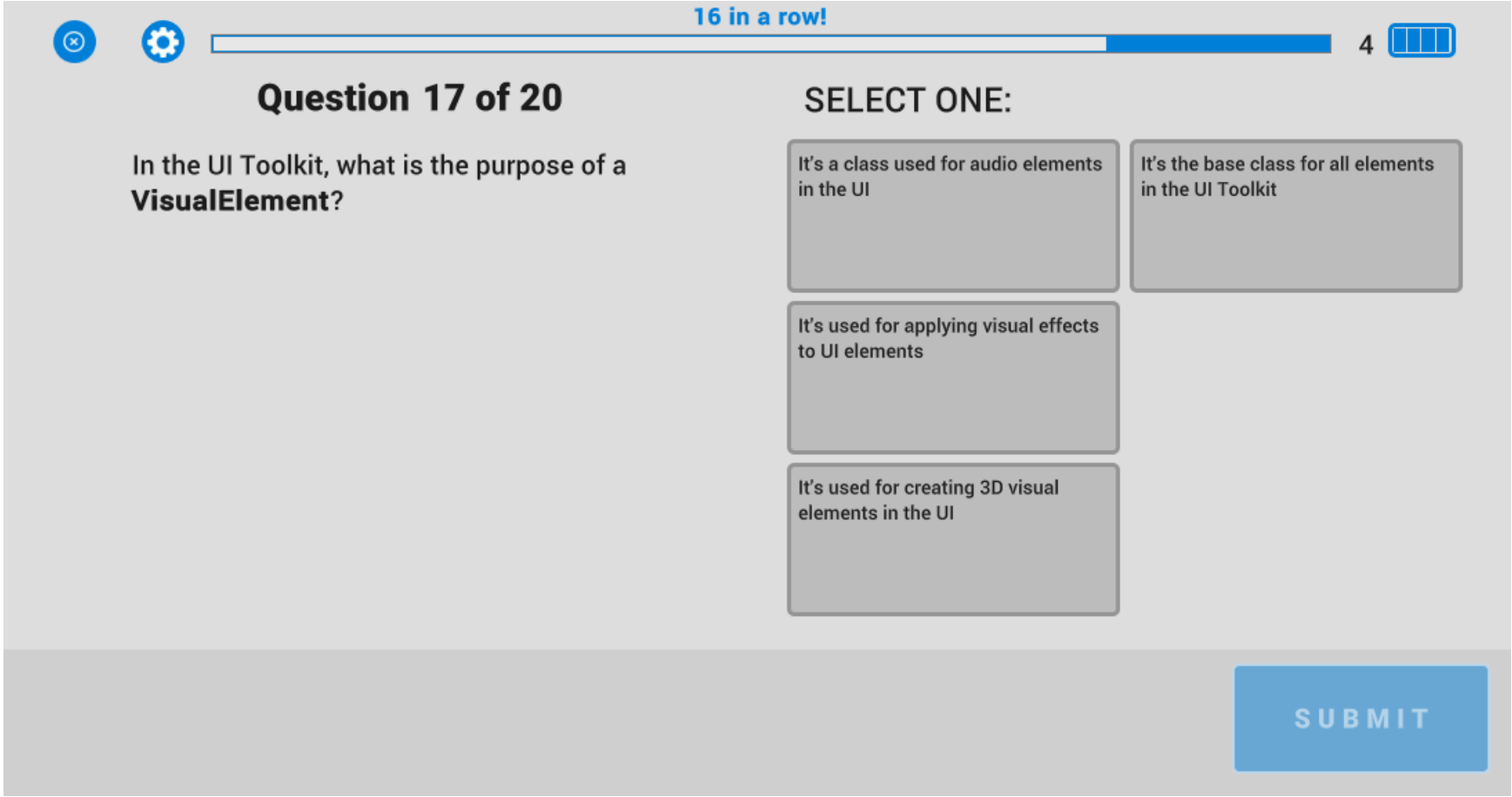
Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit

17 in a row!

4

Question 17 of 20

SELECTED: D

In the UI Toolkit, what is the purpose of a **VisualElement**?

- It's a class used for audio elements in the UI
- It's the base class for all elements in the UI Toolkit
- It's used for applying visual effects to UI elements
- It's used for creating 3D visual elements in the UI

Correct

VisualElement is the base class for all elements in the UI Toolkit. It is the core primitive from which all user interface controls and containers derive.

CONTINUE

Quiz → UI Toolkit

17 in a row!

4

Question 18 of 20

In the UI Toolkit, what is the **UQueryBuilder** used for?

SELECT ONE:

- It is used to manage multi-threading in the UI
- It is used to bind UI elements to data sources
- It is used to construct complex UI animations
- It is used to query the UI hierarchy and find elements

SUBMIT

Quiz → UI Toolkit

18 in a row!

4

Question 18 of 20

SELECTED: C

In the UI Toolkit, what is the **UQueryBuilder** used for?

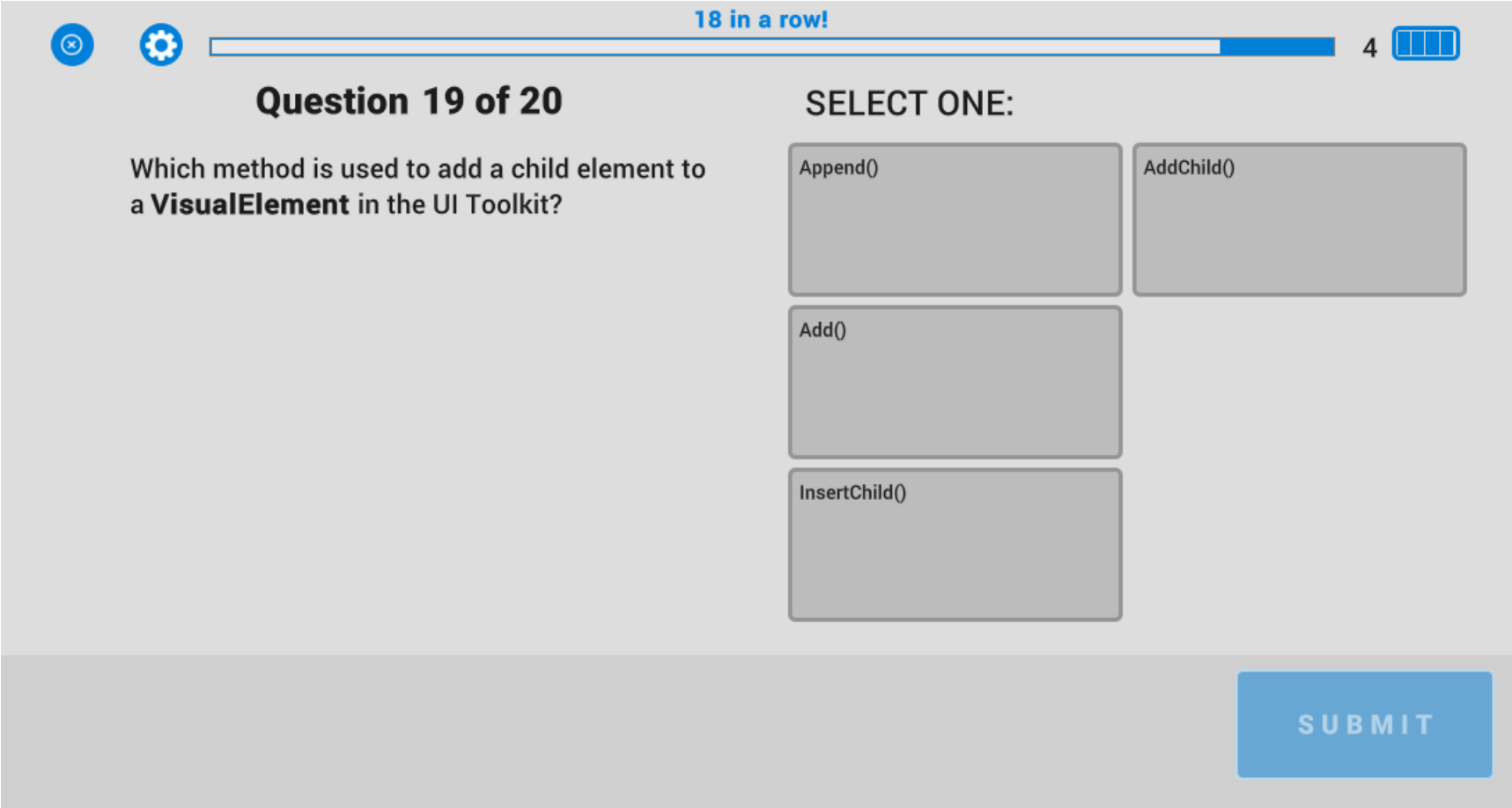
- It is used to manage multi-threading in the UI
- It is used to bind UI elements to data sources
- It is used to construct complex UI animations
- It is used to query the UI hierarchy and find elements

Correct

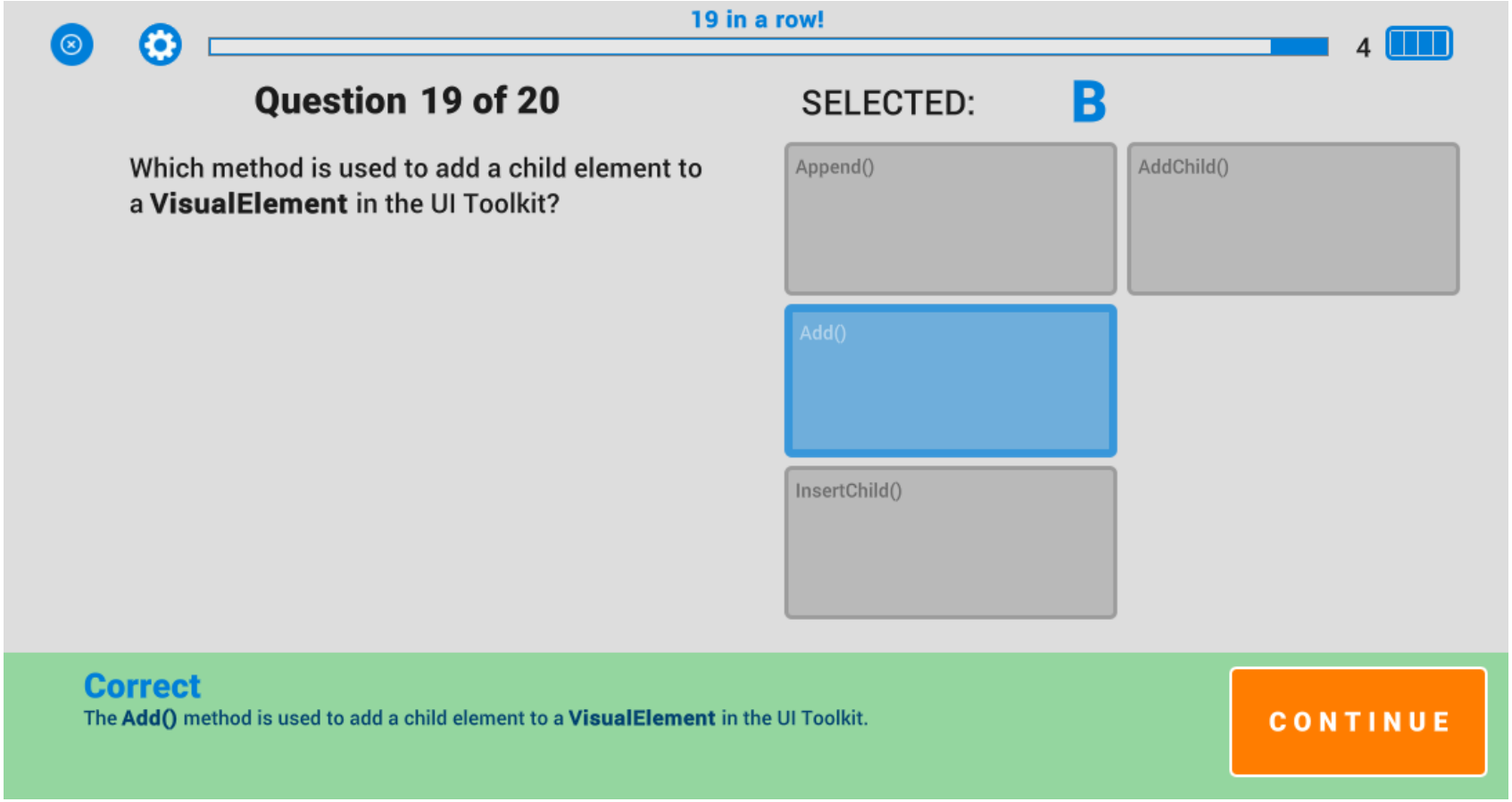
The **UQueryBuilder** is used to query the UI hierarchy and find elements. It allows for complex queries to be constructed to find specific elements.

CONTINUE

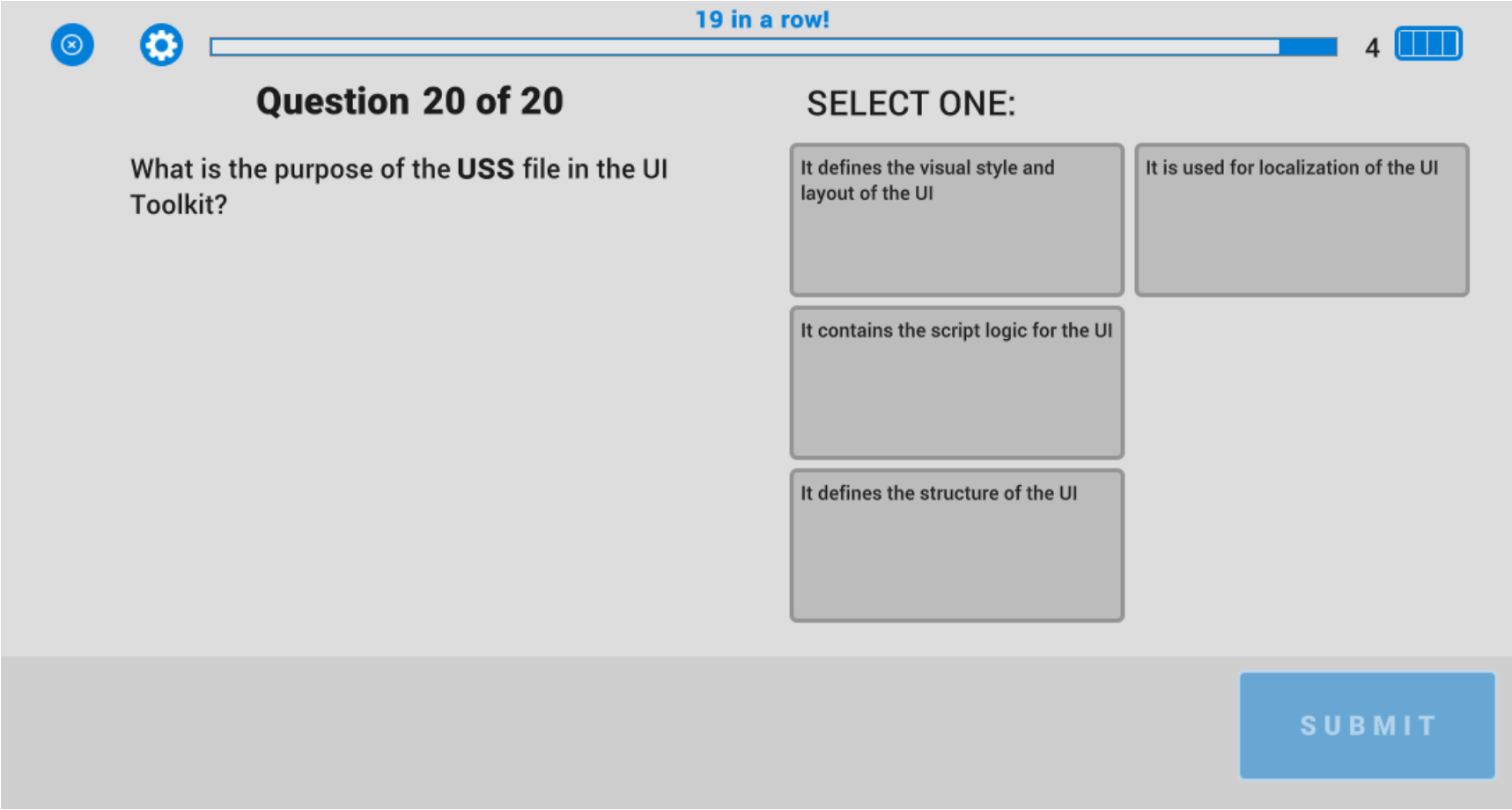
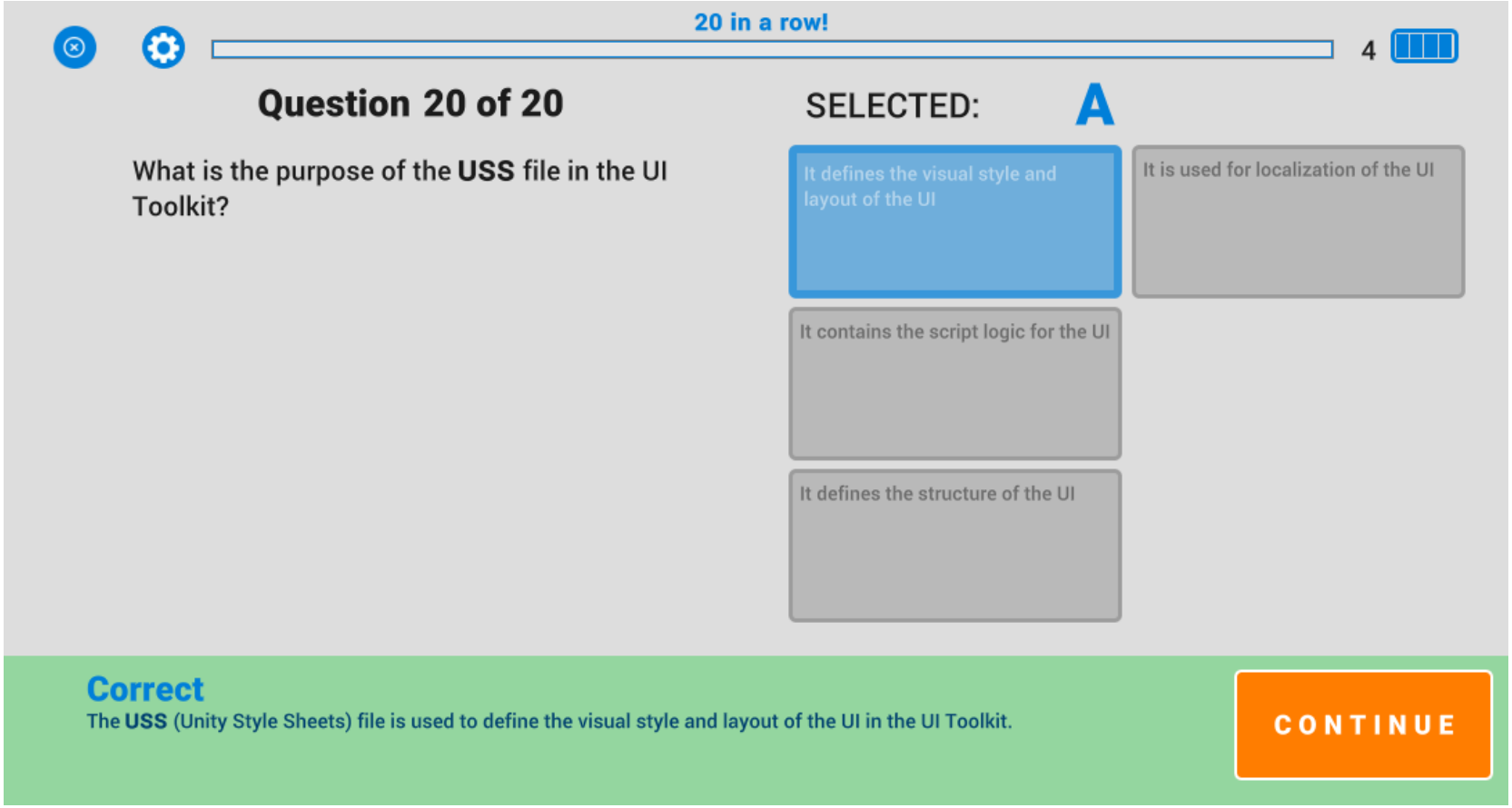
Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit



Quiz → UI Toolkit